

PinIT Python Certification: Lesson Visuals for All 12 Courses

Scope: every lesson part of every course in the Python certification plans:

Plan	Courses included
1 month	course 1
3 months	courses 1-3
6 months	courses 1-6
9 months	courses 1-9
12 months	courses 1-12

12 courses × 30 days × 6 parts = **2,160 lesson parts**.

Builder: Antigravity. **Reviewer:** Claude. **Approver:** the owner.

Version: 1.3, 7 October 2026. It builds on the approved pilot spec
PINIT_PY_M1_VISUALS_DAYS_1_3.pdf (v1.1).

Files that come with this PDF (already in the repo, do not edit):

- docs/visuals/py_cert_generator_prompt.md : the exact AI prompt. SHA-256 40f8f16b2525e42c7f0b483d22d713d09de2863485546f1f4616ab814dbe7a68 .
- docs/visuals/py_cert_tasks.json : every task in this PDF in machine-readable form (ID, allowed files, done check).
- docs/visuals/py_cert_manifest.json : all 2,160 lesson-part keys with their status.
- docs/visuals/py_cert_reference_tracer.py : the proven Python tracer. SHA-256 9be378bd9bd94a43acbda0fb6ba56372012da39aedeae244cfd49b9dd66ccd6e .

Changes in 1.3

#	Change	Where
22	CI took about 26 minutes per task. Tests now run in 4 parallel shards (none skipped), content tasks use a fast path, and builds are cached. Target: 10 minutes or less	new task S-05

Changes in 1.2 (Claude built a working prototype and ran it on all 2,160 real lesson parts)

#	Problem found by the prototype	Fixed in
9	A naive tracer changed the printed output of 374 lessons (Python 3.12 comprehension trap)	C2, reference tracer, E-02
10	One lesson defines a variable <code>time</code> , which crashed a tracer sharing its namespace	C2, E-02

#	Problem found by the prototype	Fixed in
11	Shortest-path lessons use <code>float('inf')</code> , which plain JSON cannot store	C2, E-02
12	9 parts have random or set-order values that differ between runs	R14, E-03
13	Values stored as text could not draw arrays, bars or trees; trees were cut at depth 2	C2 (structured values, depth 4)
14	SQL results have up to 5 columns, but the <code>table</code> template had exactly 2	E-01, R3
15	19 SQL parts and many pilot steps show errors or "Your turn" changes, which v1.1 bindings could not express	error binding, <code>edit</code> (C2)
16	SQL "statement k" did not match what students see. Now: the k-th result table	C2
17	Live bug: <code>AT_ORDER</code> in <code>useLessonEngine.ts</code> knows only <code>say1 - say6</code> , so later courses' pictures would jump back to step 1	E-01
18	<code>askForJson</code> needs a Zod schema, not a JSON file	C3, E-01, E-17
19	Nothing kept pictures "on concept" automatically	R12, Part D teaching hints, prompt rule 5
20	Lesson edits later would silently leave old pictures	<code>codeHash</code> , R13
21	No clean way to add templates or courses later	registry (C3), C8

Changes in 1.1 (Claude checked its own v1.0 plan against the code)

#	Mistake in v1.0	Fixed in
1	Picture steps could only follow spoken lines 1–4, but later courses have up to 14 spoken lines per part	E-01, R4
2	E-21 deleted the pilot file but did not allow the 4 files that import it, so the build would break	E-15, E-21
3	Change & Run ignored that the lesson page runs Python in a worker (<code>public/python-worker.js</code>)	E-22
4	The scope guard checked only the last commit of a push	S-03
5	Claude's own spec updates had no task ID and would be blocked	task <code>X-SPEC</code>
6	The caption-number rule rejected good captions such as "Positions 0 to 3"	R7, prompt rule 5
7	The manifest statuses <code>todo</code> and <code>pilot</code> were missing from R11	C3, R11
8	The generator ignored the existing AI helper <code>askForJson</code> , which can be tested without a key	E-17

Contents

PinIT Python Certification: Lesson Visuals for All 12 Courses

Part A. For the owner (read only this part)

Part B. Antigravity's past mistakes and the rule that now prevents each

Part C. The system (what the engine does)

Part D. Allowed templates per course

Part E. Tasks

Part F. How Claude reviews

Part G. Every task, in order (475 tasks)

Part A. For the owner (read only this part)

A1. What students get

Every lesson part that can be explained with a picture gets one, on the left of the lesson. The picture moves step by step with the teacher's voice.

The numbers and text in the pictures are never typed by anyone. They come from actually running the lesson's code, so a picture can never show a wrong value. When a student changes the code and presses Run, the picture redraws with the student's own values.

A part where no picture would really help gets **no picture**. That is on purpose: a confusing picture is worse than none.

A2. Release order

Milestone	Courses that must be finished	Unlocks
M-1m	1. Python Backend	1-month plan
M-3m	+ 2. DSA in Python, 3. Database Engineering (SQL)	3-month plan
M-6m	+ 4. AI & ML, 5. Distributed Python, 6. Cloud & MLOps	6-month plan
M-9m	+ 7. NLP, 8. Quant Systems, 9. AI Prompt Engineering	9-month plan
M-12m	+ 10. Model Training, 11. Vector Search, 12. AI Safety	12-month plan

The 24-month plan is not part of this work.

A3. Why this plan avoids Antigravity's past mistakes

Antigravity is never asked to *decide* or *type* anything that a test cannot check. Every task is the same short routine:

1. Run one command.
2. Check that one command prints PASS.
3. Commit only the files the task lists.
4. Send the GitHub CI link.

GitHub rejects the commit automatically if it touches any other file.

A4. How you check progress (no coding)

- **GitHub CI summary:** every CI run prints a table like "python 180/180 · dsa-py 74/180 ...". That table is the only progress number that counts.
- **Screenshots:** after every 5 lesson days, CI stores screenshots of 6 randomly chosen pictures. Claude checks them. You may open any of them yourself.
- **Your sign-off:** at each milestone (M-1m ... M-12m), you open 3 lessons of the newly finished courses on your phone and on a laptop, and approve or reject.

A5. Size of the work

Phase	Tasks	What happens
0. Safety	5	New API keys, a secret scanner, the task-scope guard, faster CI
1. Engine	23	Tracer, new picture templates, generator, checks
2. Content	357 day tasks + 72 checkpoints + 12 course releases + 5 plan milestones	One lesson day per task

Part B. Antigravity's past mistakes and the rule that now prevents each

#	Past mistake (seen in this project)	Rule that makes it impossible
1	Invented 18 pictures for the wrong lessons	Pictures are generated from the lesson itself. The gate checks each picture's key and part title against the real lesson file
2	Typed values that did not match the code (seats 450)	No value is ever typed. Every value is a <i>binding</i> to the real run (rule R6)
3	Screenshots of the wrong parts (counted from 0)	Screenshots pick parts by key (<code>python:1:2</code>), never by slide number
4	Claimed "0 failed" and "verified on CI" without proof	Only a GitHub CI link counts. Reports without the link are rejected
5	Changed files outside the task (sandbox, guard)	CI task-scope guard: the commit message names the task, and any file outside that task's allowed list fails CI
6	Weakened or skipped tests	Test files, CI files and guard files are protected. Only tasks that list them may change them
7	Pushed secret API keys	gitleaks secret scan in CI. New keys live only in GitHub secrets
8	Fixed one thing and broke another	Every task runs the full CI (types, unit tests, gate, Playwright) before it counts
9	Long tasks with many judgement calls	Each content task is one lesson day, with one command and one PASS check

Global rules for every task (Antigravity must follow all of them):

- One task per commit.** The commit message starts with `[task:<ID>]` , for example `[task:C-dsa-py-D04]` .
- Only touch the files listed in the task.** Do not "also fix" anything else. If something else is broken, stop and report it.
- Never edit a generated JSON file by hand.** If its check fails, run the generator again. If it still fails after 3 runs, report it.
- If any command fails,** stop and send the exact output. Do not change tests, rules, prompts or CI to make it pass.
- Work only on branch** `main-dis3ku` of `https://github.com/Vinay-N-Kashyap/Present-Career-os` , and push only there.

6. **"Done"** means the **GitHub CI run for that commit is green**. Send that run's link, and nothing else, as proof.
7. **Tasks run in order**. Do not start a task before the one before it is green.

Part C. The system (what the engine does)

C1. Picture templates (11 in total)

The first 5 already exist from the pilot. The other 6 are new, and each is built in its own Phase 1 task.

Template	Draws	Typical use
flow (exists)	2-5 boxes joined by arrows	data pipelines, request paths
boxes (exists)	named boxes, each holding one value	variables, state
table (exists; widened to 2-5 columns in E-01)	rows filled one by one	loop traces, query results
letters (exists)	one cell per character, with a pointer and a range	strings, indexing, slicing
compare (exists)	two panels side by side	before/after, wrong/right
cells (new)	one cell per list item, with index numbers and up to 3 named pointers	arrays, two pointers, windows, binary search
stack-queue (new)	items pushed and popped, stack vertical or queue horizontal	stacks, queues, call stacks, buffers
tree-graph (new)	up to 6 nodes and their edges; visited nodes lit	trees, graphs, tries, heaps
bars (new)	up to 6 labelled bars with their numbers	scores, latency, cost, accuracy
sequence (new)	2-4 actors in columns, with messages going down	client/server, consensus, retries
states (new)	2-5 states, with the current one lit	lifecycles, circuit breakers, locks

All templates keep the pilot rules (spec v1.1):

- plain HTML and CSS, with SVG only for arrows;
- motion only when the step changes (300 ms), never looping;
- `prefers-reduced-motion` turns movement off;
- theme colour tokens only;
- the 4 tones `data`, `ok`, `error`, `idle`;
- at most 6 shapes and 5 steps;
- a one-line caption with `aria-live`;
- the avatar docked in the bottom bar;
- mobile-first layout.

C2. Where the values come from (bindings)

A picture never contains a typed value. The AI writes a **spec** that contains **bindings**. The engine runs the code and turns the spec into the **filled** picture, which has the same shape the pilot templates already draw. The existing template components therefore do not change.

Binding	Meaning
<code>{"var":"total","line":4}</code>	Value of <code>total</code> right after code line 4 runs
<code>{"var":"total","line":4,"hit":3}</code>	Same, the 3rd time line 4 runs (loops)
<code>{"var":"x","line":2,"as":"type"}</code>	The type name of <code>x</code> (<code>int</code> , <code>str</code> ...)
<code>{"out":2}</code>	Output line 2
<code>{"table":2} / {"table":2,"row":1,"col":"price"}</code>	SQL only: the 2nd result table the student sees, or one cell of it
<code>{"error":true}</code>	The name of the error the code stops with (<code>NameError</code> , a PostgreSQL error ...)
<code>{"text":"balance -= 45"}</code>	Text copied exactly from the part's code, say lines, example or trylt

Changed code ("what if"). A step may carry an `edit`: `{"replaceLine":{"line":2,"text":"lunch = 150"}}` or `{"appendLines":["balance += 1000"]}`. Every new line must appear verbatim in the part's trylt, say lines or example. That step's bindings read from the run of the changed code. This keeps what the pilot did (13 uses: "Your turn" results and error demonstrations).

How the code is run. Proven on all 1,980 Python parts; see

`docs/visuals/py_cert_reference_tracer.py` .

- **Python (11 courses):**
- The code runs in Pyodide (Python 3.12) with the reference tracer, under file name `<lesson>` .
- The tracer lives in its **own namespace**, and the lesson gets a clean one. One lesson defines a variable `time` , which crashed a tracer that shared its namespace.
- It reads module variables from `f_globals` , and function variables only through `co_varnames` . Reading `f_locals` inside a 3.12 list comprehension writes `None` into variables, and changed the printed output of **374 lessons** in the naive version.
- Proven results:
 - 1,980 of 1,980 outputs are unchanged under tracing;
 - 50 parts reach the 2,000-event limit (recording stops, the code still finishes);
 - all runs take about 35 seconds in total.
- **SQL (course 3):**
- Each statement runs in PGlite through `splitSqlStatements` and `resetDatabase` (`src/lib/code/sql/sqlCore.ts`), and the result tables the student sees are captured.
- Measured: 170 of 180 parts have a table of at most 6 rows and 5 columns; 19 parts end in a deliberate error.

Values are stored as structured data, not text, so `cells` , `bars` and `tree-graph` can draw lists, numbers and trees:

- depth at most 4, at most 30 items, loops (`a.next.prev`) marked as references;

- `inf` and `nan` stored as `{"__f__": "inf"}`, because lessons such as shortest-path algorithms use `float('inf')`, which plain JSON cannot hold;
- for display, values are shown as Python `repr`, cut at 40 characters with `...`.

Stable values only. Every part is run **twice, in two fresh interpreters**. A variable whose values differ between the runs (random numbers, set order; measured in 9 parts) is marked `unstable` and may not be bound (rule R14). Printed output is already stable, because the lesson tests require it.

C3. File layout

What	Where
Picture data	<code>src/lib/data/lessonVisuals/<prefix>/day-<NN>.json</code> , one file per lesson day
Types and Zod schema	<code>src/lib/types/lessonVisual.ts</code> (types), <code>src/lib/visuals/schema.ts</code> (Zod schema, used by both the generator and the gate)
Template registry	<code>src/lib/visuals/registry.ts</code> : one entry per template (component, fill adapter, Zod schema, gallery sample)
Tracer and SQL capture	<code>src/lib/visuals/trace/</code>
Fill adapters	<code>src/lib/visuals/fill/</code>
Template components	<code>src/app/quests/lesson/components/visuals/</code>
Generator, check and status scripts	<code>scripts/visuals/</code>
Released courses (the only switch that shows pictures to students)	<code>src/lib/visuals/enabledCourses.ts</code>
Progress manifest	<code>docs/visuals/py_cert_manifest.json</code> . Statuses: <code>todo</code> , <code>pilot</code> (python Days 1-3 until E-20), <code>passed</code> , <code>none</code> , <code>needs-review</code>

A day file looks like this:

```
{ "schemaVersion": 1, "prefix", "day", "promptSha", "model", "entries": [6 × { "partTitle",
"codeHash", "spec", "filled" }] }
```

- `codeHash` is the SHA-256 of the part's code. If a lesson's code is ever edited, the gate names that day as stale, and only that day is regenerated.
- `spec` is what the AI wrote (bindings). `filled` is what the page draws.

C4. The generator (what `npm run visuals:generate` does)

For each of the 6 parts of one lesson day:

1. **Trace.** Run the part's code twice with the tracer (or SQL capture). Build a summary that shows which variables change on which lines and how often, marks unstable variables, and hides the values.
2. **Ask the AI.** Send the fixed prompt (`docs/visuals/py_cert_generator_prompt.md`, SHA-256 checked), plus:
 - the part's text, code and output;

- the trace summary;
- the course's allowed templates and teaching hint (Part D);
- the Zod schema as JSON.

Use `askForJson` from `src/lib/server/llmJson.ts`. 3. **Fill and check.** Parse the answer, fill every binding from the real run (running `edit` steps on the changed code), and run the gate (C5). 4. **Retry.** If the gate fails, send the failure message back to the AI and try again, at most 3 attempts. If it still fails, write `{"template":"none","reason":"generator:<last gate error>"}` and mark the key `needs-review`. 5. **Save.** Write the day file and update the manifest.

Configuration:

- The model name lives in `scripts/visuals/config.json`, and the API key comes only from the environment variable `OPENROUTER_API_KEY`.
- With no key, the script stops with a clear message and changes nothing.

C5. The gate (rules checked automatically, in CI, for every day file)

Rule	Check
R1	The file has exactly 6 entries, and each entry's <code>partTitle</code> equals the real part title
R2	The template is <code>none</code> or is on the course's allowed list (Part D)
R3	2–5 steps, at most 6 shapes. A table has 2–5 columns and at most 6 rows
R4	<code>at</code> values are valid for the part (<code>sayN</code> only if the part has at least N say lines) and strictly increasing (<code>say1...sayN</code> , example, <code>tryIt</code>)
R5	Each caption is one sentence of at most 80 characters, ending in <code>.</code> , with no emoji
R6	Every value is a binding. Filling the spec from a fresh run gives exactly <code>filled</code> . Any <code>text</code> binding, and every <code>edit</code> line, appears verbatim in the part
R7	Every number in a caption is one of the values bound in that step, or is written in the part's code
R8	Every tappable label appears as a whole word in the part's text or code
R9	Tones are only <code>data</code> , <code>ok</code> , <code>error</code> , <code>idle</code>
R10	At least 3 of the 6 parts in a day have a picture, otherwise the day is marked <code>needs-review</code>
R11	The manifest status matches the file. A key with a day file is <code>passed</code> , <code>none</code> or <code>needs-review</code> ; a key without one is <code>todo</code> (or <code>pilot</code> for python Days 1–3 before E-20)
R12	On-concept check. Each caption shares at least one important word (4+ letters, not a common word such as "this" or "with") with the text its step is attached to: its say line, the example or the <code>tryIt</code> . Pictures cannot drift away from what the teacher is saying
R13	Fresh check. Each entry's <code>codeHash</code> equals the SHA-256 of the part's current code
R14	Stable values. No binding points to a variable that differs between two fresh runs

C6. Change & Run (students can play with the picture)

When a student edits the code of a part that has a picture and presses Run:

1. The student's code runs in the browser (in the existing Python worker; no server) with the same reference tracer.

2. The picture's spec is filled from the student's run, and the picture redraws.
3. If a binding can't be filled (for example, the student renamed a variable), the picture keeps the lesson's values and shows the line "Your code changed the names this picture uses, so it shows the lesson's values."

For SQL, the same happens with PGlite in the browser.

C7. Release switch

`src/lib/visuals/enabledCourses.ts` lists the course prefixes whose pictures students can see. A course is added only by its release task (L-...), after all its checkpoints are approved. Until then its pictures exist in the repo but are hidden.

C8. Building further (how the system grows without rework)

- **New template:** add one component, one fill adapter, one Zod schema and one gallery sample, plus a `registry.ts` entry. The gate, generator and loader read the registry, so nothing else changes.
- **New Python course:** add its prefix, allowed templates and teaching hint to Part D (`py_cert_tasks.json` and the generator config). Everything else is the same routine.
- **Web track later:** the spec, fill, gate, manifest and templates are language-neutral. Only a JavaScript tracer is needed, built the same way as the Python one.
- **Lesson edited later:** R13 names the stale days in CI, and only those days are regenerated.
- **Better AI later:** change the model in `scripts/visuals/config.json` and regenerate. The `promptSha` and `model` saved in each day file show which pictures came from which version.

Part D. Allowed templates per course

The generator may only pick templates from that course's row (gate rule R2). `none` is always allowed.

#	Prefix	Course	Allowed templates	Teaching hint (sent to the AI)
1	python	Python Backend	flow, boxes, table, letters, compare, cells	show what each line does to variables and output, one line at a time
2	dsa-py	DSA in Python	cells, stack-queue, tree-graph, table, boxes, bars, compare	show the data structure itself (array cells, stack, tree, graph) and the pointers or visited nodes moving step by step
3	sql-mastery	Database Engineering (SQL)	table, flow, compare, bars	show the result table and how the query changes it (which rows are filtered, joined, grouped)
4	ai-py	AI & ML in Python	flow, table, bars, sequence, compare, boxes	show the pipeline (prompt, retrieval, model, answer) and the scores or tokens that change

#	Prefix	Course	Allowed templates	Teaching hint (sent to the AI)
5	dist-py	Distributed Python	sequence, flow, states, table, cells, compare	show messages between services over time, node states, and what happens when one fails
6	cloud-py	Cloud & MLOps Python	flow, sequence, states, table, bars, compare	show the request path through cloud components and the states or costs that change
7	nlp-py	NLP in Python	table, cells, bars, flow, compare	show tokens or words in cells, their counts or scores, and how the text is transformed
8	quant-py	Quant Systems Python	table, bars, flow, sequence, compare, cells	show the order book or price table, and the numbers (prices, risk) that change
9	prompt-py	AI Prompt Engineering	flow, table, compare, bars, sequence	show prompt in, answer out, and what changes when the prompt changes
10	train-py	Model Training Python	bars, table, flow, cells, sequence, compare	show memory, loss or throughput numbers as bars, and how data or work is split across devices
11	vec-py	Vector Search Python	table, bars, tree-graph, cells, flow, compare	show vectors or scores in cells and bars, ranked results, and index structure
12	safe-py	AI Safety Python	flow, table, bars, compare, states	show the guardrail pipeline, what is blocked or allowed, and the scores that decide

Part E. Tasks

How to read a task card:

Field	Meaning
ID	Put this in the commit message as [task:<ID>]
Goal	What the task achieves
Allowed files	The only files the commit may change. Anything else fails CI (after task S-03 is done)
Steps	Do exactly these, in order
Done when	The checks that must pass. Always also: the GitHub CI run for the commit is green
Must not	Things that get the task rejected
Send	What to send to Claude

Phase 0. Safety (5 tasks)

S-01. New API keys (the owner does this)

- **Goal:** the leaked OpenRouter keys stop working, and the new key never enters the code.
- **Allowed files:** none (this is account work).
- **Steps:** 1. At openrouter.ai → Keys, delete every existing key. 2. Create one new key named `pinit-visuals-generator` with a monthly spending limit. 3. In GitHub → repo Settings → Secrets and variables → Actions, add `OPENROUTER_API_KEY` with the new key. 4.

On the computer that runs the generator, put the key only in `.env.local`, which git ignores.

- **Done when:** the owner confirms in writing that the old keys are deleted.
- **Must not:** paste the key into any file, chat or commit.
- **Send:** "S-01 done" (no key).

S-02. Secret scanner in CI

- **Goal:** CI fails if any commit adds a secret.
- **Allowed files:** `.github/workflows/ci-cd.yml`, `.gitleaks.toml`.
- **Steps:** 1. Add a CI step, before `npm ci`, that runs gitleaks (the official `gitleaks/gitleaks-action` or the gitleaks binary) on **the commits of this push only**. Old history already contains revoked keys; do not scan it. 2. In `.gitleaks.toml`, use the default rules. Add no allowlist entries.
- **Done when:** CI shows a step named `Secret scan` with result success.
- **Must not:** allowlist any `sk-or-` pattern, or set `continue-on-error`.
- **Send:** the CI run link.

S-03. Task-scope guard

- **Goal:** a commit can only change the files its task allows.
- **Allowed files:** `scripts/ci/check-task-scope.mjs`, `tests/task_scope.test.ts`, `.github/workflows/ci-cd.yml`.
- **Steps:** 1. Write `scripts/ci/check-task-scope.mjs`. It:
 - reads the head commit message and finds `[task:<ID>]` (missing → fail);
 - loads `docs/visuals/py_cert_tasks.json` and finds that ID (unknown → fail);
 - checks **every commit in the push**, not only the last one: the range is `github.event.before..github.sha` on push, and `base..head` on pull requests;
 - for each commit, lists its changed files (`git diff --name-only <commit>~1 <commit>`);
 - fails if any changed file does not match one of that commit's task `allowedFiles` globs. 2. Commits without `[task:` are allowed only if they touch no file under `src/lib/data/lessonVisuals/`, `scripts/visuals/`, `src/lib/visuals/`, `docs/visuals/py_cert_*`, `tests/` or `.github/`. 3. Add a CI step `Task scope` right after checkout, with `fetch-depth: 0` so the whole push range is available. 4. In `tests/task_scope.test.ts`, use 7 cases with fake file lists:
 - allowed file passes;
 - other file fails;
 - unknown ID fails;
 - missing tag on a protected path fails;
 - missing tag on a normal path passes;
 - a glob match passes;
 - in a push of 2 commits, a bad first commit fails even when the last commit is fine.
- **Done when:** CI is green, and the test shows all 7 cases.
- **Must not:** change `docs/visuals/py_cert_tasks.json` (Claude owns it).
- **Note:** Claude's own spec updates use the task ID `X-SPEC`. It allows `docs/visuals/**`, plus `docs/visuals/py_cert_generator_prompt.md` and `docs/visuals/py_cert_tasks.json` by exact path (S-04 requires exact paths for these two).

- **Send:** the CI run link.

S-04. Protected files list

- **Goal:** tests, CI and guard files can only change in tasks that name them.
- **Allowed files:** `scripts/ci/check-task-scope.mjs`, `tests/task_scope.test.ts`.
- **Steps:** 1. In the guard, treat `tests/**`, `.github/**`, `scripts/ci/**`, `docs/visuals/py_cert_generator_prompt.md` and `docs/visuals/py_cert_tasks.json` as protected. A protected file passes only if the task's `allowedFiles` lists it **by exact path** (a glob is not enough). 2. Add 2 test cases: a protected file listed only through a glob fails; a protected file listed exactly passes.
- **Done when:** CI is green.
- **Send:** the CI run link.

S-05. CI speed: parallel tests, content fast path, caches

- **Why:** each run takes about 26 minutes, 14.5 of them in `npm test` on GitHub's 2-core runner. With 474 tasks, that's about 200 hours of waiting. Nothing may be skipped to save time; the time is saved by running work in parallel.
- **Allowed files:** `.github/workflows/ci-cd.yml`, `scripts/utils/run-tests.cjs`, `scripts/ci/check-task-scope.mjs`, `tests/task_scope.test.ts`, `tests/run_tests_shard.test.ts`.
- **Steps:** 1. **Shards.** In `scripts/utils/run-tests.cjs`, reuse `getTestFiles()`:
 - sort the list;
 - if `TEST_SHARD=N/M` is set, run only the files at positions `N-1`, `N-1+M`, `N-1+2M`, ...;
 - with `--list`, print the files that would run and exit.

With no `TEST_SHARD`, behaviour is exactly as today. 2. **Shard jobs.** In CI, replace the single `npm test` step with a job `tests` that uses a matrix `shard: [1, 2, 3, 4]`. Each shard runs `npm ci`, then `TEST_SHARD=<n>/4 npm test`. (`pretest` still runs, because it builds the sandbox runtimes the tests need.) Each shard uploads its `--list` output as an artifact. 3.

Completeness job. A job `tests-complete` (`needs: tests`) downloads the 4 lists. It fails unless together they contain every file from `node scripts/utils/run-tests.cjs --list` exactly once, and it writes "4 shards ran N of N test files" to the job summary. 4. **Content fast path.** `check-task-scope.mjs` writes `content_only=true` to `$GITHUB_OUTPUT` only when every commit in the push is a content task (`C-...`) and changed only its own `day-NN.json` and the manifest. Then: - the shard jobs and the full Playwright suite are skipped; - a job `content` runs `npm run visuals:check` for that day, the manifest check, the normal build, and the screenshot test for that day's keys.

Before E-18 exists, content tasks cannot happen, so this path is unused. 5. **Caches.** Use `actions/setup-node` with `cache: npm`. Add `actions/cache` for `.next/cache`, keyed on `package-lock.json` and the source files. 6. **Keep everything else:** - task scope and secret scan run first; - `tsc` runs on every push; - both builds stay (the normal build plus the test-mode security check, then the E2E build); - all Playwright tests run on every code commit. 7. **Tests:** - `tests/run_tests_shard.test.ts`: 4 shards of a fake list of 10 files cover all 10 exactly once, and no shard is empty; - `tests/task_scope.test.ts`: `content_only` is true for a pure content commit, and false if the same push also touches any other file. -

Done when: - a code commit's CI is green in at most 10 minutes; - the summary shows "4 shards ran N of N test files" with N equal to the full count; - the total number of passing tests across the shards equals the number from a normal `npm test` run (send both numbers). - **Must not:** - skip, delete, filter or weaken any test; - add `continue-on-error` or `|| true`; - add any runtime test-mode switch (header, cookie, secret). The login bypass exists only in the separate E2E build; - run fewer than 4 shards to save runner minutes. - **Send:** the CI run link, plus the two test counts.

Phase 1. Engine (23 tasks)

E-01. Types, Zod schema, registry, and the spoken-line bug

- **Allowed files:** `src/lib/types/lessonVisual.ts`, `src/lib/visuals/schema.ts`, `src/lib/visuals/registry.ts`, `src/app/quests/lesson/hooks/useLessonEngine.ts`, `src/app/quests/lesson/components/visuals/TableTemplate.tsx`, `tests/visual_types.test.ts`.
- **Steps:** 1. Change `VisualAt` to `'intro' | `say${number}` | 'example' | 'tryIt'`. 2. **Fix a live bug** in `useLessonEngine.ts`: `AT_ORDER` (around line 21) lists only `say1 - say6`, with `example = 7`. A step at `say7 ... say14` gets order 0, so the picture jumps back to step 1. Replace the table with a function: `intro = 0`, `sayN = N`, `example = 1000`, `tryIt = 1001`. 3. Add the `Binding` union and the `edit` field exactly as in C2. Add `VisualSpec` (with bindings) next to the existing `LessonVisual` (filled values; unchanged). 4. Let `table` have 2-5 columns: change `cells: [string, string]` to `cells: string[]`, and make `TableTemplate.tsx` draw 2-5 columns (stacked cards under 480 px, as now). 5. Add the types for the 6 new templates (C1) and the day-file type (C3). 6. Write `schema.ts` with the Zod schemas (`zod` is already used by `askForJson`), and `registry.ts` with one entry per existing template. 7. Tests:
 - `getStepIndexForPieceAt` returns the right step for `say9` in a part with 12 say lines;
 - a 5-column table spec passes the schema;
 - a spec with a typed number in a value slot fails the schema.
- **Done when:** `npx tsc --noEmit` is clean, the existing visual tests still pass, and CI is green.
- **Must not:** use `any`, or change how the 5 existing templates look.

E-02. Python tracer (copy the proven reference)

- **Allowed files:** `src/lib/visuals/trace/pythonTracer.ts`, `src/lib/visuals/trace/runPythonTrace.ts`, `tests/visual_trace.test.ts`.
- **Steps:** 1. `pythonTracer.ts` exports the content of `docs/visuals/py_cert_reference_tracer.py` **byte for byte** as a string. Do not edit the Python code. 2. `runPythonTrace(code)` follows the usage note at the top of that file:
 - `tracer` in its own dict, `lesson` in a second fresh dict;
 - returns `{events, output, error, truncated}`;
 - output is formatted exactly like `runLikeLessonPage` in `tests/python_long_lessons.test.ts` (with `[Error]` ... on the last line).
- 3. Tests:
 - the tracer string's SHA-256 equals the reference file's;
 - Day 2 Part 2 (balance) gives 500, 480 and 435 after lines 1, 3 and 5;
 - **python Day 9 Part 3** (list comprehensions) prints exactly its lesson output (the 3.12 trap);
 - **prompt-py Day 18 Part 1** runs without error (the `time` name clash);

- a value of `float('inf')` is stored as `{"__f__":"inf"}`;
- **dsa-py Day 16 Part 1:** the tree `root` is stored with all 5 nodes;
- an infinite loop returns `truncated: true` and the run still ends;
- **all 1,980 Python lesson parts** print exactly their `output` under tracing (about 35 s).
- **Done when:** the 8 tests pass, and CI is green.

E-03. Value formatter and stability

- **Allowed files:** `src/lib/visuals/trace/formatValue.ts`, `src/lib/visuals/trace/stability.ts`, `tests/visual_trace.test.ts`.
- **Steps:** 1. `formatValue` turns a stored value into display text: Python `repr`, cut at 40 characters with `...`; `{"__f__":"inf"}` shows as `inf`. 2. `findUnstable(eventsA, eventsB)` returns the variables whose values differ between two runs. 3. Tests: `'Tea'`, `20`, `4.5`, `True`, `[1, 2, 3]`, `{'a': 1}`, a 100-character string, `inf`; plus `findUnstable` on python Day 15 Part 2, which must name `roll` (a random dice roll).
- **Done when:** the tests pass, and CI is green.

E-04. SQL capture

- **Allowed files:** `src/lib/visuals/trace/runSqlCapture.ts`, `tests/visual_trace.test.ts`.
- **Steps:** 1. Run a SQL lesson's code exactly like `runSqlLesson` in `src/lib/code/sql/sqlCore.ts` (reuse `splitSqlStatements` and `resetDatabase`; do not write a new runner). 2. Return `{tables: [{columns, rows}], error, output}`, where `tables` are the result tables the student sees, in order. 3. Tests:
 - for all 180 `sql-mastery` parts, the captured `output` equals the lesson `output`;
 - one part that ends in a deliberate error returns that error.
- **Done when:** the tests pass, and CI is green.

E-05. Fill adapters for the 5 existing templates

- **Allowed files:** `src/lib/visuals/fill/*.ts`, `src/lib/visuals/registry.ts`, `tests/visual_fill.test.ts`.
- **Steps:** 1. Write `fill(spec, part)`. It runs the code with E-02 or E-04 (and once more for each `edit` step), and replaces every binding with its value. If a value can't be found, it throws a clear error naming the step and the binding. 2. Write one adapter per existing template: `flow`, `boxes`, `table`, `letters`, `compare`. Register each in `registry.ts`. 3. Test: write specs with bindings for pilot pictures 1.3, 2.1, 2.2 and 3.2. Filling them must give exactly the approved pilot values:
 - 1.3: `NameError` from an `edit`;
 - 2.1: total 185, then 215 from an `edit`;
 - 2.2: balance 500, 480, 435, then 1435 from `appendLines`;
 - 3.2: `IndexError`.
- **Done when:** the tests pass, and CI is green.

E-06. Template gallery page for tests

- **Allowed files:** `src/app/dev/visual-gallery/page.tsx`, `tests/e2e/gallery.spec.ts`.

- **Steps:** 1. Add a page that renders every template, all steps, from sample data. It is visible only when `NEXT_PUBLIC_E2E_TEST_MODE === '1'`; otherwise it returns 404. 2. Playwright checks:
 - at 390 px and 1440 px, every template shows every step;
 - no sideways scroll;
 - an axe contrast check in light and dark mode.
- **Done when:** CI is green, and a normal build returns 404 for the page.

E-07 to E-12. The 6 new templates, one task each

There is one task per template:

Task	Template
E-07	cells
E-08	stack-queue
E-09	tree-graph
E-10	bars
E-11	sequence
E-12	states

- **Allowed files (for template X):**
 - `src/app/quests/lesson/components/visuals/<X>Template.tsx`,
 - `src/app/quests/lesson/components/visuals/index.ts`, `src/lib/visuals/fill/<X>.ts`,
 - `src/app/dev/visual-gallery/page.tsx`, `tests/visual_fill.test.ts`.
- **Steps:** 1. Build the component following C1 and the v1.1 drawing rules. 2. Write its fill adapter. 3. Add sample data to the gallery. 4. Add one fill test using a real lesson part from the course that will use it most:
 - `cells`: dsa-py Day 8;
 - `stack-queue`: dsa-py Day 4;
 - `tree-graph`: dsa-py Day 16;
 - `bars`: vec-py Day 5;
 - `sequence`: dist-py Day 3;
 - `states`: dist-py Day 20.
- **Done when:** the gallery Playwright test (E-06) covers the new template at both widths and in both themes, and CI is green.
- **Must not:** add an animation library, a hex colour or a looping animation.

E-13. Gate v2

- **Allowed files:** `tests/lesson_visuals_gate.test.ts`, `src/lib/visuals/gate.ts`.
- **Steps:** 1. Put rules R1 to R14 (C5) in `src/lib/visuals/gate.ts` as one function: `checkDayFile(file) → {passed, errors[]}`. Keep and reuse the existing `src/lib/visuals/visualRules.ts` (underline and space-dot rules); do not change or replace it. 2. The test runs `checkDayFile` on every file under `src/lib/data/lessonVisuals/` and fails with every error listed. 3. Add one deliberately broken sample per rule (14 samples), and show each one failing.

- **Done when:** CI is green, and the 14 broken samples each fail with the right rule number.

E-14. Manifest check in CI

- **Allowed files:** `scripts/visuals/status.mts`, `package.json`, `.github/workflows/ci-cd.yml`, `tests/visual_manifest.test.ts`.
- **Steps:** 1. `npm run visuals:status` prints one line per course: `prefix passed/none/needs-review/todo`, and writes the same table to the GitHub job summary. 2. The test checks that the manifest has exactly 2,160 keys, that the keys equal the real lesson parts, and that each status matches the day files (R11). 3. Add the status step to CI after the tests.
- **Done when:** the CI job summary shows the table, and CI is green.
- **Must not:** edit `docs/visuals/py_cert_manifest.json` by hand. Only scripts change it.

E-15. Loader and release switch

- **Allowed files:** `src/lib/visuals/loadVisuals.ts`, `src/lib/visuals/enabledCourses.ts`, `src/app/quests/lesson/hooks/useLessonEngine.ts`, `tests/visual_loader.test.ts`.
- **Steps:** 1. Load `src/lib/data/lessonVisuals/<prefix>/day-<NN>.json` only for prefixes listed in `enabledCourses.ts`. That list starts as `['python']`. 2. Move the visual lookup out of `useLessonEngine.ts` (today it imports `PYTHON_M1_VISUALS` directly, around line 518) into `getVisual(prefix, day, partIndex)` in `loadVisuals.ts`. `useLessonEngine.ts` calls only `getVisual`. 3. Until E-21, `getVisual` still returns the pilot file `pythonMonth1Visuals.ts` for python Days 1-3.
- **Done when:** the tests pass (an enabled course loads, a disabled course gets no visuals, a missing file gets no visual and no crash), and CI is green.

E-16. Prompt lock

- **Allowed files:** `tests/visual_prompt_lock.test.ts`.
- **Steps:** the test reads `docs/visuals/py_cert_generator_prompt.md` and asserts its SHA-256 is `40f8f16b2525e42c7f0b483d22d713d09de2863485546f1f4616ab814dbe7a68`.
- **Done when:** CI is green.
- **Must not:** edit the prompt file. Only Claude changes it, with a new version of this PDF.

E-17. Generator script

- **Allowed files:** `scripts/visuals/generate.mts`, `scripts/visuals/config.json`, `package.json`, `tests/visual_generator.test.ts`.
- **Steps:** 1. Implement C4 exactly: trace twice, prompt, parse, fill, gate, up to 3 attempts, write the day file, update the manifest. Call the AI only through the existing `askForJson` in `src/lib/server/llmJson.ts`, with the Zod schema from `src/lib/visuals/schema.ts`; do not write a new AI client or a second schema. 2. The command is `npm run visuals:generate -- --course <prefix> --day <N>`. 3. It reads the key from `OPENROUTER_API_KEY` and stops with "OPENROUTER_API_KEY is not set" if it is missing. 4. It writes only `src/lib/data/lessonVisuals/<prefix>/day-<NN>.json` and `docs/visuals/py_cert_manifest.json`. 5. It logs, per part: attempt number, chosen template and gate result. 6. In `tests/visual_generator.test.ts`, use the existing fake transport (`setLlmJsonTransportForTests`), so no key and no cost. Test:
 - a valid answer is filled and passes;

- an answer that breaks R3 is retried with the gate error;
- 3 bad answers give `none` and `needs-review`;
- a typed value instead of a binding is rejected.
- **Done when:** the 4 tests pass, and CI is green. The real generator is not run in CI (no API spend). The real-AI proof is task E-20.
- **Must not:** let the AI write values (bindings only), or skip the gate.

E-18. Check command

- **Allowed files:** `scripts/visuals/check.mts`, `package.json`.
- **Steps:** 1. `npm run visuals:check -- --course <prefix> --day <N>` runs the gate on that one day file. 2. It prints `PASS <k>/6 (none: <m>)` or `FAIL` followed by every error, and exits with code 1 on `FAIL`.
- **Done when:** CI is green.

E-19. Shot list and random pick

- **Allowed files:** `scripts/visuals/pick.mts`, `tests/e2e/screenshots.spec.ts`, `package.json`, `docs/visuals/shot_keys.txt`.
- **Steps:** 1. `npm run visuals:pick -- --course <prefix> --days <a>--<b>` picks 6 keys with pictures from those days. It uses a fixed seed made from the course and the day range, so Claude can repeat the pick. It writes them to `docs/visuals/shot_keys.txt`. 2. `screenshots.spec.ts` screenshots exactly the keys in that file, at 390 px and 1440 px, in light and dark mode, selecting by `data-visual-key`.
- **Done when:** CI uploads the screenshots for the current list, and CI is green.

E-20. Pilot through the pipeline

- **Allowed files:** `src/lib/data/lessonVisuals/python/day-01.json`, `src/lib/data/lessonVisuals/python/day-02.json`, `src/lib/data/lessonVisuals/python/day-03.json`, `docs/visuals/py_cert_manifest.json`.
- **Steps:** 1. Run the generator for python Days 1, 2 and 3. 2. Run `visuals:check` for each day.
- **Done when:** each day shows `PASS`, and CI is green. Claude then compares the 18 results with the approved pilot and either approves or asks for a prompt change (a Claude task, not Antigravity's).
- **Must not:** edit the generated files.

E-21. Switch python Days 1-3 to the JSON files

- **Allowed files:** `src/lib/visuals/loadVisuals.ts`, `src/lib/data/lessonVisuals/pythonMonth1Visuals.ts` (delete), `tests/lesson_visuals.test.ts`.
- **Run only after Claude approves E-20.**
- **Steps:** 1. Load python Days 1-3 from the JSON files. 2. Delete the old pilot file. 3. Update the 4 files that import it today, so nothing imports it any more:
 - `useLessonEngine.ts` (if E-15 left any import);
 - `tests/lesson_visuals.test.ts`;
 - `tests/visual_rules.test.ts`;

- `tests/visual_templates.test.ts`. 4. Make these tests read the JSON day files through `getVisual` instead. In `tests/lesson_visuals.test.ts`, keep only what the gate test does not cover. 5. Check: `git grep -n "pythonMonth1Visuals\|PYTHON_M1_VISUALS"` returns nothing.
- **Done when:** CI is green, including the existing Playwright tests for python Days 1-3.

E-22. Change & Run

- **Allowed files:** `src/app/quests/lesson/components/visuals/VisualStage.tsx`, `src/app/quests/lesson/hooks/useLessonEngine.ts`, `src/lib/visuals/trace/runPythonTrace.ts`, `tests/e2e/change-and-run.spec.ts`.
- **Steps:** 1. The lesson page runs Python in a Web Worker (`public/python-worker.js`, called by `runPythonInBrowser` in `src/lib/code/python/pythonRunner.ts`). 2. Add an optional `trace: true` to `runPythonInBrowser`. It posts the tracer source from E-02 to the worker, which runs it before the student code and returns the events next to stdout. 3. Implement C6 on top of that, using the fill adapters from E-05.
- **Playwright tests:**
- **python Day 2 Part 1:** change `lunch = 120` to `lunch = 150`, press Run, and the `total` box shows `215`;
- **renaming `lunch`:** the "Your code changed the names..." line appears;
- **python Day 2 Part 2:** change `balance = 500` to `balance = 600`, and the first step shows `600`.
- **Done when:** CI is green.

E-23. Engine review

- **Allowed files:** none.
- **Steps:** send Claude the CI links of tasks E-01 to E-22.
- **Done when:** Claude approves the engine. Phase 2 starts only after this.

Phase 2. Content (one task per lesson day)

Every content task is the same routine. Its card gives only the course, the day, the 6 keys and their titles.

The routine for content task `C-<prefix>-D<NN>`:

1. `git checkout main-dis3ku && git pull`
2. `npm run visuals:generate -- --course <prefix> --day <N>`
3. `npm run visuals:check -- --course <prefix> --day <N>`. It must print `PASS`.
4. `git add src/lib/data/lessonVisuals/<prefix>/day-<NN>.json docs/visuals/py_cert_manifest.json`
5. `git commit -m "[task:C-<prefix>-D<NN>] visuals for <prefix> day <N>"`
6. `git push origin main-dis3ku`

Done when:

- the CI run for the commit is green;
- the CI summary shows this day's 6 keys as `passed` or `none`;

- at least 3 of the 6 have a picture.

If step 3 prints FAIL, or a key is `needs-review`: run step 2 one more time. If it still fails, commit nothing and send Claude the full output of steps 2 and 3.

Must not:

- edit the generated JSON;
- change the prompt, gate, tests or config;
- run more than one day per commit;
- skip a day.

Send: the CI run link.

Checkpoint tasks `R-<prefix>-<a>-<b>` come after every 5 days:

1. Run `npm run visuals:pick -- --course <prefix> --days <a>-<b>`.
2. Commit `docs/visuals/shot_keys.txt` with `[task:R-<prefix>-<a>-<b>]`, and push.
3. Send the CI link. Claude reviews the uploaded screenshots and replies "approved" or names the keys to regenerate.

Release tasks `L-<prefix>` come after all 6 checkpoints of a course are approved:

1. Add the prefix to `src/lib/visuals/enabledCourses.ts`.
2. Commit with `[task:L-<prefix>]`, and push.
3. Send the CI link.

Milestone tasks `M-1m`, `M-3m`, `M-6m`, `M-9m` and `M-12m`: the owner checks 3 lessons of each newly released course on a phone and a laptop, and signs off. Antigravity sends the latest green CI link with the status table.

The full ordered list of every task follows, in Part G. Each content card lists its 6 keys and part titles, so together the cards cover all 2,160 keys.

Part F. How Claude reviews

1. Every task:

- open the CI run link;
- confirm it is for the right commit, that it is green, and that the `Task scope` step passed;
- for content tasks, read the status table.

2. Engine tasks:

- read the diff;
- re-run the tests locally;
- for E-02 to E-05 and E-13, break one value on purpose and confirm the test fails;
- for templates, check the gallery screenshots.

3. **E-20:** compare the 18 generated pilot pictures with the approved pilot, using the same keys, templates and meaning. A different but correct template is acceptable if it passes the 3-second rule.

4. Checkpoints: for each of the 6 screenshots, check:

- the right part;
- the picture matches what the say line teaches;
- it is understandable in 3 seconds;
- nothing is cut off at 390 px;
- it is readable in both themes.

5. Every 30 days:

- spot-check 3 random day files by hand;
- re-run `visuals:check` locally;
- confirm no file under `tests/` or `.github/` changed outside its tasks (`git log --name-only`).

Part G. Every task, in order (475 tasks)

G1. Phase 0 and Phase 1

#	ID	Task
1	X-SPEC	Claude spec updates (Claude only)
2	S-01	New API keys (owner)
3	S-02	Secret scanner in CI
4	S-03	Task-scope guard
5	S-04	Protected files list
6	S-05	CI speed: parallel tests, content fast path, caches
7	E-01	Types, Zod schema, registry, spoken-line bug
8	E-02	Python tracer (copy the proven reference)
9	E-03	Value formatter and stability
10	E-04	SQL capture
11	E-05	Fill adapters for existing templates
12	E-06	Template gallery page for tests
13	E-07	New template: cells
14	E-08	New template: stack-queue
15	E-09	New template: tree-graph
16	E-10	New template: bars
17	E-11	New template: sequence
18	E-12	New template: states
19	E-13	Gate v2
20	E-14	Manifest check in CI
21	E-15	Loader and release switch
22	E-16	Prompt lock

#	ID	Task
23	E-17	Generator script
24	E-18	Check command
25	E-19	Shot list and random pick
26	E-20	Pilot through the pipeline
27	E-21	Switch python Days 1-3 to JSON
28	E-22	Change & Run
29	E-23	Engine review by Claude

G2. Phase 2, course by course

Each content card uses the routine in Part E.

Course 1: Python Backend (Month 1) (python)

(Day 1: Your First Python Program, already done in the pilot; converted by E-20)

(Day 2: Variables and Data Types, already done in the pilot; converted by E-20)

(Day 3: Working With Text, already done in the pilot; converted by E-20)

C-python-D04 · Day 4: Numbers and Maths

- **Command:** `npm run visuals:generate -- --course python --day 4`, then `npm run visuals:check -- --course python --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D04] visuals for python day 4`

Key	Part title
python:4:0	The four basic operators
python:4:1	Floor division and remainder
python:4:2	Percentages: GST and discounts
python:4:3	Rounding money
python:4:4	Useful number tools: abs, min, max, sum
python:4:5	Putting it together: a restaurant bill

C-python-D05 · Day 5: Making Decisions

- **Command:** `npm run visuals:generate -- --course python --day 5`, then `npm run visuals:check -- --course python --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D05] visuals for python day 5`

Key	Part title
python:5:0	if: do something only when a condition is True
python:5:1	else: what to do otherwise
python:5:2	elif: more than two choices
python:5:3	and, or, not: combining conditions
python:5:4	Short one-line choices
python:5:5	Putting it together: a budget checker

R-python-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course python --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-python-1-5] screenshots for python days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-python-D06 · Day 6: Loops: Doing Things Again and Again

- **Command:** `npm run visuals:generate -- --course python --day 6`, then `npm run visuals:check -- --course python --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D06] visuals for python day 6`

Key	Part title
python:6:0	Why we need loops
python:6:1	Looping over text and range()
python:6:2	The accumulator pattern: running totals
python:6:3	while loops
python:6:4	break and continue
python:6:5	Putting it together: a spending report

C-python-D07 · Day 7: Functions: Your Own Tools

- **Command:** `npm run visuals:generate -- --course python --day 7`, then `npm run visuals:check -- --course python --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D07] visuals for python day 7`

Key	Part title
python:7:0	What a function is
python:7:1	Parameters: giving a function information
python:7:2	return: giving back an answer

Key	Part title
python:7:3	Why return is different from print
python:7:4	Default values for parameters
python:7:5	Putting it together: tracker functions

C-python-D08 · Day 8: Lists: Keeping Many Values Together

- **Command:** `npm run visuals:generate -- --course python --day 8`, then `npm run visuals:check -- --course python --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D08] visuals for python day 8`

Key	Part title
python:8:0	Creating a list and reading items
python:8:1	Changing a list: append, insert, remove
python:8:2	Checking and searching: in, count, index
python:8:3	Slicing lists
python:8:4	Copies and the "same list" trap
python:8:5	Putting it together: a list of expenses

C-python-D09 · Day 9: Looping Over Lists and List Comprehensions

- **Command:** `npm run visuals:generate -- --course python --day 9`, then `npm run visuals:check -- --course python --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D09] visuals for python day 9`

Key	Part title
python:9:0	enumerate: items with their positions
python:9:1	Building a new list with a loop
python:9:2	List comprehensions: the one-line version
python:9:3	Filtering with if
python:9:4	When not to use a comprehension
python:9:5	Putting it together: filtered reports

C-python-D10 · Day 10: Dictionaries: Named Details

- **Command:** `npm run visuals:generate -- --course python --day 10`, then `npm run visuals:check -- --course python --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-10.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-python-D10] visuals for python day 10

Key	Part title
python:10:0	What a dictionary is
python:10:1	Adding and changing keys
python:10:2	Missing keys and get()
python:10:3	Looping over a dictionary
python:10:4	Counting with a dictionary
python:10:5	Putting it together: one expense as a dictionary

R-python-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course python --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** [task:R-python-6-10] screenshots for python days 6-10
- **Wait** for Claude's "approved" before the next day task.

C-python-D11 · Day 11: Lists of Dictionaries: Real Data

- **Command:** `npm run visuals:generate -- --course python --day 11`, then `npm run visuals:check -- --course python --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-python-D11] visuals for python day 11

Key	Part title
python:11:0	The shape of real data
python:11:1	Looping over records
python:11:2	Totals and filters with comprehensions
python:11:3	Finding one record
python:11:4	Sorting records
python:11:5	Putting it together: a monthly report

C-python-D12 · Day 12: Tuples and Sets

- **Command:** `npm run visuals:generate -- --course python --day 12`, then `npm run visuals:check -- --course python --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-python-D12] visuals for python day 12

Key	Part title
python:12:0	Tuples: fixed groups of values
python:12:1	Unpacking and returning several values

Key	Part title
python:12:2	Sets: only unique values
python:12:3	Checking membership quickly
python:12:4	Choosing the right container
python:12:5	Putting it together: categories and ranges

C-python-D13 · Day 13: f-strings: Clean, Readable Output

- **Command:** `npm run visuals:generate -- --course python --day 13`, then `npm run visuals:check -- --course python --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D13] visuals for python day 13`

Key	Part title
python:13:0	What an f-string is
python:13:1	Expressions inside the brackets
python:13:2	Formatting numbers: decimals and commas
python:13:3	Lining up columns
python:13:4	Multi-line text and join
python:13:5	Putting it together: a printed receipt

C-python-D14 · Day 14: Errors and try/except

- **Command:** `npm run visuals:generate -- --course python --day 14`, then `npm run visuals:check -- --course python --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D14] visuals for python day 14`

Key	Part title
python:14:0	Reading an error message
python:14:1	try and except
python:14:2	Using the error message
python:14:3	else and finally
python:14:4	Raising your own errors
python:14:5	Putting it together: safe input handling

C-python-D15 · Day 15: Modules and Python's Built-in Library

- **Command:** `npm run visuals:generate -- --course python --day 15`, then `npm run visuals:check -- --course python --day 15`

- **Allowed files:** `src/lib/data/lessonVisuals/python/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D15] visuals for python day 15`

Key	Part title
python:15:0	What a module is and how to import it
python:15:1	The random module
python:15:2	Dates with datetime
python:15:3	Other useful modules
python:15:4	Your own modules
python:15:5	Putting it together: due dates for bills

R-python-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course python --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-python-11-15] screenshots for python days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-python-D16 · Day 16: Working With Files

- **Command:** `npm run visuals:generate -- --course python --day 16`, then `npm run visuals:check -- --course python --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D16] visuals for python day 16`

Key	Part title
python:16:0	Why programs need files
python:16:1	The with statement
python:16:2	Appending and reading line by line
python:16:3	Turning lines into data with split
python:16:4	Saving and loading a list of records
python:16:5	Putting it together: a tracker that remembers

C-python-D17 · Day 17: JSON: Saving Structured Data

- **Command:** `npm run visuals:generate -- --course python --day 17`, then `npm run visuals:check -- --course python --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D17] visuals for python day 17`

Key	Part title
python:17:0	What JSON is
python:17:1	loads: from JSON text back to Python
python:17:2	Saving and loading JSON files
python:17:3	Handling broken or unexpected data
python:17:4	Dates and other values JSON cannot store
python:17:5	Putting it together: a JSON-backed tracker

C-python-D18 · Day 18: Classes and Objects

- **Command:** `npm run visuals:generate -- --course python --day 18`, then `npm run visuals:check -- --course python --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D18] visuals for python day 18`

Key	Part title
python:18:0	What classes and objects are
python:18:1	init and self : giving objects their data
python:18:2	Methods: actions that belong to an object
python:18:3	Methods that change the object
python:18:4	Many objects in a list
python:18:5	Putting it together: an ExpenseBook class

C-python-D19 · Day 19: Better Classes: **str** and Inheritance

- **Command:** `npm run visuals:generate -- --course python --day 19`, then `npm run visuals:check -- --course python --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D19] visuals for python day 19`

Key	Part title
python:19:0	Making objects print nicely with str
python:19:1	Inheritance: building on an existing class
python:19:2	Overriding methods
python:19:3	<code>super()</code> : reusing the parent's work
python:19:4	When to use classes and inheritance
python:19:5	Putting it together: expenses and subscriptions

C-python-D20 · Day 20: Python on Your Laptop: **Scripts**, **input()** and **pip**

- **Command:** `npm run visuals:generate -- --course python --day 20`, then `npm run visuals:check -- --course python --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D20] visuals for python day 20`

Key	Part title
python:20:0	Installing Python and VS Code
python:20:1	Running a .py file from the terminal
python:20:2	Reading what the user types with input()
python:20:3	A menu loop
python:20:4	pip and packages
python:20:5	Virtual environments

R-python-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course python --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-python-16-20] screenshots for python days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-python-D21 · Day 21: Testing Your Code with assert and pytest

- **Command:** `npm run visuals:generate -- --course python --day 21`, then `npm run visuals:check -- --course python --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D21] visuals for python day 21`

Key	Part title
python:21:0	Why developers write tests
python:21:1	assert and failure messages
python:21:2	Test functions and pytest
python:21:3	Normal cases and edge cases
python:21:4	Testing errors and keeping functions testable
python:21:5	Putting it together: tests for the tracker

C-python-D22 · Day 22: Git and GitHub: Saving and Sharing Your Code

- **Command:** `npm run visuals:generate -- --course python --day 22`, then `npm run visuals:check -- --course python --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D22] visuals for python day 22`

Key	Part title
python:22:0	What Git is and why every job uses it
python:22:1	The basic cycle: status, add, commit
python:22:2	.gitignore: files Git should skip
python:22:3	Branches: working safely on a feature
python:22:4	Pushing to GitHub
python:22:5	Putting it together: your project on GitHub

C-python-D23 · Day 23: Planning Your Expense Tracker

- **Command:** `npm run visuals:generate -- --course python --day 23`, then `npm run visuals:check -- --course python --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D23] visuals for python day 23`

Key	Part title
python:23:0	Why plan before coding
python:23:1	User stories
python:23:2	Designing the data shape
python:23:3	Breaking the program into small functions
python:23:4	Planning the build order
python:23:5	Putting it together: PLAN.md and the first functions

C-python-D24 · Day 24: Project Build 1: Adding and Listing Expenses

- **Command:** `npm run visuals:generate -- --course python --day 24`, then `npm run visuals:check -- --course python --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D24] visuals for python day 24`

Key	Part title
python:24:0	Project structure
python:24:1	<code>add_expense</code> with validation
python:24:2	Listing expenses neatly
python:24:3	The menu in <code>main.py</code>
python:24:4	Testing what you built
python:24:5	Putting it together: today's working program

C-python-D25 · Day 25: Project Build 2: Summaries and Saving

- **Command:** `npm run visuals:generate -- --course python --day 25`, then `npm run visuals:check -- --course python --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D25] visuals for python day 25`

Key	Part title
python:25:0	A summary of spending
python:25:1	Totals per category
python:25:2	Filtering by month
python:25:3	Saving and loading with JSON
python:25:4	Connecting it all in the menu
python:25:5	Putting it together: all must-haves done

R-python-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course python --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-python-21-25] screenshots for python days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-python-D26 · Day 26: Calling Web APIs

- **Command:** `npm run visuals:generate -- --course python --day 26`, then `npm run visuals:check -- --course python --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D26] visuals for python day 26`

Key	Part title
python:26:0	What a web API is
python:26:1	URLs, parameters and status codes
python:26:2	Calling an API with requests
python:26:3	Reading nested JSON safely
python:26:4	API keys and handling failures
python:26:5	Putting it together: expenses in another currency

C-python-D27 · Day 27: Building Your Own API with FastAPI

- **Command:** `npm run visuals:generate -- --course python --day 27`, then `npm run visuals:check -- --course python --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D27] visuals for python day 27`

Key	Part title
python:27:0	From a terminal menu to a web API
python:27:1	Your first FastAPI app
python:27:2	GET routes that reuse tracker.py
python:27:3	POST routes and data checking with Pydantic models
python:27:4	Trying and testing your API
python:27:5	Putting it together: the Expense Tracker API

C-python-D28 · Day 28: Debugging: Reading Tracebacks

- **Command:** `npm run visuals:generate -- --course python --day 28`, then `npm run visuals:check -- --course python --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D28] visuals for python day 28`

Key	Part title
python:28:0	Debugging is a normal part of the job
python:28:1	Reading a traceback, from the bottom up
python:28:2	The most common errors and their usual causes
python:28:3	<code>print()</code> debugging
python:28:4	Breakpoints and the VS Code debugger
python:28:5	Putting it together: fixing a real bug

C-python-D29 · Day 29: Putting Your API Online

- **Command:** `npm run visuals:generate -- --course python --day 29`, then `npm run visuals:check -- --course python --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D29] visuals for python day 29`

Key	Part title
python:29:0	What deploying means
python:29:1	Getting the project ready
python:29:2	Environment variables for settings and secrets
python:29:3	Deploying and checking it works
python:29:4	A README that sells your project
python:29:5	Putting it together: your deployment checklist

C-python-D30 · Day 30: Interview Practice and Your Next Steps

- **Command:** `npm run visuals:generate -- --course python --day 30`, then `npm run visuals:check -- --course python --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/python/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-python-D30] visuals for python day 30`

Key	Part title
python:30:0	What junior Python interviews look like
python:30:1	Common Python questions
python:30:2	Explaining your project in two minutes
python:30:3	Solving a coding task out loud
python:30:4	More classic small problems
python:30:5	Your next steps after this month

R-python-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course python --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-python-26-30] screenshots for python days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-python · Release: students can see Python Backend (Month 1) pictures

- Add 'python' to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-python] release visuals for python`.

M-1m · Milestone: the 1m plan has visuals in every course

- **The owner** opens 3 lessons of each newly released course on a phone and on a laptop, and signs off.
- **Antigravity** sends the latest green CI link with the status table.

Course 2: DSA in Python (Month 2) (dsa-py)

C-dsa-py-D01 · Day 1: Time & Space Complexity (Big-O Asymptotics & Dominant Terms)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 1`, then `npm run visuals:check -- --course dsa-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D01] visuals for dsa-py day 1`

Key	Part title
dsa-py:1:0	Why speed depends on the size of the input
dsa-py:1:1	Big-O: keep the part that grows fastest

Key	Part title
dsa-py:1:2	O(1) and O(N): constant and linear time
dsa-py:1:3	O(log N): halving the problem
dsa-py:1:4	O(N ²) and how to spot it
dsa-py:1:5	Space complexity: memory counts too

C-dsa-py-D02 · Day 2: Dynamic Arrays & Amortized Geometric Resizing

- **Command:** `npm run visuals:generate -- --course dsa-py --day 2`, then `npm run visuals:check -- --course dsa-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D02] visuals for dsa-py day 2`

Key	Part title
dsa-py:2:0	An array: items side by side in memory
dsa-py:2:1	Growing by doubling
dsa-py:2:2	Amortized O(1): the average cost
dsa-py:2:3	Changing a list in place with a write pointer
dsa-py:2:4	List operations and their costs
dsa-py:2:5	Lists of lists and the copy trap

C-dsa-py-D03 · Day 3: Singly & Doubly Linked Lists & Pointer Node Manipulation

- **Command:** `npm run visuals:generate -- --course dsa-py --day 3`, then `npm run visuals:check -- --course dsa-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D03] visuals for dsa-py day 3`

Key	Part title
dsa-py:3:0	Nodes that point to the next node
dsa-py:3:1	Walking, counting and finding
dsa-py:3:2	Reversing a list in place
dsa-py:3:3	Fast and slow pointers
dsa-py:3:4	Doubly linked lists and when to use linked lists
dsa-py:3:5	Deleting nodes with a dummy head

C-dsa-py-D04 · Day 4: Stacks (LIFO): Valid Parentheses & Monotonic Next Greater Element

- **Command:** `npm run visuals:generate -- --course dsa-py --day 4`, then `npm run visuals:check -- --course dsa-py --day 4`

- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D04] visuals for dsa-py day 4`

Key	Part title
dsa-py:4:0	A stack: last in, first out
dsa-py:4:1	Matching brackets with a stack
dsa-py:4:2	The next greater element
dsa-py:4:3	What "monotonic" means
dsa-py:4:4	A min stack: O(1) minimum
dsa-py:4:5	A calculator built on a stack

C-dsa-py-D05 · Day 5: ★ MILESTONE 1: Production LRU Cache Engine (Doubly Linked List + Hash Map)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 5`, then `npm run visuals:check -- --course dsa-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D05] visuals for dsa-py day 5`

Key	Part title
dsa-py:5:0	What a cache is and why it needs a limit
dsa-py:5:1	Two structures, two jobs
dsa-py:5:2	Building get and put
dsa-py:5:3	The OrderedDict shortcut
dsa-py:5:4	Testing your cache like an engineer
dsa-py:5:5	Measuring a cache: hit rate

R-dsa-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course dsa-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dsa-py-1-5] screenshots for dsa-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-dsa-py-D06 · Day 6: Queues (FIFO), Circular Ring Buffers & Deques

- **Command:** `npm run visuals:generate -- --course dsa-py --day 6`, then `npm run visuals:check -- --course dsa-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D06] visuals for dsa-py day 6`

Key	Part title
dsa-py:6:0	A queue: first in, first out
dsa-py:6:1	Why deque beats a list for queues
dsa-py:6:2	A ring buffer: a queue of fixed size
dsa-py:6:3	deque with a maximum length
dsa-py:6:4	Building a stack from a queue
dsa-py:6:5	Queues in real systems

C-dsa-py-D07 · Day 7: Hash Tables, Collision Resolution & Load Factors

- **Command:** `npm run visuals:generate -- --course dsa-py --day 7`, then `npm run visuals:check -- --course dsa-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D07] visuals for dsa-py day 7`

Key	Part title
dsa-py:7:0	Hashing: turning a key into a position
dsa-py:7:1	Collisions and chaining
dsa-py:7:2	Load factor and resizing
dsa-py:7:3	Counting with a dict
dsa-py:7:4	Two sum with a hash map
dsa-py:7:5	Sets and what can be a key

C-dsa-py-D08 · Day 8: Two Pointers Technique (Opposite Direction & Fast/Slow Pointers)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 8`, then `npm run visuals:check -- --course dsa-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D08] visuals for dsa-py day 8`

Key	Part title
dsa-py:8:0	Two pointers from both ends
dsa-py:8:1	Container with the most water
dsa-py:8:2	Checking a palindrome
dsa-py:8:3	Removing duplicates with same-direction pointers
dsa-py:8:4	Three sum: two pointers inside a loop
dsa-py:8:5	Choosing between two pointers and a hash map

C-dsa-py-D09 · Day 9: Sliding Window Technique (Fixed vs Dynamic Windows)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 9`, then `npm run visuals:check -- --course dsa-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D09] visuals for dsa-py day 9`

Key	Part title
dsa-py:9:0	What a sliding window is
dsa-py:9:1	Fixed window: the best sum of k in a row
dsa-py:9:2	Flexible window: longest stretch with no repeats
dsa-py:9:3	Flexible window: shortest stretch reaching a total
dsa-py:9:4	Counting letters in a window
dsa-py:9:5	Spotting a window problem

C-dsa-py-D10 · Day 10: Binary Search Algorithm & Monotonic Search Space Reduction

- **Command:** `npm run visuals:generate -- --course dsa-py --day 10`, then `npm run visuals:check -- --course dsa-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D10] visuals for dsa-py day 10`

Key	Part title
dsa-py:10:0	Binary search on a sorted list
dsa-py:10:1	The off-by-one traps
dsa-py:10:2	Searching a rotated sorted list
dsa-py:10:3	Finding the smallest in a rotated list
dsa-py:10:4	Binary search on the answer
dsa-py:10:5	Binary search in real life

R-dsa-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course dsa-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dsa-py-6-10] screenshots for dsa-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-dsa-py-D11 · Day 11: Recursion, Call Stack Mechanics & Backtracking Principles

- **Command:** `npm run visuals:generate -- --course dsa-py --day 11`, then `npm run visuals:check -- --course dsa-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-11.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-dsa-py-D11] visuals for dsa-py day 11

Key	Part title
dsa-py:11:0	A function that calls itself
dsa-py:11:1	The call stack
dsa-py:11:2	Recursion on lists and repeated work
dsa-py:11:3	Backtracking: choose, explore, un-choose
dsa-py:11:4	Permutations: every possible order
dsa-py:11:5	Pruning: stopping early

C-dsa-py-D12 · Day 12: Merge Sort & Divide-and-Conquer Recurrences

- **Command:** `npm run visuals:generate -- --course dsa-py --day 12`, then `npm run visuals:check -- --course dsa-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dsa-py-D12] visuals for dsa-py day 12

Key	Part title
dsa-py:12:0	Divide and conquer
dsa-py:12:1	Merging two sorted lists
dsa-py:12:2	Merge sort put together
dsa-py:12:3	Merging sorted linked lists
dsa-py:12:4	Counting inversions with merge sort
dsa-py:12:5	Sorting in practice with sorted() and key

C-dsa-py-D13 · Day 13: Quick Sort & Quick Select (Kth Largest Element in O(N))

- **Command:** `npm run visuals:generate -- --course dsa-py --day 13`, then `npm run visuals:check -- --course dsa-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dsa-py-D13] visuals for dsa-py day 13

Key	Part title
dsa-py:13:0	Partitioning around a pivot
dsa-py:13:1	Quick sort
dsa-py:13:2	The worst case and random pivots
dsa-py:13:3	Quickselect: the k-th largest in O(N)
dsa-py:13:4	heapq and choosing the right tool
dsa-py:13:5	Merge sort or quick sort?

C-dsa-py-D14 · Day 14: Non-Comparison Sorting: Counting Sort & Radix Sort

- **Command:** `npm run visuals:generate -- --course dsa-py --day 14`, then `npm run visuals:check -- --course dsa-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D14] visuals for dsa-py day 14`

Key	Part title
dsa-py:14:0	The O(N log N) wall and how to get round it
dsa-py:14:1	Counting sort
dsa-py:14:2	Sorting 0s, 1s and 2s in one pass
dsa-py:14:3	Stable counting sort for records
dsa-py:14:4	Radix sort: digit by digit
dsa-py:14:5	Choosing a sort

C-dsa-py-D15 · Day 15: ★ MILESTONE 2: High-Throughput Stream Median Finder (Dual Binary Heaps)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 15`, then `npm run visuals:check -- --course dsa-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D15] visuals for dsa-py day 15`

Key	Part title
dsa-py:15:0	Streams and why sorting every time is too slow
dsa-py:15:1	heapq: a min-heap in Python
dsa-py:15:2	Two heaps split the numbers in half
dsa-py:15:3	Adding a number and rebalancing
dsa-py:15:4	Running medians of a stream
dsa-py:15:5	Milestone review: what you built

R-dsa-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course dsa-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dsa-py-11-15] screenshots for dsa-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-dsa-py-D16 · Day 16: Binary Trees: Preorder, Inorder, Postorder & Level-Order BFS

- **Command:** `npm run visuals:generate -- --course dsa-py --day 16`, then `npm run visuals:check -- --course dsa-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-16.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-dsa-py-D16] visuals for dsa-py day 16

Key	Part title
dsa-py:16:0	Trees: nodes with children
dsa-py:16:1	Depth-first walks: preorder, inorder, postorder
dsa-py:16:2	Level order with a queue
dsa-py:16:3	Maximum depth
dsa-py:16:4	Walking a tree with your own stack
dsa-py:16:5	Trees from lists and tree problems in general

C-dsa-py-D17 · Day 17: Binary Search Trees (BST): Tree Invariants & Range Query Search

- **Command:** `npm run visuals:generate -- --course dsa-py --day 17`, then `npm run visuals:check -- --course dsa-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dsa-py-D17] visuals for dsa-py day 17

Key	Part title
dsa-py:17:0	The binary search tree rule
dsa-py:17:1	Searching and the cost of balance
dsa-py:17:2	Validating a BST with ranges
dsa-py:17:3	Lowest common ancestor in a BST
dsa-py:17:4	The k-th smallest and range queries
dsa-py:17:5	Deleting from a BST

C-dsa-py-D18 · Day 18: Min/Max Binary Heaps & Priority Queues

- **Command:** `npm run visuals:generate -- --course dsa-py --day 18`, then `npm run visuals:check -- --course dsa-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dsa-py-D18] visuals for dsa-py day 18

Key	Part title
dsa-py:18:0	A heap stored in a list
dsa-py:18:1	Push with sift up
dsa-py:18:2	Pop with sift down
dsa-py:18:3	heapq and the k-th smallest
dsa-py:18:4	Priority queues for tasks
dsa-py:18:5	Merging k sorted lists with a heap

C-dsa-py-D19 · Day 19: Tries (Prefix Trees) & Fast Prefix Auto-Complete

- **Command:** `npm run visuals:generate -- --course dsa-py --day 19`, then `npm run visuals:check -- --course dsa-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D19] visuals for dsa-py day 19`

Key	Part title
dsa-py:19:0	A tree of letters
dsa-py:19:1	insert, search and starts_with
dsa-py:19:2	Listing words with a prefix
dsa-py:19:3	Counting words and prefixes
dsa-py:19:4	Tries versus sets and sorted lists
dsa-py:19:5	A word-search game with a trie

C-dsa-py-D20 · Day 20: Graph Representations (Adjacency List/Matrix) & BFS/DFS

- **Command:** `npm run visuals:generate -- --course dsa-py --day 20`, then `npm run visuals:check -- --course dsa-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D20] visuals for dsa-py day 20`

Key	Part title
dsa-py:20:0	Graphs and how to store them
dsa-py:20:1	BFS: shortest path in steps
dsa-py:20:2	DFS: going deep
dsa-py:20:3	Counting connected components
dsa-py:20:4	Grids are graphs too
dsa-py:20:5	Choosing BFS or DFS and planning a graph solution

R-dsa-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course dsa-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dsa-py-16-20] screenshots for dsa-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-dsa-py-D21 · Day 21: ★ MILESTONE 3: Fast Auto-Complete Engine (Trie + Frequency Min-Heap)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 21`, then `npm run visuals:check -- --course dsa-py --day 21`

- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-21.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D21] visuals for dsa-py day 21`

Key	Part title
dsa-py:21:0	What an auto-complete engine must do
dsa-py:21:1	Storing frequencies in the trie
dsa-py:21:2	Collecting the matches below a prefix
dsa-py:21:3	Picking the top k with a heap
dsa-py:21:4	The complete AutocompleteSystem
dsa-py:21:5	Making suggestions fast at scale

C-dsa-py-D22 · Day 22: Dijkstra's Shortest Path Algorithm & Weighted Graphs

- **Command:** `npm run visuals:generate -- --course dsa-py --day 22`, then `npm run visuals:check -- --course dsa-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-22.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D22] visuals for dsa-py day 22`

Key	Part title
dsa-py:22:0	Weighted graphs
dsa-py:22:1	The idea: always settle the closest node
dsa-py:22:2	Dijkstra with heapq
dsa-py:22:3	Recovering the actual route
dsa-py:22:4	Network delay time
dsa-py:22:5	Why negative weights break Dijkstra

C-dsa-py-D23 · Day 23: Topological Sort (Kahn's In-Degree Algorithm) & DAGs

- **Command:** `npm run visuals:generate -- --course dsa-py --day 23`, then `npm run visuals:check -- --course dsa-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-23.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D23] visuals for dsa-py day 23`

Key	Part title
dsa-py:23:0	Dependencies as a directed graph
dsa-py:23:1	In-degree: counting what you still need
dsa-py:23:2	Kahn's algorithm
dsa-py:23:3	Can all courses be finished?
dsa-py:23:4	Build systems and parallel steps
dsa-py:23:5	Choosing the right graph tool

C-dsa-py-D24 · Day 24: Disjoint Set Union (Union-Find) with Path Compression

- **Command:** `npm run visuals:generate -- --course dsa-py --day 24`, then `npm run visuals:check -- --course dsa-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D24] visuals for dsa-py day 24`

Key	Part title
dsa-py:24:0	Groups that merge over time
dsa-py:24:1	Path compression
dsa-py:24:2	Union by rank and the UnionFind class
dsa-py:24:3	Finding the redundant connection
dsa-py:24:4	Kruskal's minimum spanning tree
dsa-py:24:5	Union-find or BFS?

C-dsa-py-D25 · Day 25: Dynamic Programming: 1D Memoization vs Tabulation

- **Command:** `npm run visuals:generate -- --course dsa-py --day 25`, then `npm run visuals:check -- --course dsa-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D25] visuals for dsa-py day 25`

Key	Part title
dsa-py:25:0	What dynamic programming is
dsa-py:25:1	Memoization: top-down DP
dsa-py:25:2	Tabulation: bottom-up DP
dsa-py:25:3	Climbing stairs
dsa-py:25:4	House robber: choose or skip
dsa-py:25:5	A recipe for DP problems

R-dsa-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course dsa-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dsa-py-21-25] screenshots for dsa-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-dsa-py-D26 · Day 26: ★ MILESTONE 4: 0/1 Knapsack & Coin Change Optimization Engine

- **Command:** `npm run visuals:generate -- --course dsa-py --day 26`, then `npm run visuals:check -- --course dsa-py --day 26`

- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-26.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D26] visuals for dsa-py day 26`

Key	Part title
<code>dsa-py:26:0</code>	When greedy choices fail
<code>dsa-py:26:1</code>	Coin change with a DP table
<code>dsa-py:26:2</code>	Counting the ways instead
<code>dsa-py:26:3</code>	The 0/1 knapsack problem
<code>dsa-py:26:4</code>	Which items were taken?
<code>dsa-py:26:5</code>	Milestone 4: an optimisation engine

C-dsa-py-D27 · Day 27: 2D Dynamic Programming: Longest Common Subsequence & Edit Distance

- **Command:** `npm run visuals:generate -- --course dsa-py --day 27`, then `npm run visuals:check -- --course dsa-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-27.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D27] visuals for dsa-py day 27`

Key	Part title
<code>dsa-py:27:0</code>	Comparing two strings
<code>dsa-py:27:1</code>	The LCS recurrence
<code>dsa-py:27:2</code>	Rebuilding the common letters
<code>dsa-py:27:3</code>	Edit distance
<code>dsa-py:27:4</code>	A tiny spell checker
<code>dsa-py:27:5</code>	Spotting 2D DP problems

C-dsa-py-D28 · Day 28: Backtracking: N-Queens & Constraint Satisfaction

- **Command:** `npm run visuals:generate -- --course dsa-py --day 28`, then `npm run visuals:check -- --course dsa-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-28.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D28] visuals for dsa-py day 28`

Key	Part title
<code>dsa-py:28:0</code>	Choose, explore, un-choose
<code>dsa-py:28:1</code>	Pruning: stop early
<code>dsa-py:28:2</code>	N-Queens
<code>dsa-py:28:3</code>	Printing a board
<code>dsa-py:28:4</code>	Sudoku validation with sets

Key	Part title
dsa-py:28:5	When to reach for backtracking

C-dsa-py-D29 · Day 29: Bit Manipulation & XOR Tricks (O(1) Space Magic)

- **Command:** `npm run visuals:generate -- --course dsa-py --day 29`, then `npm run visuals:check -- --course dsa-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D29] visuals for dsa-py day 29`

Key	Part title
dsa-py:29:0	Numbers in binary
dsa-py:29:1	The bitwise operators
dsa-py:29:2	XOR magic: single number
dsa-py:29:3	Counting 1 bits
dsa-py:29:4	Bitmasks as tiny sets
dsa-py:29:5	Using bits wisely

C-dsa-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Real-Time Global Flight Path Routing & Navigation Optimizer

- **Command:** `npm run visuals:generate -- --course dsa-py --day 30`, then `npm run visuals:check -- --course dsa-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/dsa-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dsa-py-D30] visuals for dsa-py day 30`

Key	Part title
dsa-py:30:0	The capstone problem
dsa-py:30:1	Bellman-Ford limited to $k + 1$ rounds
dsa-py:30:2	A heap-based alternative
dsa-py:30:3	Auditing the network
dsa-py:30:4	Putting the engine together
dsa-py:30:5	Looking back, and next steps

R-dsa-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course dsa-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dsa-py-26-30] screenshots for dsa-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-dsa-py · Release: students can see DSA in Python (Month 2) pictures

- Add 'dsa-py' to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-dsa-py] release visuals for dsa-py`.

Course 3: Database Engineering / SQL (Month 3) (`sql-mastery`)

C-sql-mastery-D01 · Day 1: What a Database Is: Tables, Rows and Your First SELECT

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 1`, then `npm run visuals:check -- --course sql-mastery --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D01] visuals for sql-mastery day 1`

Key	Part title
<code>sql-mastery:1:0</code>	What a database is, and why apps need one
<code>sql-mastery:1:1</code>	Tables, rows and columns
<code>sql-mastery:1:2</code>	Primary keys: one unique id for every row
<code>sql-mastery:1:3</code>	SELECT and FROM: reading data
<code>sql-mastery:1:4</code>	The rules of writing SQL
<code>sql-mastery:1:5</code>	Putting it together: a first look at the shop

C-sql-mastery-D02 · Day 2: Creating Tables: Data Types and Keys

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 2`, then `npm run visuals:check -- --course sql-mastery --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D02] visuals for sql-mastery day 2`

Key	Part title
<code>sql-mastery:2:0</code>	CREATE TABLE: designing the shape of your data
<code>sql-mastery:2:1</code>	Choosing data types
<code>sql-mastery:2:2</code>	NOT NULL and DEFAULT
<code>sql-mastery:2:3</code>	Primary keys and serial ids
<code>sql-mastery:2:4</code>	UNIQUE, and removing tables with DROP TABLE
<code>sql-mastery:2:5</code>	Putting it together: the expenses table

C-sql-mastery-D03 · Day 3: Adding, Changing and Deleting Rows

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 3`, then `npm run visuals:check -- --course sql-mastery --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D03] visuals for sql-mastery day 3`

Key	Part title
sql-mastery:3:0	INSERT: adding rows
sql-mastery:3:1	RETURNING: seeing what you just added
sql-mastery:3:2	UPDATE: changing rows
sql-mastery:3:3	DELETE: removing rows
sql-mastery:3:4	The most dangerous mistake: forgetting WHERE
sql-mastery:3:5	Putting it together: keeping the shop up to date

C-sql-mastery-D04 · Day 4: Filtering Rows with WHERE

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 4`, then `npm run visuals:check -- --course sql-mastery --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D04] visuals for sql-mastery day 4`

Key	Part title
sql-mastery:4:0	WHERE and comparisons
sql-mastery:4:1	AND, OR and NOT
sql-mastery:4:2	NULL: the unknown value
sql-mastery:4:3	Filtering dates
sql-mastery:4:4	WHERE with UPDATE and DELETE
sql-mastery:4:5	Putting it together: answering shop questions

C-sql-mastery-D05 · Day 5: Searching Text and Ranges: LIKE, IN, BETWEEN

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 5`, then `npm run visuals:check -- --course sql-mastery --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D05] visuals for sql-mastery day 5`

Key	Part title
sql-mastery:5:0	LIKE: searching inside text
sql-mastery:5:1	ILIKE: search without worrying about capitals
sql-mastery:5:2	IN: matching a list of values
sql-mastery:5:3	BETWEEN: ranges of numbers and dates
sql-mastery:5:4	Combining text, list and range filters
sql-mastery:5:5	Putting it together: a product search

R-sql-mastery-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course sql-mastery --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-sql-mastery-1-5] screenshots for sql-mastery days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-sql-mastery-D06 · Day 6: Sorting and Pages: ORDER BY, LIMIT, OFFSET

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 6`, then `npm run visuals:check -- --course sql-mastery --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D06] visuals for sql-mastery day 6`

Key	Part title
sql-mastery:6:0	Why order is never guaranteed
sql-mastery:6:1	ORDER BY: ascending and descending
sql-mastery:6:2	Sorting by more than one column
sql-mastery:6:3	LIMIT: the top few rows
sql-mastery:6:4	OFFSET: pages of results
sql-mastery:6:5	Putting it together: a catalogue with pages

C-sql-mastery-D07 · Day 7: Useful Functions for Text, Numbers and Dates

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 7`, then `npm run visuals:check -- --course sql-mastery --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D07] visuals for sql-mastery day 7`

Key	Part title
sql-mastery:7:0	What a function is in SQL
sql-mastery:7:1	Text functions
sql-mastery:7:2	Numbers: maths and rounding
sql-mastery:7:3	Dates: today, differences and parts
sql-mastery:7:4	Formatting dates and numbers for people
sql-mastery:7:5	Putting it together: an invoice view of products

C-sql-mastery-D08 · Day 8: Counting and Totals: COUNT, SUM, AVG, MIN, MAX

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 8`, then `npm run visuals:check -- --course sql-mastery --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D08] visuals for sql-mastery day 8`

Key	Part title
sql-mastery:8:0	Aggregate functions: many rows, one answer
sql-mastery:8:1	COUNT(*) and COUNT(column)
sql-mastery:8:2	SUM and AVG, and how they treat NULL
sql-mastery:8:3	MIN and MAX on numbers, text and dates
sql-mastery:8:4	Aggregates with WHERE and calculations
sql-mastery:8:5	Putting it together: the catalogue in numbers

C-sql-mastery-D09 · Day 9: Groups: GROUP BY and HAVING

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 9`, then `npm run visuals:check -- --course sql-mastery --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D09] visuals for sql-mastery day 9`

Key	Part title
sql-mastery:9:0	GROUP BY: one summary per group
sql-mastery:9:1	The grouping rule
sql-mastery:9:2	Grouping by more than one column, and by calculations
sql-mastery:9:3	HAVING: filtering groups
sql-mastery:9:4	Common grouping mistakes
sql-mastery:9:5	Putting it together: a category report

C-sql-mastery-D10 · Day 10: Linking Tables: Relationships and Foreign Keys

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 10`, then `npm run visuals:check -- --course sql-mastery --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D10] visuals for sql-mastery day 10`

Key	Part title
sql-mastery:10:0	Why data is split into several tables
sql-mastery:10:1	One-to-many relationships
sql-mastery:10:2	FOREIGN KEY: letting PostgreSQL protect the link
sql-mastery:10:3	Many-to-many relationships
sql-mastery:10:4	Reading a database design
sql-mastery:10:5	Putting it together: adding reviews to the shop

R-sql-mastery-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course sql-mastery --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-sql-mastery-6-10] screenshots for sql-mastery days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-sql-mastery-D11 · Day 11: Joining Tables: INNER JOIN

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 11`, then `npm run visuals:check -- --course sql-mastery --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D11] visuals for sql-mastery day 11`

Key	Part title
sql-mastery:11:0	What a join does
sql-mastery:11:1	Table aliases: shorter joins
sql-mastery:11:2	INNER JOIN keeps only matching rows
sql-mastery:11:3	Joining and then filtering, sorting and totalling
sql-mastery:11:4	Joining a junction table
sql-mastery:11:5	Putting it together: an orders report

C-sql-mastery-D12 · Day 12: Keeping Rows Without a Match: LEFT JOIN

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 12`, then `npm run visuals:check -- --course sql-mastery --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D12] visuals for sql-mastery day 12`

Key	Part title
sql-mastery:12:0	LEFT JOIN: keep every row from the first table
sql-mastery:12:1	Counting zero correctly
sql-mastery:12:2	Finding rows with no match
sql-mastery:12:3	The WHERE trap with LEFT JOIN
sql-mastery:12:4	RIGHT JOIN and FULL JOIN
sql-mastery:12:5	Putting it together: a customer activity report

C-sql-mastery-D13 · Day 13: Joining Many Tables, and a Table to Itself

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 13`, then `npm run visuals:check -- --course sql-mastery --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D13] visuals for sql-mastery day 13`

Key	Part title
sql-mastery:13:0	Chaining joins step by step
sql-mastery:13:1	Filtering and grouping across many tables
sql-mastery:13:2	Self joins: a table joined to itself
sql-mastery:13:3	More uses of self joins
sql-mastery:13:4	Choosing the right join at each step
sql-mastery:13:5	Putting it together: who bought what

C-sql-mastery-D14 · Day 14: Combining Results: UNION, INTERSECT, EXCEPT

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 14`, then `npm run visuals:check -- --course sql-mastery --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D14] visuals for sql-mastery day 14`

Key	Part title
sql-mastery:14:0	UNION: stacking results
sql-mastery:14:1	UNION ALL: keep every row
sql-mastery:14:2	The rules for combining queries
sql-mastery:14:3	INTERSECT and EXCEPT: comparing lists
sql-mastery:14:4	UNION with joins and totals
sql-mastery:14:5	Putting it together: where the shop is

C-sql-mastery-D15 · Day 15: Queries Inside Queries: Subqueries

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 15`, then `npm run visuals:check -- --course sql-mastery --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D15] visuals for sql-mastery day 15`

Key	Part title
sql-mastery:15:0	A subquery that returns one value
sql-mastery:15:1	IN with a subquery
sql-mastery:15:2	Subqueries in FROM
sql-mastery:15:3	EXISTS and correlated subqueries
sql-mastery:15:4	Subqueries in SELECT, and when to use a join instead
sql-mastery:15:5	Putting it together: finding the most valuable customers

R-sql-mastery-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course sql-mastery --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-sql-mastery-11-15] screenshots for sql-mastery days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-sql-mastery-D16 · Day 16: Readable Queries with WITH

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 16`, then `npm run visuals:check -- --course sql-mastery --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D16] visuals for sql-mastery day 16`

Key	Part title
sql-mastery:16:0	Why long queries need names
sql-mastery:16:1	Several steps, one after another
sql-mastery:16:2	WITH versus subqueries
sql-mastery:16:3	Recursive WITH: counting and sequences
sql-mastery:16:4	Recursive WITH: walking a hierarchy
sql-mastery:16:5	Putting it together: a monthly report in steps

C-sql-mastery-D17 · Day 17: Ranking Rows: Window Functions

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 17`, then `npm run visuals:check -- --course sql-mastery --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D17] visuals for sql-mastery day 17`

Key	Part title
sql-mastery:17:0	What a window function is
sql-mastery:17:1	ROW_NUMBER, RANK and DENSE_RANK
sql-mastery:17:2	PARTITION BY: ranking inside each group
sql-mastery:17:3	The top item per group
sql-mastery:17:4	Comparing with the previous row: LAG and LEAD
sql-mastery:17:5	Putting it together: top products per category

C-sql-mastery-D18 · Day 18: Running Totals and Moving Averages

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 18`, then `npm run visuals:check -- --course sql-mastery --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D18] visuals for sql-mastery day 18`

Key	Part title
sql-mastery:18:0	Running totals with SUM OVER
sql-mastery:18:1	Frames: which rows the window covers
sql-mastery:18:2	Moving averages
sql-mastery:18:3	Running totals per group
sql-mastery:18:4	Period-over-period growth
sql-mastery:18:5	Putting it together: a sales trend report

C-sql-mastery-D19 · Day 19: Decisions Inside a Query: CASE

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 19`, then `npm run visuals:check -- --course sql-mastery --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D19] visuals for sql-mastery day 19`

Key	Part title
sql-mastery:19:0	CASE: if-then-else inside SQL
sql-mastery:19:1	Simple CASE and cleaning codes into words
sql-mastery:19:2	CASE in ORDER BY: custom sort orders
sql-mastery:19:3	Counting by condition
sql-mastery:19:4	Pivot-style reports
sql-mastery:19:5	Putting it together: a product dashboard

C-sql-mastery-D20 · Day 20: Designing Good Tables: Normalization

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 20`, then `npm run visuals:check -- --course sql-mastery --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D20] visuals for sql-mastery day 20`

Key	Part title
sql-mastery:20:0	What goes wrong with repeated data
sql-mastery:20:1	First normal form: one value per cell
sql-mastery:20:2	Second normal form: facts about the whole key
sql-mastery:20:3	Third normal form: no facts about other facts
sql-mastery:20:4	When to break the rules: denormalization
sql-mastery:20:5	Putting it together: splitting a flat table

R-sql-mastery-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course sql-mastery --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-sql-mastery-16-20] screenshots for sql-mastery days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-sql-mastery-D21 · Day 21: Constraints That Protect Your Data

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 21`, then `npm run visuals:check -- --course sql-mastery --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D21] visuals for sql-mastery day 21`

Key	Part title
sql-mastery:21:0	Why rules belong in the database
sql-mastery:21:1	CHECK rules
sql-mastery:21:2	UNIQUE on one or several columns
sql-mastery:21:3	ALTER TABLE: changing a table that already exists
sql-mastery:21:4	What happens on delete: CASCADE, SET NULL, RESTRICT
sql-mastery:21:5	Putting it together: a well-protected table

C-sql-mastery-D22 · Day 22: Transactions: All or Nothing

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 22`, then `npm run visuals:check -- --course sql-mastery --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D22] visuals for sql-mastery day 22`

Key	Part title
sql-mastery:22:0	Why some changes must happen together
sql-mastery:22:1	ROLLBACK: undoing on purpose
sql-mastery:22:2	Errors inside a transaction
sql-mastery:22:3	ACID in plain words
sql-mastery:22:4	When two people change the same row
sql-mastery:22:5	Putting it together: placing an order safely

C-sql-mastery-D23 · Day 23: Faster Queries: Indexes and EXPLAIN

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 23`, then `npm run visuals:check -- --course sql-mastery --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D23] visuals for sql-mastery day 23`

Key	Part title
sql-mastery:23:0	Why big tables get slow
sql-mastery:23:1	EXPLAIN: seeing the plan
sql-mastery:23:2	CREATE INDEX
sql-mastery:23:3	What indexes cost
sql-mastery:23:4	Indexes on several columns and on expressions
sql-mastery:23:5	Putting it together: speeding up a slow report

C-sql-mastery-D24 · Day 24: Saved Queries: Views

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 24`, then `npm run visuals:check -- --course sql-mastery --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D24] visuals for sql-mastery day 24`

Key	Part title
sql-mastery:24:0	What a view is
sql-mastery:24:1	Views always show current data
sql-mastery:24:2	Views for simplicity and safety
sql-mastery:24:3	Building on views
sql-mastery:24:4	Materialized views: saved results you refresh
sql-mastery:24:5	Putting it together: a reporting view for the shop

C-sql-mastery-D25 · Day 25: JSON Data in PostgreSQL

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 25`, then `npm run visuals:check -- --course sql-mastery --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D25] visuals for sql-mastery day 25`

Key	Part title
sql-mastery:25:0	Why store JSON in a database
sql-mastery:25:1	Reading values: <code>-></code> and <code>->></code>
sql-mastery:25:2	Filtering and grouping by JSON values
sql-mastery:25:3	Building and changing JSON
sql-mastery:25:4	Columns or JSON: choosing well
sql-mastery:25:5	Putting it together: analysing app events

R-sql-mastery-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course sql-mastery --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-sql-mastery-21-25] screenshots for sql-mastery days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-sql-mastery-D26 · Day 26: Using a Database from Python and Node.js

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 26`, then `npm run visuals:check -- --course sql-mastery --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D26] visuals for sql-mastery day 26`

Key	Part title
sql-mastery:26:0	How an app talks to a database
sql-mastery:26:1	SQL injection: the most important security rule
sql-mastery:26:2	Parameters: the safe way
sql-mastery:26:3	Transactions and errors from code
sql-mastery:26:4	ORMs and query builders
sql-mastery:26:5	Putting it together: a safe search feature

C-sql-mastery-D27 · Day 27: Project: Designing the Canteen Database

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 27`, then `npm run visuals:check -- --course sql-mastery --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D27] visuals for sql-mastery day 27`

Key	Part title
sql-mastery:27:0	The project and its user stories
sql-mastery:27:1	From stories to tables
sql-mastery:27:2	Choosing types, keys and rules
sql-mastery:27:3	Building the schema
sql-mastery:27:4	Testing the rules
sql-mastery:27:5	Putting it together: placing a canteen order

C-sql-mastery-D28 · Day 28: Project: Reports for the Canteen

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 28`, then `npm run visuals:check -- --course sql-mastery --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D28] visuals for sql-mastery day 28`

Key	Part title
sql-mastery:28:0	From a manager's question to a query
sql-mastery:28:1	Takings per day
sql-mastery:28:2	Spending per student
sql-mastery:28:3	Veg and non-veg, and the busiest hour
sql-mastery:28:4	Saving the reports as views
sql-mastery:28:5	Putting it together: the manager's dashboard query

C-sql-mastery-D29 · Day 29: Backups, Permissions, and SQL vs NoSQL

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 29`, then `npm run visuals:check -- --course sql-mastery --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D29] visuals for sql-mastery day 29`

Key	Part title
sql-mastery:29:0	Why backups matter
sql-mastery:29:1	Copying and restoring tables
sql-mastery:29:2	Users, roles and permissions
sql-mastery:29:3	Seeing permissions in action
sql-mastery:29:4	SQL vs NoSQL
sql-mastery:29:5	Putting it together: a production checklist

C-sql-mastery-D30 · Day 30: SQL Interview Practice

- **Command:** `npm run visuals:generate -- --course sql-mastery --day 30`, then `npm run visuals:check -- --course sql-mastery --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/sql-mastery/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-sql-mastery-D30] visuals for sql-mastery day 30`

Key	Part title
sql-mastery:30:0	What SQL interviews look like
sql-mastery:30:1	Second highest value
sql-mastery:30:2	Duplicates, and top N per group
sql-mastery:30:3	Explaining concepts clearly
sql-mastery:30:4	Talking about your Canteen project
sql-mastery:30:5	Your next steps

R-sql-mastery-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course sql-mastery --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-sql-mastery-26-30] screenshots for sql-mastery days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-sql-mastery · Release: students can see Database Engineering / SQL (Month 3) pictures

- Add 'sql-mastery' to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only.
Commit `[task:L-sql-mastery] release visuals for sql-mastery`.

M-3m · Milestone: the 3m plan has visuals in every course

- **The owner** opens 3 lessons of each newly released course on a phone and on a laptop, and signs off.
- **Antigravity** sends the latest green CI link with the status table.

Course 4: AI & ML in Python (Month 4) (ai-py)

C-ai-py-D01 · Day 1: Generative AI Foundations & Transformer Self-Attention

- **Command:** `npm run visuals:generate -- --course ai-py --day 1`, then `npm run visuals:check -- --course ai-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D01] visuals for ai-py day 1`

Key	Part title
ai-py:1:0	A model that predicts the next word
ai-py:1:1	From scores to probabilities: softmax
ai-py:1:2	Temperature: how adventurous the model is
ai-py:1:3	Attention: which words matter for this one?
ai-py:1:4	Scaled dot-product attention
ai-py:1:5	The transformer, in one picture

C-ai-py-D02 · Day 2: LLM Tokenization, Byte-Pair Encoding (BPE) & Context Economics

- **Command:** `npm run visuals:generate -- --course ai-py --day 2`, then `npm run visuals:check -- --course ai-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D02] visuals for ai-py day 2`

Key	Part title
ai-py:2:0	Tokens are pieces of words

Key	Part title
ai-py:2:1	Byte-pair encoding: learning the pieces
ai-py:2:2	Applying merge rules
ai-py:2:3	The context window
ai-py:2:4	What does a call cost?
ai-py:2:5	Context economics: spending tokens wisely

C-ai-py-D03 · Day 3: System Prompts, Personas & Guardrail Instructions

- **Command:** `npm run visuals:generate -- --course ai-py --day 3`, then `npm run visuals:check -- --course ai-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D03] visuals for ai-py day 3`

Key	Part title
ai-py:3:0	Messages and roles
ai-py:3:1	Anatomy of a good system prompt
ai-py:3:2	Structuring prompts with tags
ai-py:3:3	Prompt injection and stripping tags
ai-py:3:4	Guardrails on the output
ai-py:3:5	Personas, tone and testing prompts

C-ai-py-D04 · Day 4: Few-Shot Prompting & Chain-of-Thought (CoT) Reasoning

- **Command:** `npm run visuals:generate -- --course ai-py --day 4`, then `npm run visuals:check -- --course ai-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D04] visuals for ai-py day 4`

Key	Part title
ai-py:4:0	Zero-shot and few-shot prompts
ai-py:4:1	Formatting examples cleanly
ai-py:4:2	Choosing good examples
ai-py:4:3	Chain-of-thought reasoning
ai-py:4:4	Self-consistency: majority vote
ai-py:4:5	When reasoning is worth the tokens

C-ai-py-D05 · Day 5: ★ MILESTONE 1: Structured JSON Outputs & Pydantic/Zod Schema Enforcement

- **Command:** `npm run visuals:generate -- --course ai-py --day 5`, then `npm run visuals:check -- --course ai-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D05] visuals for ai-py day 5`

Key	Part title
ai-py:5:0	Why structured output
ai-py:5:1	Cleaning up the model's JSON
ai-py:5:2	Checking required keys
ai-py:5:3	Checking value types
ai-py:5:4	Retry with a repair prompt
ai-py:5:5	Milestone 1: a schema-checked extractor

R-ai-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course ai-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-ai-py-1-5] screenshots for ai-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-ai-py-D06 · Day 6: Function Calling & Tool Declaration Protocols

- **Command:** `npm run visuals:generate -- --course ai-py --day 6`, then `npm run visuals:check -- --course ai-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D06] visuals for ai-py day 6`

Key	Part title
ai-py:6:0	Letting a model use your code
ai-py:6:1	Declaring a tool
ai-py:6:2	Dispatching tool calls safely
ai-py:6:3	Sending results back to the model
ai-py:6:4	Validating arguments and staying safe
ai-py:6:5	Many tools, one registry

C-ai-py-D07 · Day 7: Text Embeddings & Vector Cosine Similarity Mathematics

- **Command:** `npm run visuals:generate -- --course ai-py --day 7`, then `npm run visuals:check -- --course ai-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D07] visuals for ai-py day 7`

Key	Part title
ai-py:7:0	Meaning as a list of numbers
ai-py:7:1	Vector length
ai-py:7:2	Cosine similarity
ai-py:7:3	Ranking documents by meaning
ai-py:7:4	Normalising vectors
ai-py:7:5	Embeddings in practice

C-ai-py-D08 · Day 8: Vector Databases: Indexing & Approximate Nearest Neighbors (HNSW)

- **Command:** `npm run visuals:generate -- --course ai-py --day 8`, then `npm run visuals:check -- --course ai-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D08] visuals for ai-py day 8`

Key	Part title
ai-py:8:0	Why brute force stops scaling
ai-py:8:1	Search with metadata filters
ai-py:8:2	Selecting the top k
ai-py:8:3	IVF: search only the nearby buckets
ai-py:8:4	HNSW: a graph you can walk
ai-py:8:5	Measuring recall

C-ai-py-D09 · Day 9: Document Chunking Strategies & Overlap Math

- **Command:** `npm run visuals:generate -- --course ai-py --day 9`, then `npm run visuals:check -- --course ai-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D09] visuals for ai-py day 9`

Key	Part title
ai-py:9:0	Why documents are cut into chunks
ai-py:9:1	Sliding window with overlap
ai-py:9:2	Overlap maths
ai-py:9:3	Splitting on natural boundaries
ai-py:9:4	Chunk metadata for citations
ai-py:9:5	Choosing a chunk size

C-ai-py-D10 · Day 10: Naive RAG vs Hybrid Search (Dense Vectors + BM25 Sparse)

- **Command:** `npm run visuals:generate -- --course ai-py --day 10`, then `npm run visuals:check -- --course ai-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D10] visuals for ai-py day 10`

Key	Part title
ai-py:10:0	Retrieval-augmented generation
ai-py:10:1	Keyword search: counting whole words
ai-py:10:2	BM25 in brief
ai-py:10:3	Dense plus sparse: hybrid search
ai-py:10:4	Reciprocal rank fusion
ai-py:10:5	A grounded prompt with citations

R-ai-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course ai-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-ai-py-6-10] screenshots for ai-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-ai-py-D11 · Day 11: Cross-Encoder Reranking & Context Precision (Cohere Rerank)

- **Command:** `npm run visuals:generate -- --course ai-py --day 11`, then `npm run visuals:check -- --course ai-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D11] visuals for ai-py day 11`

Key	Part title
ai-py:11:0	Recall first, precision second
ai-py:11:1	Bi-encoders and cross-encoders
ai-py:11:2	Reranking the candidates
ai-py:11:3	Dropping weak chunks
ai-py:11:4	Measuring context precision
ai-py:11:5	Cost and speed of reranking

C-ai-py-D12 · Day 12: Context Compression & The 'Lost in the Middle' Invariant

- **Command:** `npm run visuals:generate -- --course ai-py --day 12`, then `npm run visuals:check -- --course ai-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D12] visuals for ai-py day 12`

Key	Part title
ai-py:12:0	Lost in the middle
ai-py:12:1	Arranging chunks from the outside in
ai-py:12:2	Estimating context size
ai-py:12:3	Compressing chunks to what matters
ai-py:12:4	Removing near-duplicates
ai-py:12:5	Fitting a token budget

C-ai-py-D13 · Day 13: RAG Evaluation: Faithfulness, Answer Relevance & Context Recall (Ragas)

- **Command:** `npm run visuals:generate -- --course ai-py --day 13`, then `npm run visuals:check -- --course ai-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D13] visuals for ai-py day 13`

Key	Part title
ai-py:13:0	Why evaluation comes first
ai-py:13:1	Context recall
ai-py:13:2	Faithfulness
ai-py:13:3	Answer relevance and model judges
ai-py:13:4	Combining scores with a harmonic mean
ai-py:13:5	Evaluation as a safety gate

C-ai-py-D14 · Day 14: LLM Security: Prompt Injection & Jailbreak Defenses

- **Command:** `npm run visuals:generate -- --course ai-py --day 14`, then `npm run visuals:check -- --course ai-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D14] visuals for ai-py day 14`

Key	Part title
ai-py:14:0	The threat model
ai-py:14:1	Detecting classic injections
ai-py:14:2	Why patterns are not enough
ai-py:14:3	Cleaning retrieved content
ai-py:14:4	Canary tokens and output checks
ai-py:14:5	Least privilege and human approval

C-ai-py-D15 · Day 15: ★ MILESTONE 2: Production End-to-End Hybrid RAG Pipeline with Reranking

- **Command:** `npm run visuals:generate -- --course ai-py --day 15`, then `npm run visuals:check -- --course ai-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D15] visuals for ai-py day 15`

Key	Part title
ai-py:15:0	The pipeline at a glance
ai-py:15:1	Merging results without duplicates
ai-py:15:2	The run_rag_pipeline function
ai-py:15:3	Measuring latency
ai-py:15:4	Handling failures gracefully
ai-py:15:5	Tracing each stage

R-ai-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course ai-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-ai-py-11-15] screenshots for ai-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-ai-py-D16 · Day 16: LLM Memory Architectures: Sliding Windows & Summary Buffers

- **Command:** `npm run visuals:generate -- --course ai-py --day 16`, then `npm run visuals:check -- --course ai-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D16] visuals for ai-py day 16`

Key	Part title
ai-py:16:0	Models do not remember
ai-py:16:1	Sliding window memory
ai-py:16:2	Summary buffer memory
ai-py:16:3	Counting messages by role
ai-py:16:4	Long-term memory: facts worth keeping
ai-py:16:5	Choosing a memory strategy

C-ai-py-D17 · Day 17: Autonomous Agents: The ReAct (Reason + Act) Pattern

- **Command:** `npm run visuals:generate -- --course ai-py --day 17`, then `npm run visuals:check -- --course ai-py --day 17`

- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-17.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D17] visuals for ai-py day 17`

Key	Part title
ai-py:17:0	From one tool call to an agent
ai-py:17:1	Parsing a ReAct step
ai-py:17:2	The agent loop
ai-py:17:3	Stopping runaway agents
ai-py:17:4	Writing a good agent prompt
ai-py:17:5	When not to use an agent

C-ai-py-D18 · Day 18: Multi-Agent Collaboration: Supervisor & Swarm Architectures

- **Command:** `npm run visuals:generate -- --course ai-py --day 18`, then `npm run visuals:check -- --course ai-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-18.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D18] visuals for ai-py day 18`

Key	Part title
ai-py:18:0	Why several agents?
ai-py:18:1	A supervisor that routes tasks
ai-py:18:2	Supervisor versus swarm
ai-py:18:3	Handing over work clearly
ai-py:18:4	Readable status logs
ai-py:18:5	Keeping multi-agent systems under control

C-ai-py-D19 · Day 19: Agentic Planning: Plan-and-Solve & Reflection Self-Correction

- **Command:** `npm run visuals:generate -- --course ai-py --day 19`, then `npm run visuals:check -- --course ai-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-19.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D19] visuals for ai-py day 19`

Key	Part title
ai-py:19:0	Plan first, then act
ai-py:19:1	Tracking progress
ai-py:19:2	Reflection: checking your own work
ai-py:19:3	Repair prompts from errors
ai-py:19:4	A write, test, repair loop
ai-py:19:5	Balancing planning, reflection and cost

C-ai-py-D20 · Day 20: Real-Time Token Streaming with Server-Sent Events (SSE)

- **Command:** `npm run visuals:generate -- --course ai-py --day 20`, then `npm run visuals:check -- --course ai-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D20] visuals for ai-py day 20`

Key	Part title
ai-py:20:0	Why stream?
ai-py:20:1	The SSE format
ai-py:20:2	Parsing a streamed chunk
ai-py:20:3	Writing events for the browser
ai-py:20:4	Partial lines and buffering
ai-py:20:5	Cancellation and good streaming UX

R-ai-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course ai-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-ai-py-16-20] screenshots for ai-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-ai-py-D21 · Day 21: ★ MILESTONE 3: Autonomous Multi-Agent Research Assistant with Web & Code Tools

- **Command:** `npm run visuals:generate -- --course ai-py --day 21`, then `npm run visuals:check -- --course ai-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D21] visuals for ai-py day 21`

Key	Part title
ai-py:21:0	What the research assistant does
ai-py:21:1	The supervisor interface
ai-py:21:2	Orchestrating the plan
ai-py:21:3	Validating the report
ai-py:21:4	Tools with sources and safe code
ai-py:21:5	Hardening the assistant

C-ai-py-D22 · Day 22: LLM Caching: Exact vs Semantic Caching with Vector DBs (GPTCache)

- **Command:** `npm run visuals:generate -- --course ai-py --day 22`, then `npm run visuals:check -- --course ai-py --day 22`

- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D22] visuals for ai-py day 22`

Key	Part title
ai-py:22:0	Why cache LLM answers
ai-py:22:1	Exact caching
ai-py:22:2	Semantic caching
ai-py:22:3	Exact first, then semantic
ai-py:22:4	Measuring hit rate and savings
ai-py:22:5	Stale, personal and unsafe cache entries

C-ai-py-D23 · Day 23: PEFT: LoRA & QLoRA Fine-Tuning Adapters

- **Command:** `npm run visuals:generate -- --course ai-py --day 23`, then `npm run visuals:check -- --course ai-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D23] visuals for ai-py day 23`

Key	Part title
ai-py:23:0	Prompting, RAG or fine-tuning?
ai-py:23:1	Why full fine-tuning is expensive
ai-py:23:2	LoRA: small adapters
ai-py:23:3	Counting LoRA parameters
ai-py:23:4	Quantisation and QLoRA
ai-py:23:5	Preparing training data

C-ai-py-D24 · Day 24: Direct Preference Optimization (DPO) & RLHF Alignment

- **Command:** `npm run visuals:generate -- --course ai-py --day 24`, then `npm run visuals:check -- --course ai-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D24] visuals for ai-py day 24`

Key	Part title
ai-py:24:0	From imitation to preference
ai-py:24:1	RLHF in outline
ai-py:24:2	Log probabilities
ai-py:24:3	Direct preference optimisation
ai-py:24:4	Collecting good preference data
ai-py:24:5	Measuring alignment with win rates

C-ai-py-D25 · Day 25: Open-Source LLMs: vLLM High-Throughput Serving & GGUF Quantization

- **Command:** `npm run visuals:generate -- --course ai-py --day 25`, then `npm run visuals:check -- --course ai-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D25] visuals for ai-py day 25`

Key	Part title
ai-py:25:0	Why self-host an open model?
ai-py:25:1	The KV cache
ai-py:25:2	PagedAttention
ai-py:25:3	Batching and throughput
ai-py:25:4	GGUF and quantisation names
ai-py:25:5	Choosing how to run a model

R-ai-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course ai-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-ai-py-21-25] screenshots for ai-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-ai-py-D26 · Day 26: Multimodal AI: Vision-Language Models & Cross-Modal Embeddings

- **Command:** `npm run visuals:generate -- --course ai-py --day 26`, then `npm run visuals:check -- --course ai-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D26] visuals for ai-py day 26`

Key	Part title
ai-py:26:0	Models that see
ai-py:26:1	Images become patch tokens
ai-py:26:2	The cost of images
ai-py:26:3	Keeping the aspect ratio
ai-py:26:4	Shared embeddings for text and images
ai-py:26:5	Prompting with images, safely

C-ai-py-D27 · Day 27: LLMOps: Token Rate Limiting & Cost Budget Allocation

- **Command:** `npm run visuals:generate -- --course ai-py --day 27`, then `npm run visuals:check -- --course ai-py --day 27`

- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D27] visuals for ai-py day 27`

Key	Part title
ai-py:27:0	Why limits matter
ai-py:27:1	The token bucket
ai-py:27:2	Allowing, rejecting and refusing
ai-py:27:3	Counting calls per minute
ai-py:27:4	Retries with exponential backoff
ai-py:27:5	Budgets and graceful degradation

C-ai-py-D28 · Day 28: LLM Observability & Distributed Tracing (Langfuse / Helicone)

- **Command:** `npm run visuals:generate -- --course ai-py --day 28`, then `npm run visuals:check -- --course ai-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D28] visuals for ai-py day 28`

Key	Part title
ai-py:28:0	Why LLM apps need observability
ai-py:28:1	Traces and spans
ai-py:28:2	Aggregating a trace
ai-py:28:3	User feedback
ai-py:28:4	Dashboards and alerts
ai-py:28:5	Privacy in logs

C-ai-py-D29 · Day 29: Knowledge Graph RAG (GraphRAG) with Neo4j

- **Command:** `npm run visuals:generate -- --course ai-py --day 29`, then `npm run visuals:check -- --course ai-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D29] visuals for ai-py day 29`

Key	Part title
ai-py:29:0	When chunk retrieval struggles
ai-py:29:1	Building a graph from triples
ai-py:29:2	Traversing a relation
ai-py:29:3	Multi-hop questions
ai-py:29:4	Cypher queries, safely
ai-py:29:5	Combining graphs with vector search

C-ai-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Enterprise Agentic RAG Platform with Guardrails, Semantic Caching & Multi-Tool Execution

- **Command:** `npm run visuals:generate -- --course ai-py --day 30`, then `npm run visuals:check -- --course ai-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/ai-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-ai-py-D30] visuals for ai-py day 30`

Key	Part title
ai-py:30:0	The platform flow
ai-py:30:1	The run_ai_platform function
ai-py:30:2	Wrapping services with tracing
ai-py:30:3	Testing every path
ai-py:30:4	Certification audit
ai-py:30:5	Your AI engineering checklist

R-ai-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course ai-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-ai-py-26-30] screenshots for ai-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-ai-py · Release: students can see AI & ML in Python (Month 4) pictures

- Add `'ai-py'` to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-ai-py] release visuals for ai-py`.

Course 5: Distributed Python (Month 5) (dist-py)

C-dist-py-D01 · Day 1: Distributed Systems Foundations & Fallacies

- **Command:** `npm run visuals:generate -- --course dist-py --day 1`, then `npm run visuals:check -- --course dist-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D01] visuals for dist-py day 1`

Key	Part title
dist-py:1:0	Many machines acting as one
dist-py:1:1	The fallacies of distributed computing
dist-py:1:2	Partial failure and timeouts
dist-py:1:3	Retrying with exponential backoff
dist-py:1:4	Backoff delays, jitter and retry budgets

Key	Part title
dist-py:1:5	Latency and the long tail

C-dist-py-D02 · Day 2: The CAP Theorem & PACELC Theorem

- **Command:** `npm run visuals:generate -- --course dist-py --day 2`, then `npm run visuals:check -- --course dist-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D02] visuals for dist-py day 2`

Key	Part title
dist-py:2:0	Why keep copies
dist-py:2:1	The CAP theorem
dist-py:2:2	PACELC: the trade-off without failures
dist-py:2:3	Quorums and majorities
dist-py:2:4	Tunable consistency: $R + W > N$
dist-py:2:5	Choosing for real products

C-dist-py-D03 · Day 3: RPC Communication & Protocol Buffers Binary Serialization

- **Command:** `npm run visuals:generate -- --course dist-py --day 3`, then `npm run visuals:check -- --course dist-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D03] visuals for dist-py day 3`

Key	Part title
dist-py:3:0	Remote procedure calls
dist-py:3:1	Text versus binary serialisation
dist-py:3:2	Varints: small numbers, few bytes
dist-py:3:3	Tags and wire types
dist-py:3:4	Evolving schemas safely
dist-py:3:5	Deadlines, retries and status codes

C-dist-py-D04 · Day 4: Consistent Hashing & Virtual Nodes Distribution

- **Command:** `npm run visuals:generate -- --course dist-py --day 4`, then `npm run visuals:check -- --course dist-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D04] visuals for dist-py day 4`

Key	Part title
dist-py:4:0	The problem with hash mod N
dist-py:4:1	The hash ring
dist-py:4:2	Building the ring with virtual nodes
dist-py:4:3	Why virtual nodes balance load
dist-py:4:4	How many keys move?
dist-py:4:5	Replicas on the ring

C-dist-py-D05 · Day 5: ★ MILESTONE 1: High-Performance Distributed Cache with Cache-Aside & Thundering Herd Defense

- **Command:** `npm run visuals:generate -- --course dist-py --day 5`, then `npm run visuals:check -- --course dist-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D05] visuals for dist-py day 5`

Key	Part title
dist-py:5:0	The cache-aside pattern
dist-py:5:1	Expiry with TTL
dist-py:5:2	The thundering herd
dist-py:5:3	Spreading expiries with jitter
dist-py:5:4	Invalidation on writes
dist-py:5:5	Milestone 1: a bounded, distributed cache

R-dist-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course dist-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dist-py-1-5] screenshots for dist-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-dist-py-D06 · Day 6: Distributed Locks: Redis Redlock & Fencing Tokens

- **Command:** `npm run visuals:generate -- --course dist-py --day 6`, then `npm run visuals:check -- --course dist-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D06] visuals for dist-py day 6`

Key	Part title
dist-py:6:0	Why locks across machines
dist-py:6:1	Leases: locks that expire

Key	Part title
dist-py:6:2	Fencing tokens
dist-py:6:3	Clock drift and safety margins
dist-py:6:4	Redlock and majority locks
dist-py:6:5	Avoiding locks when you can

C-dist-py-D07 · Day 7: Leader Election: Bully Algorithm & Raft Heartbeats

- **Command:** `npm run visuals:generate -- --course dist-py --day 7`, then `npm run visuals:check -- --course dist-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D07] visuals for dist-py day 7`

Key	Part title
dist-py:7:0	Why have a leader?
dist-py:7:1	Detecting failure with heartbeats
dist-py:7:2	The Bully algorithm
dist-py:7:3	Majorities and terms
dist-py:7:4	Raft elections with randomised timeouts
dist-py:7:5	Safety: at most one leader per term

C-dist-py-D08 · Day 8: Distributed Unique ID Generation: Twitter Snowflake & ULID

- **Command:** `npm run visuals:generate -- --course dist-py --day 8`, then `npm run visuals:check -- --course dist-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D08] visuals for dist-py day 8`

Key	Part title
dist-py:8:0	Why not a simple counter?
dist-py:8:1	The Snowflake layout
dist-py:8:2	Building the generator
dist-py:8:3	Decoding an ID
dist-py:8:4	Time-sortable text IDs
dist-py:8:5	Choosing an ID scheme

C-dist-py-D09 · Day 9: Consensus Protocols: Raft Log Replication & Quorum Mathematics

- **Command:** `npm run visuals:generate -- --course dist-py --day 9`, then `npm run visuals:check -- --course dist-py --day 9`

- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D09] visuals for dist-py day 9`

Key	Part title
<code>dist-py:9:0</code>	Replicated state machines
<code>dist-py:9:1</code>	Log entries, terms and AppendEntries
<code>dist-py:9:2</code>	The follower's consistency check
<code>dist-py:9:3</code>	When is an entry committed?
<code>dist-py:9:4</code>	Repairing a lagging follower
<code>dist-py:9:5</code>	Reads, snapshots and Raft in practice

C-dist-py-D10 · Day 10: Two-Phase Commit (2PC) vs Three-Phase Commit (3PC)

- **Command:** `npm run visuals:generate -- --course dist-py --day 10`, then `npm run visuals:check -- --course dist-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D10] visuals for dist-py day 10`

Key	Part title
<code>dist-py:10:0</code>	All or nothing across services
<code>dist-py:10:1</code>	The two phases
<code>dist-py:10:2</code>	Implementing the coordinator
<code>dist-py:10:3</code>	Counting votes and keeping a log
<code>dist-py:10:4</code>	The blocking problem
<code>dist-py:10:5</code>	Three-phase commit and alternatives

R-dist-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course dist-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dist-py-6-10] screenshots for dist-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-dist-py-D11 · Day 11: The Saga Pattern: Orchestration vs Choreography & Compensating Actions

- **Command:** `npm run visuals:generate -- --course dist-py --day 11`, then `npm run visuals:check -- --course dist-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D11] visuals for dist-py day 11`

Key	Part title
dist-py:11:0	A saga is a chain of local transactions
dist-py:11:1	Designing compensations
dist-py:11:2	Running a saga
dist-py:11:3	The saga log
dist-py:11:4	Orchestration or choreography
dist-py:11:5	Isolation and other pitfalls

C-dist-py-D12 · Day 12: Event-Driven Messaging: Kafka Partitions & Consumer Group Rebalancing

- **Command:** `npm run visuals:generate -- --course dist-py --day 12`, then `npm run visuals:check -- --course dist-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D12] visuals for dist-py day 12`

Key	Part title
dist-py:12:0	Events, producers and consumers
dist-py:12:1	Partitions and ordering by key
dist-py:12:2	Consumer groups
dist-py:12:3	Rebalancing
dist-py:12:4	Offsets and commits
dist-py:12:5	Consumer lag and scaling

C-dist-py-D13 · Day 13: Message Delivery Guarantees: At-Least-Once, At-Most-Once & Exactly-Once Idempotency

- **Command:** `npm run visuals:generate -- --course dist-py --day 13`, then `npm run visuals:check -- --course dist-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D13] visuals for dist-py day 13`

Key	Part title
dist-py:13:0	Three delivery guarantees
dist-py:13:1	Why duplicates hurt
dist-py:13:2	Processing each message once
dist-py:13:3	Idempotency keys in APIs
dist-py:13:4	Exactly-once processing in practice
dist-py:13:5	Designing naturally idempotent operations

C-dist-py-D14 · Day 14: Dead Letter Queues (DLQ), Exponential Backoff & Poison Pill Handling

- **Command:** `npm run visuals:generate -- --course dist-py --day 14`, then `npm run visuals:check -- --course dist-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D14] visuals for dist-py day 14`

Key	Part title
dist-py:14:0	The poison pill
dist-py:14:1	Temporary or permanent?
dist-py:14:2	Retry, then dead-letter
dist-py:14:3	What to store in the DLQ
dist-py:14:4	Redrive and retry topics
dist-py:14:5	Monitoring and ordering concerns

C-dist-py-D15 · Day 15: ★ MILESTONE 2: Resilient Event-Driven Transaction Engine with Sagas & Idempotency Keys

- **Command:** `npm run visuals:generate -- --course dist-py --day 15`, then `npm run visuals:check -- --course dist-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D15] visuals for dist-py day 15`

Key	Part title
dist-py:15:0	The engine's flow
dist-py:15:1	Dropping duplicates
dist-py:15:2	The run_transaction function
dist-py:15:3	Measuring duration
dist-py:15:4	Testing every failure path
dist-py:15:5	Publishing events reliably: the outbox

R-dist-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course dist-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dist-py-11-15] screenshots for dist-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-dist-py-D16 · Day 16: Physical Clocks, NTP Drift, Lamport Timestamps & Vector Clocks

- **Command:** `npm run visuals:generate -- --course dist-py --day 16`, then `npm run visuals:check -- --course dist-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D16] visuals for dist-py day 16`

Key	Part title
dist-py:16:0	Physical clocks drift
dist-py:16:1	Happened-before
dist-py:16:2	Lamport timestamps
dist-py:16:3	Vector clocks
dist-py:16:4	Comparing vector clocks
dist-py:16:5	Resolving conflicts

C-dist-py-D17 · Day 17: Conflict-Free Replicated Data Types (CRDTs): G-Counter, PN-Counter & LWW-Set

- **Command:** `npm run visuals:generate -- --course dist-py --day 17`, then `npm run visuals:check -- --course dist-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D17] visuals for dist-py day 17`

Key	Part title
dist-py:17:0	Merging without coordination
dist-py:17:1	The G-Counter
dist-py:17:2	The PN-Counter
dist-py:17:3	Last-writer-wins registers
dist-py:17:4	Sets that merge: add-wins sets
dist-py:17:5	When to use CRDTs

C-dist-py-D18 · Day 18: Database Sharding Strategies: Range, Hash & Directory Sharding

- **Command:** `npm run visuals:generate -- --course dist-py --day 18`, then `npm run visuals:check -- --course dist-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D18] visuals for dist-py day 18`

Key	Part title
dist-py:18:0	Why shard?
dist-py:18:1	Range sharding

Key	Part title
dist-py:18:2	Hash and directory sharding
dist-py:18:3	Choosing a shard key
dist-py:18:4	Resharding
dist-py:18:5	Queries across shards

C-dist-py-D19 · Day 19: Read Replicas, Replication Lag & Read-Your-Own-Writes Consistency

- **Command:** `npm run visuals:generate -- --course dist-py --day 19`, then `npm run visuals:check -- --course dist-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D19] visuals for dist-py day 19`

Key	Part title
dist-py:19:0	Primary and read replicas
dist-py:19:1	The read-your-own-writes problem
dist-py:19:2	Routing queries by session
dist-py:19:3	Measuring and alerting on lag
dist-py:19:4	Synchronous, asynchronous and semi-synchronous
dist-py:19:5	Other session guarantees

C-dist-py-D20 · Day 20: Circuit Breakers (Resilience4j / Envoy) & Bulkhead Isolation

- **Command:** `npm run visuals:generate -- --course dist-py --day 20`, then `npm run visuals:check -- --course dist-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D20] visuals for dist-py day 20`

Key	Part title
dist-py:20:0	Cascading failures
dist-py:20:1	The circuit breaker states
dist-py:20:2	Implementing the breaker
dist-py:20:3	Reporting breaker status
dist-py:20:4	Bulkheads
dist-py:20:5	Fallbacks and graceful degradation

R-dist-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course dist-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`

- **Commit message:** [task:R-dist-py-16-20] screenshots for dist-py days 16-20
- **Wait** for Claude's "approved" before the next day task.

C-dist-py-D21 · Day 21: ★ MILESTONE 3: Distributed Rate Limiter & Circuit Breaker API Gateway

- **Command:** `npm run visuals:generate -- --course dist-py --day 21`, then `npm run visuals:check -- --course dist-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dist-py-D21] visuals for dist-py day 21

Key	Part title
dist-py:21:0	What a gateway does
dist-py:21:1	Per-client rate limiting
dist-py:21:2	Handling a request end to end
dist-py:21:3	Timing headers
dist-py:21:4	Testing every gateway path
dist-py:21:5	Running gateways in production

C-dist-py-D22 · Day 22: Gossip Protocols: SWIM Failure Detection & Cluster Membership

- **Command:** `npm run visuals:generate -- --course dist-py --day 22`, then `npm run visuals:check -- --course dist-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dist-py-D22] visuals for dist-py day 22

Key	Part title
dist-py:22:0	Why gossip?
dist-py:22:1	Choosing gossip peers
dist-py:22:2	SWIM: probing for failures
dist-py:22:3	Suspicion before declaring death
dist-py:22:4	Membership and dissemination
dist-py:22:5	Where gossip fits

C-dist-py-D23 · Day 23: Load Balancing Algorithms: Weighted Round-Robin, Least Connections & Consistent Hash Ring

- **Command:** `npm run visuals:generate -- --course dist-py --day 23`, then `npm run visuals:check -- --course dist-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-dist-py-D23] visuals for dist-py day 23

Key	Part title
dist-py:23:0	Why load balance?
dist-py:23:1	Smooth weighted round-robin
dist-py:23:2	Least connections
dist-py:23:3	Hashing for stickiness
dist-py:23:4	Health checks and removing servers
dist-py:23:5	Choosing an algorithm

C-dist-py-D24 · Day 24: Service Discovery & Heartbeat Health Checking (Consul / Zookeeper)

- **Command:** `npm run visuals:generate -- --course dist-py --day 24`, then `npm run visuals:check -- --course dist-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D24] visuals for dist-py day 24`

Key	Part title
dist-py:24:0	Why discovery?
dist-py:24:1	Registration with leases
dist-py:24:2	Building instance URLs
dist-py:24:3	Client-side and server-side discovery
dist-py:24:4	DNS-based discovery and caching
dist-py:24:5	Health beyond heartbeats

C-dist-py-D25 · Day 25: API Gateways & Backend-For-Frontend (BFF) Pattern

- **Command:** `npm run visuals:generate -- --course dist-py --day 25`, then `npm run visuals:check -- --course dist-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D25] visuals for dist-py day 25`

Key	Part title
dist-py:25:0	The chatty client problem
dist-py:25:1	Aggregating a profile
dist-py:25:2	Calling services in parallel
dist-py:25:3	CORS headers for browsers
dist-py:25:4	One BFF per client type
dist-py:25:5	Operating the edge

R-dist-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course dist-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dist-py-21-25] screenshots for dist-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-dist-py-D26 · Day 26: Distributed Tracing: OpenTelemetry, W3C TraceContext & Span Propagation

- **Command:** `npm run visuals:generate -- --course dist-py --day 26`, then `npm run visuals:check -- --course dist-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D26] visuals for dist-py day 26`

Key	Part title
dist-py:26:0	Following one request through many services
dist-py:26:1	The W3C traceparent header
dist-py:26:2	Creating a child span
dist-py:26:3	Validating headers
dist-py:26:4	Sampling
dist-py:26:5	Traces, logs and metrics together

C-dist-py-D27 · Day 27: Data Consistency Models: Linearizable vs Sequential vs Eventual Consistency

- **Command:** `npm run visuals:generate -- --course dist-py --day 27`, then `npm run visuals:check -- --course dist-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D27] visuals for dist-py day 27`

Key	Part title
dist-py:27:0	A spectrum of guarantees
dist-py:27:1	Choosing a level by name
dist-py:27:2	Detecting stale reads
dist-py:27:3	Causal consistency in practice
dist-py:27:4	Eventual consistency done well
dist-py:27:5	Choosing per feature

C-dist-py-D28 · Day 28: Reverse Proxies & CDN Edge Caching with Cache-Control Invalidation

- **Command:** `npm run visuals:generate -- --course dist-py --day 28`, then `npm run visuals:check -- --course dist-py --day 28`

- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D28] visuals for dist-py day 28`

Key	Part title
<code>dist-py:28:0</code>	Reverse proxies and CDNs
<code>dist-py:28:1</code>	Cache-Control headers
<code>dist-py:28:2</code>	Fresh, stale or expired?
<code>dist-py:28:3</code>	Purging with surrogate keys
<code>dist-py:28:4</code>	What to cache, and what not
<code>dist-py:28:5</code>	Protecting the origin

C-dist-py-D29 · Day 29: Disaster Recovery: Multi-Region Active-Passive vs Active-Active Deployments

- **Command:** `npm run visuals:generate -- --course dist-py --day 29`, then `npm run visuals:check -- --course dist-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D29] visuals for dist-py day 29`

Key	Part title
<code>dist-py:29:0</code>	When a region fails
<code>dist-py:29:1</code>	Checking compliance
<code>dist-py:29:2</code>	The DR strategies
<code>dist-py:29:3</code>	Active-passive failover
<code>dist-py:29:4</code>	Active-active challenges
<code>dist-py:29:5</code>	Drills and reporting

C-dist-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Enterprise Global Real-Time Financial Trading & Ledger Exchange Engine

- **Command:** `npm run visuals:generate -- --course dist-py --day 30`, then `npm run visuals:check -- --course dist-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/dist-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-dist-py-D30] visuals for dist-py day 30`

Key	Part title
<code>dist-py:30:0</code>	The trading engine's requirements
<code>dist-py:30:1</code>	The <code>execute_trade</code> function
<code>dist-py:30:2</code>	Testing every failure path
<code>dist-py:30:3</code>	Fencing tokens protect the ledger

Key	Part title
dist-py:30:4	The certification audit
dist-py:30:5	Looking back, and next steps

R-dist-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course dist-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-dist-py-26-30] screenshots for dist-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-dist-py · Release: students can see Distributed Python (Month 5) pictures

- Add `'dist-py'` to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-dist-py] release visuals for dist-py`.

Course 6: Cloud & MLOps Python (Month 6) (cloud-py)

C-cloud-py-D01 · Day 1: Cloud Computing Models (IaaS, PaaS, SaaS) & Shared Responsibility

- **Command:** `npm run visuals:generate -- --course cloud-py --day 1`, then `npm run visuals:check -- --course cloud-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D01] visuals for cloud-py day 1`

Key	Part title
cloud-py:1:0	What the cloud really is
cloud-py:1:1	IaaS, PaaS and SaaS
cloud-py:1:2	The shared responsibility model
cloud-py:1:3	Paying for what you use
cloud-py:1:4	How you talk to AWS: console, CLI and code
cloud-py:1:5	Practice time: model the cloud in code

C-cloud-py-D02 · Day 2: AWS Global Infrastructure, Regions & Availability Zones

- **Command:** `npm run visuals:generate -- --course cloud-py --day 2`, then `npm run visuals:check -- --course cloud-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D02] visuals for cloud-py day 2`

Key	Part title
cloud-py:2:0	Regions: separate clouds around the world

Key	Part title
cloud-py:2:1	Availability Zones: failure boundaries inside a Region
cloud-py:2:2	Checking region codes with a regular expression
cloud-py:2:3	Edge locations and the speed of light
cloud-py:2:4	Designing for failure
cloud-py:2:5	Practice time: AZ checks and region codes

C-cloud-py-D03 · Day 3: Virtual Private Cloud (VPC) Architecture & CIDR Subnetting

- **Command:** `npm run visuals:generate -- --course cloud-py --day 3`, then `npm run visuals:check -- --course cloud-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D03] visuals for cloud-py day 3`

Key	Part title
cloud-py:3:0	Your own private network: the VPC
cloud-py:3:1	Reading CIDR notation
cloud-py:3:2	The five addresses AWS keeps
cloud-py:3:3	Splitting a VPC into subnets
cloud-py:3:4	Public and private subnets: it is all in the route table
cloud-py:3:5	Practice time: sizes and routes

C-cloud-py-D04 · Day 4: Security Groups vs Network Access Control Lists (NACLs)

- **Command:** `npm run visuals:generate -- --course cloud-py --day 4`, then `npm run visuals:check -- --course cloud-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D04] visuals for cloud-py day 4`

Key	Part title
cloud-py:4:0	Two layers of firewall
cloud-py:4:1	Stateful versus stateless
cloud-py:4:2	NACL rule numbers: first match wins
cloud-py:4:3	Security groups that reference each other
cloud-py:4:4	Least privilege on the network
cloud-py:4:5	Practice time: evaluate the firewalls

C-cloud-py-D05 · Day 5: ★ MILESTONE 1: High-Availability Multi-AZ VPC Network Topology & Bastion Host

- **Command:** `npm run visuals:generate -- --course cloud-py --day 5`, then `npm run visuals:check -- --course cloud-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D05] visuals for cloud-py day 5`

Key	Part title
cloud-py:5:0	The reference architecture
cloud-py:5:1	NAT gateways: out but not in
cloud-py:5:2	The bastion host
cloud-py:5:3	Writing a validator that lists every problem
cloud-py:5:4	Proving subnets do not overlap
cloud-py:5:5	Milestone review: from plan to production

R-cloud-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course cloud-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-cloud-py-1-5] screenshots for cloud-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-cloud-py-D06 · Day 6: IAM Role Least-Privilege, Policies & Principal Trust

- **Command:** `npm run visuals:generate -- --course cloud-py --day 6`, then `npm run visuals:check -- --course cloud-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D06] visuals for cloud-py day 6`

Key	Part title
cloud-py:6:0	Identity and Access Management
cloud-py:6:1	How AWS decides: explicit deny wins
cloud-py:6:2	Wildcards and least privilege
cloud-py:6:3	Roles and temporary credentials
cloud-py:6:4	Amazon Resource Names
cloud-py:6:5	Practice time: build the policy engine

C-cloud-py-D07 · Day 7: EC2 Compute Classes, Spot Instances & Auto-Scaling Groups

- **Command:** `npm run visuals:generate -- --course cloud-py --day 7`, then `npm run visuals:check -- --course cloud-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D07] visuals for cloud-py day 7`

Key	Part title
cloud-py:7:0	EC2 instances and families
cloud-py:7:1	On-Demand, Reserved, Savings Plans and Spot
cloud-py:7:2	Auto Scaling groups
cloud-py:7:3	Target tracking: the thermostat of scaling
cloud-py:7:4	Surviving Spot interruptions
cloud-py:7:5	Practice time: scale and survive

C-cloud-py-D08 · Day 8: Application Load Balancer (ALB), Target Groups & Health Probes

- **Command:** `npm run visuals:generate -- --course cloud-py --day 8`, then `npm run visuals:check -- --course cloud-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D08] visuals for cloud-py day 8`

Key	Part title
cloud-py:8:0	Why a load balancer
cloud-py:8:1	Listeners, rules and target groups
cloud-py:8:2	Health checks and thresholds
cloud-py:8:3	HTTPS and TLS termination
cloud-py:8:4	Sticky sessions and connection draining
cloud-py:8:5	Practice time: route and check health

C-cloud-py-D09 · Day 9: Amazon S3 Object Storage & Lifecycle Management Tiering

- **Command:** `npm run visuals:generate -- --course cloud-py --day 9`, then `npm run visuals:check -- --course cloud-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D09] visuals for cloud-py day 9`

Key	Part title
cloud-py:9:0	Object storage and S3
cloud-py:9:1	Storage classes
cloud-py:9:2	Lifecycle rules
cloud-py:9:3	Versioning and delete markers
cloud-py:9:4	Uploading big files and presigned URLs
cloud-py:9:5	Practice time: classes and versions

C-cloud-py-D10 · Day 10: Amazon S3 Security, Block Public Access & Bucket Policies

- **Command:** `npm run visuals:generate -- --course cloud-py --day 10`, then `npm run visuals:check -- --course cloud-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D10] visuals for cloud-py day 10`

Key	Part title
cloud-py:10:0	How buckets leak
cloud-py:10:1	Bucket policies
cloud-py:10:2	Enforcing HTTPS with aws:SecureTransport
cloud-py:10:3	Bucket naming rules
cloud-py:10:4	Encryption at rest
cloud-py:10:5	Practice time: lock the bucket down

R-cloud-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course cloud-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-cloud-py-6-10] screenshots for cloud-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-cloud-py-D11 · Day 11: Serverless AWS Lambda: Concurrency, Memory & Cold Starts

- **Command:** `npm run visuals:generate -- --course cloud-py --day 11`, then `npm run visuals:check -- --course cloud-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D11] visuals for cloud-py day 11`

Key	Part title
cloud-py:11:0	Functions without servers
cloud-py:11:1	How Lambda is priced
cloud-py:11:2	Cold starts and warm starts
cloud-py:11:3	Concurrency
cloud-py:11:4	Fighting cold starts: SnapStart and provisioned concurrency
cloud-py:11:5	Practice time: cost and cold starts

C-cloud-py-D12 · Day 12: Amazon API Gateway V2 HTTP & Lambda Authorizers

- **Command:** `npm run visuals:generate -- --course cloud-py --day 12`, then `npm run visuals:check -- --course cloud-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-12.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-cloud-py-D12] visuals for cloud-py day 12

Key	Part title
cloud-py:12:0	API Gateway in front of Lambda
cloud-py:12:1	HTTP APIs and REST APIs
cloud-py:12:2	Lambda authorizers
cloud-py:12:3	Throttling and usage limits
cloud-py:12:4	CORS for browser apps
cloud-py:12:5	Practice time: authorize and share

C-cloud-py-D13 · Day 13: Amazon DynamoDB Partition Keys & Global Secondary Indexes (GSI)

- **Command:** `npm run visuals:generate -- --course cloud-py --day 13`, then `npm run visuals:check -- --course cloud-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-cloud-py-D13] visuals for cloud-py day 13

Key	Part title
cloud-py:13:0	Tables, items and keys
cloud-py:13:1	Designing keys from access patterns
cloud-py:13:2	Hot partitions
cloud-py:13:3	Global secondary indexes
cloud-py:13:4	Capacity: RCUs and WCUs
cloud-py:13:5	Practice time: hot keys and capacity

C-cloud-py-D14 · Day 14: Amazon RDS Multi-AZ High Availability & Read Replicas

- **Command:** `npm run visuals:generate -- --course cloud-py --day 14`, then `npm run visuals:check -- --course cloud-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-cloud-py-D14] visuals for cloud-py day 14

Key	Part title
cloud-py:14:0	Managed relational databases
cloud-py:14:1	Multi-AZ: a standby in another zone
cloud-py:14:2	Read replicas: scaling reads
cloud-py:14:3	Routing queries in the application
cloud-py:14:4	Backups, snapshots and connection pooling
cloud-py:14:5	Practice time: fail over and route

C-cloud-py-D15 · Day 15: ★ MILESTONE 2: Serverless Event-Driven Video Processing Engine

- **Command:** `npm run visuals:generate -- --course cloud-py --day 15`, then `npm run visuals:check -- --course cloud-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D15] visuals for cloud-py day 15`

Key	Part title
cloud-py:15:0	The architecture
cloud-py:15:1	Reading S3 event records
cloud-py:15:2	Processing every record and isolating failures
cloud-py:15:3	Idempotency: safe to run twice
cloud-py:15:4	Long jobs, costs and limits
cloud-py:15:5	Milestone practice: build the engine

R-cloud-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course cloud-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-cloud-py-11-15] screenshots for cloud-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-cloud-py-D16 · Day 16: Amazon CloudFront Global CDN & Edge Functions (Lambda@Edge)

- **Command:** `npm run visuals:generate -- --course cloud-py --day 16`, then `npm run visuals:check -- --course cloud-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D16] visuals for cloud-py day 16`

Key	Part title
cloud-py:16:0	What a CDN does
cloud-py:16:1	Cache-Control and TTL
cloud-py:16:2	Cache keys and hit ratio
cloud-py:16:3	Invalidations and versioned file names
cloud-py:16:4	Code at the edge
cloud-py:16:5	Practice time: TTLs and cache keys

C-cloud-py-D17 · Day 17: Amazon Route 53 DNS Routing Policies & Health Checks

- **Command:** `npm run visuals:generate -- --course cloud-py --day 17`, then `npm run visuals:check -- --course cloud-py --day 17`

- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-17.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D17] visuals for cloud-py day 17`

Key	Part title
cloud-py:17:0	How DNS finds your app
cloud-py:17:1	Record types and alias records
cloud-py:17:2	Health checks and failover routing
cloud-py:17:3	Weighted routing and canary releases
cloud-py:17:4	Latency and geolocation routing
cloud-py:17:5	Practice time: failover and weights

C-cloud-py-D18 · Day 18: Amazon SQS: Standard vs FIFO Queues & Visibility Timeouts

- **Command:** `npm run visuals:generate -- --course cloud-py --day 18`, then `npm run visuals:check -- --course cloud-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-18.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D18] visuals for cloud-py day 18`

Key	Part title
cloud-py:18:0	Why queues
cloud-py:18:1	Receive, process, delete: the visibility timeout
cloud-py:18:2	Standard and FIFO queues
cloud-py:18:3	FIFO deduplication
cloud-py:18:4	Long polling, batches and scaling consumers
cloud-py:18:5	Practice time: build the queue

C-cloud-py-D19 · Day 19: Amazon SNS: Pub/Sub Topic Fanout & Push Notifications

- **Command:** `npm run visuals:generate -- --course cloud-py --day 19`, then `npm run visuals:check -- --course cloud-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-19.json`,
`docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D19] visuals for cloud-py day 19`

Key	Part title
cloud-py:19:0	Publish and subscribe
cloud-py:19:1	Fan-out with SNS and SQS
cloud-py:19:2	Filter policies
cloud-py:19:3	Delivery retries and dead-letter queues
cloud-py:19:4	Push notifications, email and SMS

Key	Part title
cloud-py:19:5	Practice time: fan out and filter

C-cloud-py-D20 · Day 20: Amazon EventBridge: Serverless Event Bus & Schema Registry

- **Command:** `npm run visuals:generate -- --course cloud-py --day 20`, then `npm run visuals:check -- --course cloud-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D20] visuals for cloud-py day 20`

Key	Part title
cloud-py:20:0	Events and the event bus
cloud-py:20:1	Event patterns
cloud-py:20:2	Publishing your own events
cloud-py:20:3	Schedules, archives and replay
cloud-py:20:4	SNS, SQS or EventBridge?
cloud-py:20:5	Practice time: match and build events

R-cloud-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course cloud-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-cloud-py-16-20] screenshots for cloud-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-cloud-py-D21 · Day 21: ★ MILESTONE 3: High-Scale E-Commerce Microservices Event Bus with SQS/SNS Fanout

- **Command:** `npm run visuals:generate -- --course cloud-py --day 21`, then `npm run visuals:check -- --course cloud-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D21] visuals for cloud-py day 21`

Key	Part title
cloud-py:21:0	The shop as independent services
cloud-py:21:1	Routing events to service queues
cloud-py:21:2	Poison messages and dead-letter queues
cloud-py:21:3	Idempotent consumers and exactly-once effects
cloud-py:21:4	Tracing an order through the system
cloud-py:21:5	Milestone practice: build the backbone

C-cloud-py-D22 · Day 22: AWS ECS & AWS Fargate Serverless Container Architecture

- **Command:** `npm run visuals:generate -- --course cloud-py --day 22`, then `npm run visuals:check -- --course cloud-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D22] visuals for cloud-py day 22`

Key	Part title
cloud-py:22:0	Containers and images
cloud-py:22:1	ECS: clusters, task definitions and services
cloud-py:22:2	Fargate sizes
cloud-py:22:3	Configuration and secrets
cloud-py:22:4	Scaling and deploying services
cloud-py:22:5	Practice time: size and configure

C-cloud-py-D23 · Day 23: AWS Step Functions & Distributed Saga Pattern Orchestration

- **Command:** `npm run visuals:generate -- --course cloud-py --day 23`, then `npm run visuals:check -- --course cloud-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D23] visuals for cloud-py day 23`

Key	Part title
cloud-py:23:0	Orchestration and Step Functions
cloud-py:23:1	Amazon States Language
cloud-py:23:2	The saga pattern
cloud-py:23:3	Retries, catches and timeouts
cloud-py:23:4	Parallel steps and human approval
cloud-py:23:5	Practice time: saga and validator

C-cloud-py-D24 · Day 24: Infrastructure as Code (IaC) with Terraform & State Management

- **Command:** `npm run visuals:generate -- --course cloud-py --day 24`, then `npm run visuals:check -- --course cloud-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D24] visuals for cloud-py day 24`

Key	Part title
cloud-py:24:0	Infrastructure as Code

Key	Part title
cloud-py:24:1	Resource addresses and modules
cloud-py:24:2	State: Terraform's memory
cloud-py:24:3	Locking state
cloud-py:24:4	Plan, apply and reading a plan
cloud-py:24:5	Practice time: addresses and plans

C-cloud-py-D25 · Day 25: Amazon CloudWatch Metrics, Log Insights & Alarms

- **Command:** `npm run visuals:generate -- --course cloud-py --day 25`, then `npm run visuals:check -- --course cloud-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D25] visuals for cloud-py day 25`

Key	Part title
cloud-py:25:0	Metrics, logs and traces
cloud-py:25:1	Percentiles beat averages
cloud-py:25:2	Alarms over consecutive datapoints
cloud-py:25:3	Logs and Logs Insights
cloud-py:25:4	Dashboards and alerts people act on
cloud-py:25:5	Practice time: alarms and log summaries

R-cloud-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course cloud-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-cloud-py-21-25] screenshots for cloud-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-cloud-py-D26 · Day 26: AWS Key Management Service (KMS) & Envelope Encryption

- **Command:** `npm run visuals:generate -- --course cloud-py --day 26`, then `npm run visuals:check -- --course cloud-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D26] visuals for cloud-py day 26`

Key	Part title
cloud-py:26:0	Why encryption, and where
cloud-py:26:1	KMS keys, policies and grants
cloud-py:26:2	Envelope encryption

Key	Part title
cloud-py:26:3	Auditing key policies
cloud-py:26:4	Rotation, secrets and CloudTrail
cloud-py:26:5	Practice time: envelopes and policies

C-cloud-py-D27 · Day 27: AWS WAF & AWS Shield: DDoS & SQLi/XSS Protection

- **Command:** `npm run visuals:generate -- --course cloud-py --day 27`, then `npm run visuals:check -- --course cloud-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D27] visuals for cloud-py day 27`

Key	Part title
cloud-py:27:0	Threats at the front door
cloud-py:27:1	AWS WAF web ACLs
cloud-py:27:2	Rate-based rules
cloud-py:27:3	IP sets and CIDR validation
cloud-py:27:4	DDoS and AWS Shield
cloud-py:27:5	Practice time: inspect and validate

C-cloud-py-D28 · Day 28: AWS FinOps: Cost Optimization, Compute Savings Plans & Cost Allocation Tags

- **Command:** `npm run visuals:generate -- --course cloud-py --day 28`, then `npm run visuals:check -- --course cloud-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D28] visuals for cloud-py day 28`

Key	Part title
cloud-py:28:0	FinOps: engineering meets finance
cloud-py:28:1	Savings Plans and commitments
cloud-py:28:2	Finding idle and oversized resources
cloud-py:28:3	Cost allocation tags
cloud-py:28:4	Budgets, alerts and anomaly detection
cloud-py:28:5	Practice time: bills and tags

C-cloud-py-D29 · Day 29: Disaster Recovery (DR) Strategies: Backup, Pilot Light & Warm Standby

- **Command:** `npm run visuals:generate -- --course cloud-py --day 29`, then `npm run visuals:check -- --course cloud-py --day 29`

- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D29] visuals for cloud-py day 29`

Key	Part title
cloud-py:29:0	RTO and RPO
cloud-py:29:1	The four DR strategies
cloud-py:29:2	Choosing a strategy from targets
cloud-py:29:3	Pricing downtime and data loss
cloud-py:29:4	Backups that actually work
cloud-py:29:5	Practice time: choose and price

C-cloud-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Global Resilient Multi-Region FinTech Banking Infrastructure with Active-Active Failover

- **Command:** `npm run visuals:generate -- --course cloud-py --day 30`, then `npm run visuals:check -- --course cloud-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/cloud-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-cloud-py-D30] visuals for cloud-py day 30`

Key	Part title
cloud-py:30:0	The capstone architecture
cloud-py:30:1	Routing transactions: health, latency and residency
cloud-py:30:2	Keeping money consistent across Regions
cloud-py:30:3	Failing over a whole Region
cloud-py:30:4	The readiness audit
cloud-py:30:5	Capstone practice and what comes next

R-cloud-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course cloud-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-cloud-py-26-30] screenshots for cloud-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-cloud-py · Release: students can see Cloud & MLOps Python (Month 6) pictures

- Add 'cloud-py' to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-cloud-py] release visuals for cloud-py`.

M-6m · Milestone: the 6m plan has visuals in every course

- **The owner** opens 3 lessons of each newly released course on a phone and on a laptop, and signs off.

- **Antigravity** sends the latest green CI link with the status table.

Course 7: NLP in Python (Month 7) (nlp-py)

C-nlp-py-D01 · Day 1: Text Preprocessing Pipeline: Unicode Normalization & Regex Tokenization

- **Command:** `npm run visuals:generate -- --course nlp-py --day 1`, then `npm run visuals:check -- --course nlp-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D01] visuals for nlp-py day 1`

Key	Part title
nlp-py:1:0	Why text needs cleaning
nlp-py:1:1	Unicode and normalisation
nlp-py:1:2	Removing accents
nlp-py:1:3	Tokenising with regular expressions
nlp-py:1:4	Stopwords
nlp-py:1:5	Practice time: the full pipeline

C-nlp-py-D02 · Day 2: Morphological Analysis: Heuristic Stemming vs POS-Aware Lemmatization

- **Command:** `npm run visuals:generate -- --course nlp-py --day 2`, then `npm run visuals:check -- --course nlp-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D02] visuals for nlp-py day 2`

Key	Part title
nlp-py:2:0	Many forms, one meaning
nlp-py:2:1	Stemming with suffix rules
nlp-py:2:2	Parts of speech
nlp-py:2:3	Dictionary lemmatisation
nlp-py:2:4	Choosing between them
nlp-py:2:5	Practice time: stem and lemmatise

C-nlp-py-D03 · Day 3: N-Gram Language Models: Maximum Likelihood & Laplace Smoothing

- **Command:** `npm run visuals:generate -- --course nlp-py --day 3`, then `npm run visuals:check -- --course nlp-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-03.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-nlp-py-D03] visuals for nlp-py day 3

Key	Part title
nlp-py:3:0	What a language model does
nlp-py:3:1	Maximum likelihood estimates
nlp-py:3:2	The zero problem
nlp-py:3:3	Laplace (add-one) smoothing
nlp-py:3:4	Evaluating with perplexity
nlp-py:3:5	Practice time: smooth and evaluate

C-nlp-py-D04 · Day 4: Vector Space Models: Bag-of-Words & TF-IDF Weighting

- **Command:** `npm run visuals:generate -- --course nlp-py --day 4`, then `npm run visuals:check -- --course nlp-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-nlp-py-D04] visuals for nlp-py day 4

Key	Part title
nlp-py:4:0	Documents as vectors
nlp-py:4:1	Bag of words
nlp-py:4:2	Term frequency
nlp-py:4:3	Inverse document frequency
nlp-py:4:4	TF-IDF
nlp-py:4:5	Practice time: weights and vectors

C-nlp-py-D05 · Day 5: ★ MILESTONE 1: Complete Text Normalization, TF-IDF & Vector Space Search Engine

- **Command:** `npm run visuals:generate -- --course nlp-py --day 5`, then `npm run visuals:check -- --course nlp-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-nlp-py-D05] visuals for nlp-py day 5

Key	Part title
nlp-py:5:0	How a search engine works
nlp-py:5:1	The inverted index
nlp-py:5:2	Scoring documents for a query
nlp-py:5:3	Putting the pipeline together
nlp-py:5:4	Evaluating search quality
nlp-py:5:5	Milestone practice: build the engine

R-nlp-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course nlp-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-nlp-py-1-5] screenshots for nlp-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-nlp-py-D06 · Day 6: Vector Similarity & Semantic Document Search: Cosine Similarity

- **Command:** `npm run visuals:generate -- --course nlp-py --day 6`, then `npm run visuals:check -- --course nlp-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D06] visuals for nlp-py day 6`

Key	Part title
nlp-py:6:0	Similarity as geometry
nlp-py:6:1	The dot product and vector length
nlp-py:6:2	Writing cosine similarity safely
nlp-py:6:3	Ranking by similarity
nlp-py:6:4	TF-IDF vectors and semantic limits
nlp-py:6:5	Practice time: similarity and ranking

C-nlp-py-D07 · Day 7: Distributed Representations: Word2Vec Skip-Gram & CBOW Architectures

- **Command:** `npm run visuals:generate -- --course nlp-py --day 7`, then `npm run visuals:check -- --course nlp-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D07] visuals for nlp-py day 7`

Key	Part title
nlp-py:7:0	You shall know a word by the company it keeps
nlp-py:7:1	Skip-gram and CBOW
nlp-py:7:2	Similarity in embedding space
nlp-py:7:3	Analogies with vector arithmetic
nlp-py:7:4	Finding the nearest word
nlp-py:7:5	Practice time: analogies

C-nlp-py-D08 · Day 8: Subword Embeddings: FastText & Out-Of-Vocabulary (OOV) Resilience

- **Command:** `npm run visuals:generate -- --course nlp-py --day 8`, then `npm run visuals:check -- --course nlp-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D08] visuals for nlp-py day 8`

Key	Part title
nlp-py:8:0	The out-of-vocabulary problem
nlp-py:8:1	Character n-grams
nlp-py:8:2	Vectors from n-grams
nlp-py:8:3	Vectors for unseen words
nlp-py:8:4	Subwords everywhere
nlp-py:8:5	Practice time: n-grams and OOV vectors

C-nlp-py-D09 · Day 9: Global Vectors for Word Representation: GloVe Co-Occurrence Matrix Factorization

- **Command:** `npm run visuals:generate -- --course nlp-py --day 9`, then `npm run visuals:check -- --course nlp-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D09] visuals for nlp-py day 9`

Key	Part title
nlp-py:9:0	Counting co-occurrences
nlp-py:9:1	Ratios carry meaning
nlp-py:9:2	The weighting function
nlp-py:9:3	GloVe versus Word2Vec
nlp-py:9:4	Using pre-trained embeddings
nlp-py:9:5	Practice time: weights and counts

C-nlp-py-D10 · Day 10: Part-of-Speech Tagging with Hidden Markov Models: Viterbi Trellis Algorithm

- **Command:** `npm run visuals:generate -- --course nlp-py --day 10`, then `npm run visuals:check -- --course nlp-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D10] visuals for nlp-py day 10`

Key	Part title
nlp-py:10:0	Tagging is a sequence problem
nlp-py:10:1	The hidden Markov model

Key	Part title
nlp-py:10:2	Why brute force fails
nlp-py:10:3	The Viterbi algorithm
nlp-py:10:4	From HMMs to modern taggers
nlp-py:10:5	Practice time: Viterbi

R-nlp-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course nlp-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-nlp-py-6-10] screenshots for nlp-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-nlp-py-D11 · Day 11: Named Entity Recognition (NER): BIO Scheme & Sequence Chunking

- **Command:** `npm run visuals:generate -- --course nlp-py --day 11`, then `npm run visuals:check -- --course nlp-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D11] visuals for nlp-py day 11`

Key	Part title
nlp-py:11:0	What named entity recognition does
nlp-py:11:1	The BIO scheme
nlp-py:11:2	Valid and invalid tag sequences
nlp-py:11:3	From tags back to entities
nlp-py:11:4	Evaluating NER
nlp-py:11:5	Practice time: validate and extract

C-nlp-py-D12 · Day 12: Sentiment Analysis & Text Classification: Naive Bayes Log-Likelihood

- **Command:** `npm run visuals:generate -- --course nlp-py --day 12`, then `npm run visuals:check -- --course nlp-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D12] visuals for nlp-py day 12`

Key	Part title
nlp-py:12:0	Text classification
nlp-py:12:1	Bayes' rule and the naive assumption
nlp-py:12:2	Training: counting words per class
nlp-py:12:3	Working in log space

Key	Part title
nlp-py:12:4	Classifying and evaluating
nlp-py:12:5	Practice time: score and classify

C-nlp-py-D13 · Day 13: Recurrent Neural Networks (RNNs): Hidden State Recurrence & Vanishing Gradients

- **Command:** `npm run visuals:generate -- --course nlp-py --day 13`, then `npm run visuals:check -- --course nlp-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D13] visuals for nlp-py day 13`

Key	Part title
nlp-py:13:0	Why order and memory matter
nlp-py:13:1	The RNN update
nlp-py:13:2	Running an RNN over a sequence
nlp-py:13:3	Vanishing and exploding gradients
nlp-py:13:4	Gradient clipping and what comes next
nlp-py:13:5	Practice time: RNN steps and gradients

C-nlp-py-D14 · Day 14: Gated Memory Cells: Long Short-Term Memory (LSTM) & GRU Networks

- **Command:** `npm run visuals:generate -- --course nlp-py --day 14`, then `npm run visuals:check -- --course nlp-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D14] visuals for nlp-py day 14`

Key	Part title
nlp-py:14:0	Gates: learned valves
nlp-py:14:1	The forget and input gates
nlp-py:14:2	The output gate and hidden state
nlp-py:14:3	The GRU: fewer gates
nlp-py:14:4	Why gates fix vanishing gradients
nlp-py:14:5	Practice time: cells and updates

C-nlp-py-D15 · Day 15: ★ MILESTONE 2: Complete Word2Vec Embeddings, Viterbi POS Tagger & Bidirectional LSTM Classifier

- **Command:** `npm run visuals:generate -- --course nlp-py --day 15`, then `npm run visuals:check -- --course nlp-py --day 15`

- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D15] visuals for nlp-py day 15`

Key	Part title
nlp-py:15:0	The milestone pipeline
nlp-py:15:1	Sentence embeddings by averaging
nlp-py:15:2	Bidirectional reading
nlp-py:15:3	The BiLSTM tagger
nlp-py:15:4	The BiLSTM classifier and its limits
nlp-py:15:5	Milestone practice: connect the pieces

R-nlp-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course nlp-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-nlp-py-11-15] screenshots for nlp-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-nlp-py-D16 · Day 16: Sequence-to-Sequence (Seq2Seq) Architecture: Encoder-Decoder & Teacher Forcing

- **Command:** `npm run visuals:generate -- --course nlp-py --day 16`, then `npm run visuals:check -- --course nlp-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D16] visuals for nlp-py day 16`

Key	Part title
nlp-py:16:0	Sequence in, sequence out
nlp-py:16:1	The context vector bottleneck
nlp-py:16:2	Decoding one token at a time
nlp-py:16:3	Teacher forcing
nlp-py:16:4	Scheduled sampling
nlp-py:16:5	Practice time: schedules and inputs

C-nlp-py-D17 · Day 17: Attention Mechanisms: Bahdanau Additive & Luong Multiplicative Alignment

- **Command:** `npm run visuals:generate -- --course nlp-py --day 17`, then `npm run visuals:check -- --course nlp-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D17] visuals for nlp-py day 17`

Key	Part title
nlp-py:17:0	Looking back at the whole input
nlp-py:17:1	Scores to weights: softmax
nlp-py:17:2	The context vector
nlp-py:17:3	Bahdanau and Luong scoring
nlp-py:17:4	Attention everywhere
nlp-py:17:5	Practice time: softmax and context

C-nlp-py-D18 · Day 18: The Transformer Architecture: Scaled Dot-Product Self-Attention

- **Command:** `npm run visuals:generate -- --course nlp-py --day 18`, then `npm run visuals:check -- --course nlp-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D18] visuals for nlp-py day 18`

Key	Part title
nlp-py:18:0	Attention is all you need
nlp-py:18:1	Queries, keys and values
nlp-py:18:2	Scaling by the square root of d_k
nlp-py:18:3	Scaled dot-product attention in full
nlp-py:18:4	The encoder block
nlp-py:18:5	Practice time: scores and attention

C-nlp-py-D19 · Day 19: Multi-Head Self-Attention: Representation Subspaces & Linear Projections

- **Command:** `npm run visuals:generate -- --course nlp-py --day 19`, then `npm run visuals:check -- --course nlp-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D19] visuals for nlp-py day 19`

Key	Part title
nlp-py:19:0	Why more than one head
nlp-py:19:1	Splitting the model dimension
nlp-py:19:2	Splitting and merging vectors
nlp-py:19:3	Running the heads
nlp-py:19:4	What heads learn, and efficient variants
nlp-py:19:5	Practice time: dimensions and heads

C-nlp-py-D20 · Day 20: Positional Encoding: Sinusoidal Frequencies & Rotary Embeddings (RoPE)

- **Command:** `npm run visuals:generate -- --course nlp-py --day 20`, then `npm run visuals:check -- --course nlp-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D20] visuals for nlp-py day 20`

Key	Part title
nlp-py:20:0	Attention is order-blind
nlp-py:20:1	Sinusoidal positional encoding
nlp-py:20:2	Adding positions to embeddings
nlp-py:20:3	Rotary position embeddings (RoPE)
nlp-py:20:4	Choosing a position method
nlp-py:20:5	Practice time: encodings and rotations

R-nlp-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course nlp-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-nlp-py-16-20] screenshots for nlp-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-nlp-py-D21 · Day 21: ★ MILESTONE 3: Complete Scaled Dot-Product Self-Attention & Positional Encoding Engine

- **Command:** `npm run visuals:generate -- --course nlp-py --day 21`, then `npm run visuals:check -- --course nlp-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D21] visuals for nlp-py day 21`

Key	Part title
nlp-py:21:0	The encoder layer, step by step
nlp-py:21:1	Adding positional encodings to a sequence
nlp-py:21:2	Layer normalisation
nlp-py:21:3	Residual connections in the layer
nlp-py:21:4	A complete tiny encoder layer
nlp-py:21:5	Milestone practice: positions and normalisation

C-nlp-py-D22 · Day 22: Modern Subword Tokenization: Byte-Pair Encoding (BPE) & WordPiece

- **Command:** `npm run visuals:generate -- --course nlp-py --day 22`, then `npm run visuals:check -- --course nlp-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D22] visuals for nlp-py day 22`

Key	Part title
nlp-py:22:0	Words, characters or subwords?
nlp-py:22:1	How byte-pair encoding learns
nlp-py:22:2	Applying a merge
nlp-py:22:3	Tokenising new text
nlp-py:22:4	WordPiece and SentencePiece
nlp-py:22:5	Practice time: merges

C-nlp-py-D23 · Day 23: BERT: Bidirectional Encoder Representations from Transformers

- **Command:** `npm run visuals:generate -- --course nlp-py --day 23`, then `npm run visuals:check -- --course nlp-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D23] visuals for nlp-py day 23`

Key	Part title
nlp-py:23:0	Contextual embeddings
nlp-py:23:1	Masked language modelling
nlp-py:23:2	Counting the masking plan
nlp-py:23:3	BERT's input format
nlp-py:23:4	Fine-tuning BERT
nlp-py:23:5	Practice time: masking and inputs

C-nlp-py-D24 · Day 24: GPT: Autoregressive Language Modeling & Causal Masking

- **Command:** `npm run visuals:generate -- --course nlp-py --day 24`, then `npm run visuals:check -- --course nlp-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D24] visuals for nlp-py day 24`

Key	Part title
nlp-py:24:0	Predicting the next token
nlp-py:24:1	The causal mask
nlp-py:24:2	Masked attention in action

Key	Part title
nlp-py:24:3	Generation and the KV cache
nlp-py:24:4	Encoders, decoders and both
nlp-py:24:5	Practice time: masks and caches

C-nlp-py-D25 · Day 25: Extractive Question Answering: SQuAD Span Prediction

- **Command:** `npm run visuals:generate -- --course nlp-py --day 25`, then `npm run visuals:check -- --course nlp-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D25] visuals for nlp-py day 25`

Key	Part title
nlp-py:25:0	Extractive question answering
nlp-py:25:1	Start and end scores
nlp-py:25:2	Choosing the best valid span
nlp-py:25:3	Normalising answers
nlp-py:25:4	Exact match and F1
nlp-py:25:5	Practice time: spans and matches

R-nlp-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course nlp-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-nlp-py-21-25] screenshots for nlp-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-nlp-py-D26 · Day 26: Dense Retrieval vs Cross-Encoder Re-Ranking: Two-Stage Information Retrieval

- **Command:** `npm run visuals:generate -- --course nlp-py --day 26`, then `npm run visuals:check -- --course nlp-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D26] visuals for nlp-py day 26`

Key	Part title
nlp-py:26:0	Sparse and dense retrieval
nlp-py:26:1	Bi-encoders and approximate nearest neighbours
nlp-py:26:2	Cross-encoders
nlp-py:26:3	Designing the funnel
nlp-py:26:4	Re-ranking stably

Key	Part title
nlp-py:26:5	Practice time: funnel and re-rank

C-nlp-py-D27 · Day 27: Sequence Generation Decoding: Temperature, Top-k & Nucleus (Top-p) Sampling

- **Command:** `npm run visuals:generate -- --course nlp-py --day 27`, then `npm run visuals:check -- --course nlp-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D27] visuals for nlp-py day 27`

Key	Part title
nlp-py:27:0	From logits to a choice
nlp-py:27:1	Temperature
nlp-py:27:2	Top-k sampling
nlp-py:27:3	Nucleus (top-p) sampling
nlp-py:27:4	Choosing decoding settings
nlp-py:27:5	Practice time: nucleus and temperature

C-nlp-py-D28 · Day 28: NLP Evaluation Metrics: BLEU, ROUGE & Exact Match (EM)

- **Command:** `npm run visuals:generate -- --course nlp-py --day 28`, then `npm run visuals:check -- --course nlp-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D28] visuals for nlp-py day 28`

Key	Part title
nlp-py:28:0	Why automatic metrics
nlp-py:28:1	BLEU: clipped n-gram precision
nlp-py:28:2	The brevity penalty
nlp-py:28:3	ROUGE for summaries
nlp-py:28:4	The limits of overlap metrics
nlp-py:28:5	Practice time: penalties and recall

C-nlp-py-D29 · Day 29: Parameter-Efficient Fine-Tuning (PEFT): Low-Rank Adaptation (LoRA)

- **Command:** `npm run visuals:generate -- --course nlp-py --day 29`, then `npm run visuals:check -- --course nlp-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D29] visuals for nlp-py day 29`

Key	Part title
nlp-py:29:0	Why full fine-tuning is expensive
nlp-py:29:1	Low-rank matrices
nlp-py:29:2	How LoRA works
nlp-py:29:3	Merging LoRA into the model
nlp-py:29:4	Choosing rank and what to adapt
nlp-py:29:5	Practice time: savings and merging

C-nlp-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Sovereign Natural Language Processing & LLM Infrastructure Engine

- **Command:** `npm run visuals:generate -- --course nlp-py --day 30`, then `npm run visuals:check -- --course nlp-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/nlp-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-nlp-py-D30] visuals for nlp-py day 30`

Key	Part title
nlp-py:30:0	The capstone architecture
nlp-py:30:1	Preparing passages
nlp-py:30:2	Retrieving passages
nlp-py:30:3	Answering and grounding
nlp-py:30:4	Evaluating and auditing the system
nlp-py:30:5	Capstone practice and what comes next

R-nlp-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course nlp-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-nlp-py-26-30] screenshots for nlp-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-nlp-py · Release: students can see NLP in Python (Month 7) pictures

- Add `'nlp-py'` to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-nlp-py] release visuals for nlp-py`.

Course 8: Quant Systems Python (Month 8) (`quant-py`)

C-quant-py-D01 · Day 1: Quantitative Engineering & Electronic Trading Foundations

- **Command:** `npm run visuals:generate -- --course quant-py --day 1`, then `npm run visuals:check -- --course quant-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-01.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-quant-py-D01] visuals for quant-py day 1

Key	Part title
quant-py:1:0	What an exchange does
quant-py:1:1	Bid, ask and spread
quant-py:1:2	The midpoint and basis points
quant-py:1:3	Ticks and the price grid
quant-py:1:4	Liquidity and market quality
quant-py:1:5	Practice time: quotes and ticks

C-quant-py-D02 · Day 2: Limit Order Book (LOB) Architecture

- **Command:** `npm run visuals:generate -- --course quant-py --day 2`, then `npm run visuals:check -- --course quant-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D02] visuals for quant-py day 2

Key	Part title
quant-py:2:0	Limit orders and market orders
quant-py:2:1	Price-time priority
quant-py:2:2	Inserting an order
quant-py:2:3	Price levels and depth
quant-py:2:4	Cancels, modifies and data structures
quant-py:2:5	Practice time: build the book

C-quant-py-D03 · Day 3: Order Book Matching Engine Implementation

- **Command:** `npm run visuals:generate -- --course quant-py --day 3`, then `npm run visuals:check -- --course quant-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D03] visuals for quant-py day 3

Key	Part title
quant-py:3:0	When does an order trade?
quant-py:3:1	Walking the book
quant-py:3:2	Remainders rest in the book
quant-py:3:3	Average fill price
quant-py:3:4	Determinism and testing
quant-py:3:5	Practice time: match orders

C-quant-py-D04 · Day 4: Algorithmic Execution: VWAP & TWAP Strategies

- **Command:** `npm run visuals:generate -- --course quant-py --day 4`, then `npm run visuals:check -- --course quant-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D04] visuals for quant-py day 4`

Key	Part title
quant-py:4:0	Why split an order?
quant-py:4:1	The VWAP benchmark
quant-py:4:2	Building a VWAP schedule
quant-py:4:3	TWAP: equal slices over time
quant-py:4:4	Choosing and measuring an algorithm
quant-py:4:5	Practice time: schedules

C-quant-py-D05 · Day 5: ★ MILESTONE 1: Complete Limit Order Book & Matching Engine Kernel

- **Command:** `npm run visuals:generate -- --course quant-py --day 5`, then `npm run visuals:check -- --course quant-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D05] visuals for quant-py day 5`

Key	Part title
quant-py:5:0	The exchange core
quant-py:5:1	Processing a stream of orders
quant-py:5:2	Measuring throughput
quant-py:5:3	Latency distributions
quant-py:5:4	Market data and recovery
quant-py:5:5	Milestone practice: run and measure

R-quant-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course quant-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-quant-py-1-5] screenshots for quant-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-quant-py-D06 · Day 6: Market Impact & Slippage Models: Almgren-Chriss Framework

- **Command:** `npm run visuals:generate -- --course quant-py --day 6`, then `npm run visuals:check -- --course quant-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-06.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-quant-py-D06] visuals for quant-py day 6

Key	Part title
quant-py:6:0	What market impact is
quant-py:6:1	The Almgren-Chriss model
quant-py:6:2	Speed versus risk
quant-py:6:3	Slippage
quant-py:6:4	Transaction cost analysis
quant-py:6:5	Practice time: costs and slippage

C-quant-py-D07 · Day 7: Order Book Imbalance (OBI) & Micro-Price Estimation

- **Command:** `npm run visuals:generate -- --course quant-py --day 7`, then `npm run visuals:check -- --course quant-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D07] visuals for quant-py day 7

Key	Part title
quant-py:7:0	Imbalance at the top of the book
quant-py:7:1	The micro-price
quant-py:7:2	From imbalance to a signal
quant-py:7:3	Testing a signal honestly
quant-py:7:4	Deeper book signals
quant-py:7:5	Practice time: micro-price and signal

C-quant-py-D08 · Day 8: High-Frequency Market Making: Avellaneda-Stoikov Model

- **Command:** `npm run visuals:generate -- --course quant-py --day 8`, then `npm run visuals:check -- --course quant-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D08] visuals for quant-py day 8

Key	Part title
quant-py:8:0	What a market maker does
quant-py:8:1	The reservation price
quant-py:8:2	Skewed quotes
quant-py:8:3	Inventory limits
quant-py:8:4	Adverse selection and speed
quant-py:8:5	Practice time: quotes and limits

C-quant-py-D09 · Day 9: Financial Information eXchange (FIX 4.4) Protocol & FAST Compression

- **Command:** `npm run visuals:generate -- --course quant-py --day 9`, then `npm run visuals:check -- --course quant-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D09] visuals for quant-py day 9`

Key	Part title
quant-py:9:0	The Financial Information eXchange protocol
quant-py:9:1	Header, body and trailer
quant-py:9:2	Validating incoming messages
quant-py:9:3	Sessions, logon and sequence numbers
quant-py:9:4	Execution reports and order state
quant-py:9:5	Practice time: build and validate

C-quant-py-D10 · Day 10: NASDAQ TotalView-ITCH 5.0 & OUCH Protocols: Direct Binary Market Feeds

- **Command:** `npm run visuals:generate -- --course quant-py --day 10`, then `npm run visuals:check -- --course quant-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D10] visuals for quant-py day 10`

Key	Part title
quant-py:10:0	Why binary feeds
quant-py:10:1	Bytes, endianness and struct
quant-py:10:2	Decoding an Add Order message
quant-py:10:3	Fixed-point prices
quant-py:10:4	Rebuilding the book from a feed
quant-py:10:5	Practice time: decode and convert

R-quant-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course quant-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-quant-py-6-10] screenshots for quant-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-quant-py-D11 · Day 11: Kernel Bypass Networking: Solarflare Onload & DPDK Zero-Copy

- **Command:** `npm run visuals:generate -- --course quant-py --day 11`, then `npm run visuals:check -- --course quant-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D11] visuals for quant-py day 11`

Key	Part title
quant-py:11:0	The path of a packet
quant-py:11:1	Kernel bypass
quant-py:11:2	Ring buffers between the NIC and the program
quant-py:11:3	Zero copy
quant-py:11:4	CPU pinning and jitter
quant-py:11:5	Practice time: rings and savings

C-quant-py-D12 · Day 12: Lock-Free Ring Buffers: Single-Producer Single-Consumer (SPSC) Architecture

- **Command:** `npm run visuals:generate -- --course quant-py --day 12`, then `npm run visuals:check -- --course quant-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D12] visuals for quant-py day 12`

Key	Part title
quant-py:12:0	Threads and the cost of locks
quant-py:12:1	The SPSC ring queue
quant-py:12:2	Why powers of two
quant-py:12:3	Full, empty and back-pressure
quant-py:12:4	Testing concurrent code
quant-py:12:5	Practice time: build the queue

C-quant-py-D13 · Day 13: CPU Cacheline Alignment & False Sharing Elimination in C++

- **Command:** `npm run visuals:generate -- --course quant-py --day 13`, then `npm run visuals:check -- --course quant-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D13] visuals for quant-py day 13`

Key	Part title
quant-py:13:0	Caches and cache lines
quant-py:13:1	Cache coherence

Key	Part title
quant-py:13:2	False sharing
quant-py:13:3	Padding and alignment
quant-py:13:4	Data layout for speed
quant-py:13:5	Practice time: audit and pad

C-quant-py-D14 · Day 14: SIMD Vectorization (AVX-512) for Pricing & Risk Kernels

- **Command:** `npm run visuals:generate -- --course quant-py --day 14`, then `npm run visuals:check -- --course quant-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D14] visuals for quant-py day 14`

Key	Part title
quant-py:14:0	Single instruction, multiple data
quant-py:14:1	Chunks and the scalar tail
quant-py:14:2	Amdahl's law
quant-py:14:3	What vectorises well in finance
quant-py:14:4	Measuring before optimising
quant-py:14:5	Practice time: vectors and speedups

C-quant-py-D15 · Day 15: ★ MILESTONE 2: Complete Ultra-Low-Latency Order Messaging & Concurrency Engine

- **Command:** `npm run visuals:generate -- --course quant-py --day 15`, then `npm run visuals:check -- --course quant-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D15] visuals for quant-py day 15`

Key	Part title
quant-py:15:0	The tick-to-trade pipeline
quant-py:15:1	Bursts and bounded queues
quant-py:15:2	Latency percentiles
quant-py:15:3	Where latency hides
quant-py:15:4	Observability without slowing down
quant-py:15:5	Milestone practice: gateway and percentiles

R-quant-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course quant-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`

- **Commit message:** [task:R-quant-py-11-15] screenshots for quant-py days 11-15
- **Wait** for Claude's "approved" before the next day task.

C-quant-py-D16 · Day 16: Option Pricing & Greeks: Black-Scholes-Merton (BSM) Analytical Engine

- **Command:** `npm run visuals:generate -- --course quant-py --day 16`, then `npm run visuals:check -- --course quant-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D16] visuals for quant-py day 16

Key	Part title
quant-py:16:0	What an option is
quant-py:16:1	The Black-Scholes-Merton idea
quant-py:16:2	The formula
quant-py:16:3	The Greeks: delta and gamma
quant-py:16:4	Put-call parity
quant-py:16:5	Practice time: price and check

C-quant-py-D17 · Day 17: Implied Volatility Surface: Newton-Raphson Solver

- **Command:** `npm run visuals:generate -- --course quant-py --day 17`, then `npm run visuals:check -- --course quant-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D17] visuals for quant-py day 17

Key	Part title
quant-py:17:0	Implied volatility
quant-py:17:1	Vega
quant-py:17:2	Newton-Raphson
quant-py:17:3	When there is no answer
quant-py:17:4	The smile and the surface
quant-py:17:5	Practice time: solve for volatility

C-quant-py-D18 · Day 18: Risk Management: Parametric & Historical Value at Risk (VaR)

- **Command:** `npm run visuals:generate -- --course quant-py --day 18`, then `npm run visuals:check -- --course quant-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D18] visuals for quant-py day 18

Key	Part title
quant-py:18:0	What Value at Risk means
quant-py:18:1	Parametric VaR
quant-py:18:2	Historical VaR
quant-py:18:3	Scaling to longer horizons
quant-py:18:4	Limits of VaR
quant-py:18:5	Practice time: VaR calculators

C-quant-py-D19 · Day 19: Tail Risk & Expected Shortfall (CVaR / Conditional VaR)

- **Command:** `npm run visuals:generate -- --course quant-py --day 19`, then `npm run visuals:check -- --course quant-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D19] visuals for quant-py day 19`

Key	Part title
quant-py:19:0	Expected Shortfall
quant-py:19:1	Coherent risk measures
quant-py:19:2	Diversification in numbers
quant-py:19:3	Stress testing
quant-py:19:4	Reporting risk
quant-py:19:5	Practice time: tails and diversification

C-quant-py-D20 · Day 20: Portfolio Optimization: Modern Portfolio Theory (Markowitz Frontier)

- **Command:** `npm run visuals:generate -- --course quant-py --day 20`, then `npm run visuals:check -- --course quant-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D20] visuals for quant-py day 20`

Key	Part title
quant-py:20:0	Return and risk of a portfolio
quant-py:20:1	The Sharpe ratio
quant-py:20:2	The minimum-variance portfolio
quant-py:20:3	The efficient frontier
quant-py:20:4	Estimation error and practical limits
quant-py:20:5	Practice time: build a portfolio

R-quant-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course quant-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-quant-py-16-20] screenshots for quant-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-quant-py-D21 · Day 21: ★ MILESTONE 3: Complete Quantitative Pricing, Greeks & Risk Engine

- **Command:** `npm run visuals:generate -- --course quant-py --day 21`, then `npm run visuals:check -- --course quant-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D21] visuals for quant-py day 21`

Key	Part title
quant-py:21:0	Aggregating Greeks
quant-py:21:1	Scenario revaluation
quant-py:21:2	VaR and ES from scenarios
quant-py:21:3	Delta hedging
quant-py:21:4	Running the engine continuously
quant-py:21:5	Milestone practice: engine and hedge

C-quant-py-D22 · Day 22: High-Frequency Alpha Signals & Statistical Arbitrage

- **Command:** `npm run visuals:generate -- --course quant-py --day 22`, then `npm run visuals:check -- --course quant-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D22] visuals for quant-py day 22`

Key	Part title
quant-py:22:0	What alpha means
quant-py:22:1	Pairs and the spread
quant-py:22:2	Z-score signals
quant-py:22:3	Mean reversion speed: half-life
quant-py:22:4	Validating and combining signals
quant-py:22:5	Practice time: pairs and half-life

C-quant-py-D23 · Day 23: Smart Order Routing (SOR) & Best Execution Algorithms

- **Command:** `npm run visuals:generate -- --course quant-py --day 23`, then `npm run visuals:check -- --course quant-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-23.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-quant-py-D23] visuals for quant-py day 23

Key	Part title
quant-py:23:0	Fragmented markets
quant-py:23:1	Fees, rebates and the maker-taker model
quant-py:23:2	The smart order router
quant-py:23:3	Dark pools and hidden liquidity
quant-py:23:4	Measuring execution quality
quant-py:23:5	Practice time: route and account

C-quant-py-D24 · Day 24: Exchange Colocation & Cross-Connect Physics

- **Command:** `npm run visuals:generate -- --course quant-py --day 24`, then `npm run visuals:check -- --course quant-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D24] visuals for quant-py day 24

Key	Part title
quant-py:24:0	The speed of light in fibre
quant-py:24:1	Colocation
quant-py:24:2	Cross-connects and network design
quant-py:24:3	Fibre versus air
quant-py:24:4	The latency arms race and its limits
quant-py:24:5	Practice time: physics calculators

C-quant-py-D25 · Day 25: Microwave, Millimeter-Wave & Shortwave Radio Trading Networks

- **Command:** `npm run visuals:generate -- --course quant-py --day 25`, then `npm run visuals:check -- --course quant-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-quant-py-D25] visuals for quant-py day 25

Key	Part title
quant-py:25:0	Line of sight and towers
quant-py:25:1	Link latency
quant-py:25:2	Rain fade and availability
quant-py:25:3	Millimetre wave and laser links
quant-py:25:4	Shortwave across oceans
quant-py:25:5	Practice time: link and towers

R-quant-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course quant-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-quant-py-21-25] screenshots for quant-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-quant-py-D26 · Day 26: Backtesting Pitfalls: Lookahead Bias & Overfitting Elimination

- **Command:** `npm run visuals:generate -- --course quant-py --day 26`, then `npm run visuals:check -- --course quant-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D26] visuals for quant-py day 26`

Key	Part title
quant-py:26:0	Why backtests lie
quant-py:26:1	Lookahead bias
quant-py:26:2	Survivorship bias
quant-py:26:3	Overfitting and multiple testing
quant-py:26:4	Honest research practice
quant-py:26:5	Practice time: audit and haircut

C-quant-py-D27 · Day 27: Pre-Trade Risk Controls & Fat-Finger Circuit Breakers

- **Command:** `npm run visuals:generate -- --course quant-py --day 27`, then `npm run visuals:check -- --course quant-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D27] visuals for quant-py day 27`

Key	Part title
quant-py:27:0	Lessons from disasters
quant-py:27:1	Order-level checks
quant-py:27:2	Aggregate limits and kill switches
quant-py:27:3	Where the checks live
quant-py:27:4	Testing the safety net
quant-py:27:5	Practice time: gate and switch

C-quant-py-D28 · Day 28: Crypto Derivatives: Perpetual Futures & Funding Rate Arbitrage

- **Command:** `npm run visuals:generate -- --course quant-py --day 28`, then `npm run visuals:check -- --course quant-py --day 28`

- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D28] visuals for quant-py day 28`

Key	Part title
quant-py:28:0	Perpetual futures
quant-py:28:1	Funding payments
quant-py:28:2	Leverage and margin
quant-py:28:3	Funding rate arbitrage
quant-py:28:4	Liquidation cascades
quant-py:28:5	Practice time: funding and liquidation

C-quant-py-D29 · Day 29: High-Frequency Trading Infrastructure: FPGA & ASIC Offloading

- **Command:** `npm run visuals:generate -- --course quant-py --day 29`, then `npm run visuals:check -- --course quant-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D29] visuals for quant-py day 29`

Key	Part title
quant-py:29:0	Why hardware
quant-py:29:1	Clocks and pipelines
quant-py:29:2	Resources and fitting the design
quant-py:29:3	Hardware and software together
quant-py:29:4	Costs and trade-offs
quant-py:29:5	Practice time: pipeline and resources

C-quant-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Ultra-Low-Latency Quantitative Trading & Market Making System

- **Command:** `npm run visuals:generate -- --course quant-py --day 30`, then `npm run visuals:check -- --course quant-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/quant-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-quant-py-D30] visuals for quant-py day 30`

Key	Part title
quant-py:30:0	The tick handler
quant-py:30:1	Inventory skew in action
quant-py:30:2	Risk gating each quote
quant-py:30:3	Production readiness

Key	Part title
quant-py:30:4	Going live safely
quant-py:30:5	Capstone practice: tick handler and readiness

R-quant-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course quant-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-quant-py-26-30] screenshots for quant-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-quant-py · Release: students can see Quant Systems Python (Month 8) pictures

- Add 'quant-py' to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-quant-py] release visuals for quant-py`.

Course 9: AI Prompt Engineering Python (Month 9) (prompt-py)

C-prompt-py-D01 · Day 1: What is AI? — Prompts, Context, and Getting Useful Responses

- **Command:** `npm run visuals:generate -- --course prompt-py --day 1`, then `npm run visuals:check -- --course prompt-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D01] visuals for prompt-py day 1`

Key	Part title
prompt-py:1:0	What generative AI is
prompt-py:1:1	Tokens and next-word prediction
prompt-py:1:2	What a prompt is
prompt-py:1:3	Checking prompts with code
prompt-py:1:4	Strengths and limits
prompt-py:1:5	Practice time: check and build prompts

C-prompt-py-D02 · Day 2: System Prompts & Persona Role Framing: The C-R-E-A-T-E Framework

- **Command:** `npm run visuals:generate -- --course prompt-py --day 2`, then `npm run visuals:check -- --course prompt-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D02] visuals for prompt-py day 2`

Key	Part title
prompt-py:2:0	System prompts and user prompts
prompt-py:2:1	The C-R-E-A-T-E framework
prompt-py:2:2	Personas and roles
prompt-py:2:3	Negative constraints and guardrails
prompt-py:2:4	Auditing existing prompts
prompt-py:2:5	Practice time: assemble and audit

C-prompt-py-D03 · Day 3: In-Context Learning: Zero-Shot, One-Shot & Few-Shot Demonstration Pairs

- **Command:** `npm run visuals:generate -- --course prompt-py --day 3`, then `npm run visuals:check -- --course prompt-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D03] visuals for prompt-py day 3`

Key	Part title
prompt-py:3:0	In-context learning
prompt-py:3:1	Zero-shot, one-shot and few-shot
prompt-py:3:2	Formatting example pairs
prompt-py:3:3	Choosing examples automatically
prompt-py:3:4	Common few-shot mistakes
prompt-py:3:5	Practice time: build and select

C-prompt-py-D04 · Day 4: Chain-of-Thought (CoT) & Step-by-Step Deliberative Reasoning: Self-Consistency

- **Command:** `npm run visuals:generate -- --course prompt-py --day 4`, then `npm run visuals:check -- --course prompt-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D04] visuals for prompt-py day 4`

Key	Part title
prompt-py:4:0	Chain-of-thought prompting
prompt-py:4:1	Extracting the final answer
prompt-py:4:2	Self-consistency voting
prompt-py:4:3	Tree of thoughts
prompt-py:4:4	When reasoning prompts help
prompt-py:4:5	Practice time: vote and extract

C-prompt-py-D05 · Day 5: ★ MILESTONE 1: Complete AI Tokenomics, Persona Role Framing & Chain-of-Thought Prompting Engine

- **Command:** `npm run visuals:generate -- --course prompt-py --day 5`, then `npm run visuals:check -- --course prompt-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D05] visuals for prompt-py day 5`

Key	Part title
<code>prompt-py:5:0</code>	Tokens in practice
<code>prompt-py:5:1</code>	The context window
<code>prompt-py:5:2</code>	What AI costs
<code>prompt-py:5:3</code>	Trimming prompts
<code>prompt-py:5:4</code>	Putting the week together
<code>prompt-py:5:5</code>	Milestone practice: budget and cost

R-prompt-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course prompt-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-prompt-py-1-5] screenshots for prompt-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-prompt-py-D06 · Day 6: Decoding Hyperparameters: Temperature, Top-P (Nucleus) & Frequency Penalties

- **Command:** `npm run visuals:generate -- --course prompt-py --day 6`, then `npm run visuals:check -- --course prompt-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D06] visuals for prompt-py day 6`

Key	Part title
<code>prompt-py:6:0</code>	From scores to probabilities
<code>prompt-py:6:1</code>	Temperature
<code>prompt-py:6:2</code>	Top-p (nucleus) sampling
<code>prompt-py:6:3</code>	Frequency and presence penalties
<code>prompt-py:6:4</code>	Choosing settings for the task
<code>prompt-py:6:5</code>	Practice time: temperature and top-p

C-prompt-py-D07 · Day 7: Structured Data Generation: Enforcing Strict JSON Schemas & Function Calling

- **Command:** `npm run visuals:generate -- --course prompt-py --day 7`, then `npm run visuals:check -- --course prompt-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D07] visuals for prompt-py day 7`

Key	Part title
<code>prompt-py:7:0</code>	Why structured output
<code>prompt-py:7:1</code>	Describing the schema
<code>prompt-py:7:2</code>	Validating JSON output
<code>prompt-py:7:3</code>	Extracting JSON from chatty replies
<code>prompt-py:7:4</code>	Function calling and tools
<code>prompt-py:7:5</code>	Practice time: validate and extract

C-prompt-py-D08 · Day 8: Text Summarization & Distillation: Extractive vs Abstractive Executive Briefings

- **Command:** `npm run visuals:generate -- --course prompt-py --day 8`, then `npm run visuals:check -- --course prompt-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D08] visuals for prompt-py day 8`

Key	Part title
<code>prompt-py:8:0</code>	Two kinds of summary
<code>prompt-py:8:1</code>	A simple extractive summariser
<code>prompt-py:8:2</code>	Prompting for good summaries
<code>prompt-py:8:3</code>	Documents longer than the context window
<code>prompt-py:8:4</code>	Checking summary length and quality
<code>prompt-py:8:5</code>	Practice time: summarise and check

C-prompt-py-D09 · Day 9: Retrieval-Augmented Generation (RAG) for Everyday Users: Grounding & Citations

- **Command:** `npm run visuals:generate -- --course prompt-py --day 9`, then `npm run visuals:check -- --course prompt-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D09] visuals for prompt-py day 9`

Key	Part title
<code>prompt-py:9:0</code>	Why grounding matters
<code>prompt-py:9:1</code>	Chunking documents

Key	Part title
prompt-py:9:2	Retrieving relevant chunks
prompt-py:9:3	The grounded prompt
prompt-py:9:4	Checking citations
prompt-py:9:5	Practice time: retrieve and check

C-prompt-py-D10 · Day 10: AI-Powered Deep Web Research: Perplexity AI, Fact-Checking & Source Verification

- **Command:** `npm run visuals:generate -- --course prompt-py --day 10`, then `npm run visuals:check -- --course prompt-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D10] visuals for prompt-py day 10`

Key	Part title
prompt-py:10:0	AI search tools
prompt-py:10:1	Evaluating a source
prompt-py:10:2	Cross-checking claims
prompt-py:10:3	Common misinformation patterns
prompt-py:10:4	Writing up research
prompt-py:10:5	Practice time: score and cross-check

R-prompt-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course prompt-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-prompt-py-6-10] screenshots for prompt-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-prompt-py-D11 · Day 11: Prompt Chaining & Multi-Step Workflows: Decomposing Complex Tasks

- **Command:** `npm run visuals:generate -- --course prompt-py --day 11`, then `npm run visuals:check -- --course prompt-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D11] visuals for prompt-py day 11`

Key	Part title
prompt-py:11:0	Why chain prompts
prompt-py:11:1	Designing a chain
prompt-py:11:2	Running a chain safely
prompt-py:11:3	Decomposing instructions

Key	Part title
prompt-py:11:4	Chains with checks in between
prompt-py:11:5	Practice time: chain and split

C-prompt-py-D12 · Day 12: Professional Writing & Communication: Tone Shifting & Executive Memos

- **Command:** `npm run visuals:generate -- --course prompt-py --day 12`, then `npm run visuals:check -- --course prompt-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D12] visuals for prompt-py day 12`

Key	Part title
prompt-py:12:0	Tone and audience
prompt-py:12:1	Measuring formality
prompt-py:12:2	Executive memos
prompt-py:12:3	Emails with AI
prompt-py:12:4	Editing and proofreading
prompt-py:12:5	Practice time: tone and memos

C-prompt-py-D13 · Day 13: Creative Ideation & Brainstorming: SCAMPER Framework & Devil's Advocate

- **Command:** `npm run visuals:generate -- --course prompt-py --day 13`, then `npm run visuals:check -- --course prompt-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D13] visuals for prompt-py day 13`

Key	Part title
prompt-py:13:0	AI as a brainstorming partner
prompt-py:13:1	The SCAMPER technique
prompt-py:13:2	Devil's advocate and critics
prompt-py:13:3	Cleaning up the idea list
prompt-py:13:4	Selecting the best ideas
prompt-py:13:5	Practice time: SCAMPER and de-duplication

C-prompt-py-D14 · Day 14: Data Analysis with Code Interpreter: Automated Python Scripts & Visualizations

- **Command:** `npm run visuals:generate -- --course prompt-py --day 14`, then `npm run visuals:check -- --course prompt-py --day 14`

- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D14] visuals for prompt-py day 14`

Key	Part title
<code>prompt-py:14:0</code>	Code interpreter tools
<code>prompt-py:14:1</code>	Describing data
<code>prompt-py:14:2</code>	Grouping and totals
<code>prompt-py:14:3</code>	Common data problems
<code>prompt-py:14:4</code>	Checking AI analysis
<code>prompt-py:14:5</code>	Practice time: describe and group

C-prompt-py-D15 · Day 15: ★ MILESTONE 2: Complete Structured JSON, RAG Grounding, Prompt Chaining & Data Analysis Engine

- **Command:** `npm run visuals:generate -- --course prompt-py --day 15`, then `npm run visuals:check -- --course prompt-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D15] visuals for prompt-py day 15`

Key	Part title
<code>prompt-py:15:0</code>	The pipeline
<code>prompt-py:15:1</code>	Retrieval and the grounded prompt
<code>prompt-py:15:2</code>	Asking for a JSON answer
<code>prompt-py:15:3</code>	Parsing and validating the reply
<code>prompt-py:15:4</code>	Failure handling and user experience
<code>prompt-py:15:5</code>	Milestone practice: grounded prompt and reply parser

R-prompt-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course prompt-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-prompt-py-11-15] screenshots for prompt-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-prompt-py-D16 · Day 16: Multimodal AI & Vision Understanding: OCR, UI Inspection & Document Extraction

- **Command:** `npm run visuals:generate -- --course prompt-py --day 16`, then `npm run visuals:check -- --course prompt-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D16] visuals for prompt-py day 16`

Key	Part title
prompt-py:16:0	Multimodal AI
prompt-py:16:1	How OCR works
prompt-py:16:2	Extracting fields with regular expressions
prompt-py:16:3	Confidence and human review
prompt-py:16:4	Screenshots and user interfaces
prompt-py:16:5	Practice time: extract and review

C-prompt-py-D17 · Day 17: AI Image Generation & Diffusion Prompting: Midjourney & DALL-E 3 Mastery

- **Command:** `npm run visuals:generate -- --course prompt-py --day 17`, then `npm run visuals:check -- --course prompt-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D17] visuals for prompt-py day 17`

Key	Part title
prompt-py:17:0	How image generators work
prompt-py:17:1	Anatomy of an image prompt
prompt-py:17:2	Parameters: aspect ratio and negatives
prompt-py:17:3	Weighting parts of a prompt
prompt-py:17:4	Responsible image generation
prompt-py:17:5	Practice time: compose and parse

C-prompt-py-D18 · Day 18: Speech-to-Text & Audio AI: Whisper Transcription & Meeting Action Items

- **Command:** `npm run visuals:generate -- --course prompt-py --day 18`, then `npm run visuals:check -- --course prompt-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D18] visuals for prompt-py day 18`

Key	Part title
prompt-py:18:0	Speech-to-text
prompt-py:18:1	Cleaning transcripts
prompt-py:18:2	Finding action items
prompt-py:18:3	Measuring transcription quality
prompt-py:18:4	Text-to-speech and voice assistants
prompt-py:18:5	Practice time: action items and WER

C-prompt-py-D19 · Day 19: AI Ethics, Bias & Hallucination Mitigation: Fallbacks & Guardrails

- **Command:** `npm run visuals:generate -- --course prompt-py --day 19`, then `npm run visuals:check -- --course prompt-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D19] visuals for prompt-py day 19`

Key	Part title
<code>prompt-py:19:0</code>	Why models hallucinate
<code>prompt-py:19:1</code>	Checking claims against sources
<code>prompt-py:19:2</code>	Where bias comes from
<code>prompt-py:19:3</code>	Fallbacks and guardrails
<code>prompt-py:19:4</code>	A verification checklist
<code>prompt-py:19:5</code>	Practice time: unsupported claims and guardrails

C-prompt-py-D20 · Day 20: Privacy, Security & Prompt Injection Defense: Jailbreaks & PII Anonymization

- **Command:** `npm run visuals:generate -- --course prompt-py --day 20`, then `npm run visuals:check -- --course prompt-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D20] visuals for prompt-py day 20`

Key	Part title
<code>prompt-py:20:0</code>	Privacy basics
<code>prompt-py:20:1</code>	Redacting personal data with Python
<code>prompt-py:20:2</code>	Prompt injection
<code>prompt-py:20:3</code>	Jailbreaks and detection
<code>prompt-py:20:4</code>	Designing safer AI features
<code>prompt-py:20:5</code>	Practice time: redact and detect

R-prompt-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course prompt-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-prompt-py-16-20] screenshots for prompt-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-prompt-py-D21 · Day 21: ★ MILESTONE 3: Complete Multimodal Vision, Image Generation, Voice AI & Safety/Injection Defense Engine

- **Command:** `npm run visuals:generate -- --course prompt-py --day 21`, then `npm run visuals:check -- --course prompt-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D21] visuals for prompt-py day 21`

Key	Part title
<code>prompt-py:21:0</code>	A multimodal workflow
<code>prompt-py:21:1</code>	The safe request gate
<code>prompt-py:21:2</code>	Audit logs without personal data
<code>prompt-py:21:3</code>	Choosing the right modality
<code>prompt-py:21:4</code>	Testing the whole flow
<code>prompt-py:21:5</code>	Milestone practice: gate and audit log

C-prompt-py-D22 · Day 22: AI-Powered Coding Assistance: GitHub Copilot, Cursor & Unit Test Generation

- **Command:** `npm run visuals:generate -- --course prompt-py --day 22`, then `npm run visuals:check -- --course prompt-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D22] visuals for prompt-py day 22`

Key	Part title
<code>prompt-py:22:0</code>	AI coding assistants
<code>prompt-py:22:1</code>	Prompting for code
<code>prompt-py:22:2</code>	Generating tests from examples
<code>prompt-py:22:3</code>	Automated code review with ast
<code>prompt-py:22:4</code>	Reviewing AI code: a checklist
<code>prompt-py:22:5</code>	Practice time: tests and review

C-prompt-py-D23 · Day 23: Autonomous AI Agents & Tool Calling: ReAct Loops (Reason + Act + Observe)

- **Command:** `npm run visuals:generate -- --course prompt-py --day 23`, then `npm run visuals:check -- --course prompt-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D23] visuals for prompt-py day 23`

Key	Part title
<code>prompt-py:23:0</code>	What an agent is
<code>prompt-py:23:1</code>	The ReAct loop

Key	Part title
prompt-py:23:2	Parsing actions
prompt-py:23:3	Running the loop
prompt-py:23:4	Designing agents safely
prompt-py:23:5	Practice time: parse and run

C-prompt-py-D24 · Day 24: Workflow Automation with Zapier / Make & AI: Webhooks & Automated Pipelines

- **Command:** `npm run visuals:generate -- --course prompt-py --day 24`, then `npm run visuals:check -- --course prompt-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D24] visuals for prompt-py day 24`

Key	Part title
prompt-py:24:0	Triggers and actions
prompt-py:24:1	Webhooks
prompt-py:24:2	Routing events with rules
prompt-py:24:3	Retries and exponential backoff
prompt-py:24:4	Monitoring automations
prompt-py:24:5	Practice time: route and back off

C-prompt-py-D25 · Day 25: Custom GPTs & Knowledge Base Assistants: Knowledge Grounding & Action APIs

- **Command:** `npm run visuals:generate -- --course prompt-py --day 25`, then `npm run visuals:check -- --course prompt-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-prompt-py-D25] visuals for prompt-py day 25`

Key	Part title
prompt-py:25:0	What a custom assistant is
prompt-py:25:1	Writing assistant instructions
prompt-py:25:2	Answering from a FAQ
prompt-py:25:3	Actions: connecting to APIs
prompt-py:25:4	Validating assistant configurations
prompt-py:25:5	Practice time: FAQ answers and config checks

R-prompt-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course prompt-py --days 21-25`

- **Allowed files:** docs/visuals/shot_keys.txt
- **Commit message:** [task:R-prompt-py-21-25] screenshots for prompt-py days 21-25
- **Wait** for Claude's "approved" before the next day task.

C-prompt-py-D26 · Day 26: Everyday AI for Personal Productivity: Meal Planning, Travel & Habit Coaching

- **Command:** npm run visuals:generate -- --course prompt-py --day 26, then npm run visuals:check -- --course prompt-py --day 26
- **Allowed files:** src/lib/data/lessonVisuals/prompt-py/day-26.json, docs/visuals/py_cert_manifest.json
- **Commit message:** [task:C-prompt-py-D26] visuals for prompt-py day 26

Key	Part title
prompt-py:26:0	AI as a personal planner
prompt-py:26:1	Meals and budgets
prompt-py:26:2	Habits and streaks
prompt-py:26:3	Travel planning
prompt-py:26:4	Splitting costs fairly
prompt-py:26:5	Practice time: streaks and bills

C-prompt-py-D27 · Day 27: Domain-Specific AI Workflows: Legal, Medical, Marketing & Financial Analysis

- **Command:** npm run visuals:generate -- --course prompt-py --day 27, then npm run visuals:check -- --course prompt-py --day 27
- **Allowed files:** src/lib/data/lessonVisuals/prompt-py/day-27.json, docs/visuals/py_cert_manifest.json
- **Commit message:** [task:C-prompt-py-D27] visuals for prompt-py day 27

Key	Part title
prompt-py:27:0	High-stakes domains
prompt-py:27:1	Disclaimers and scope
prompt-py:27:2	AI in legal work
prompt-py:27:3	AI in healthcare
prompt-py:27:4	AI in marketing and finance
prompt-py:27:5	Practice time: disclaimers and clauses

C-prompt-py-D28 · Day 28: Model Evaluation & Benchmarking: GPT-4o vs Claude 3.5 Sonnet vs Gemini 1.5 Pro

- **Command:** npm run visuals:generate -- --course prompt-py --day 28, then npm run visuals:check -- --course prompt-py --day 28
- **Allowed files:** src/lib/data/lessonVisuals/prompt-py/day-28.json, docs/visuals/py_cert_manifest.json

- **Commit message:** [task:C-prompt-py-D28] visuals for prompt-py day 28

Key	Part title
prompt-py:28:0	Why evaluate
prompt-py:28:1	Scoring and leaderboards
prompt-py:28:2	Comparing open-ended answers
prompt-py:28:3	Quality, cost and speed
prompt-py:28:4	Evaluation pitfalls
prompt-py:28:5	Practice time: leaderboard and win rates

C-prompt-py-D29 · Day 29: Continuous Learning & Open-Source LLMs: Ollama, Llama 3 & Future AI Trends

- **Command:** `npm run visuals:generate -- --course prompt-py --day 29`, then `npm run visuals:check -- --course prompt-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-prompt-py-D29] visuals for prompt-py day 29

Key	Part title
prompt-py:29:0	Open-weight models
prompt-py:29:1	Running models with Ollama
prompt-py:29:2	Memory and hardware
prompt-py:29:3	Quantisation
prompt-py:29:4	Keeping up with AI
prompt-py:29:5	Practice time: fit and quantise

C-prompt-py-D30 · Day 30: 🏆 FINAL CAPSTONE: Sovereign Everyday AI Literacy & Master Prompt Engineering Suite

- **Command:** `npm run visuals:generate -- --course prompt-py --day 30`, then `npm run visuals:check -- --course prompt-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/prompt-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-prompt-py-D30] visuals for prompt-py day 30

Key	Part title
prompt-py:30:0	What makes a great prompt
prompt-py:30:1	Safety in the linter
prompt-py:30:2	Certificate readiness
prompt-py:30:3	Your AI toolkit
prompt-py:30:4	Using AI responsibly, every day
prompt-py:30:5	Capstone practice: linter and readiness

R-prompt-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course prompt-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-prompt-py-26-30] screenshots for prompt-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-prompt-py · Release: students can see AI Prompt Engineering Python (Month 9) pictures

- Add `'prompt-py'` to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-prompt-py] release visuals for prompt-py`.

M-9m · Milestone: the 9m plan has visuals in every course

- **The owner** opens 3 lessons of each newly released course on a phone and on a laptop, and signs off.
- **Antigravity** sends the latest green CI link with the status table.

Course 10: Model Training Python (Month 10) (`train-py`)

C-train-py-D01 · Day 1: Why Models Need Many GPUs: Memory and Compute Arithmetic

- **Command:** `npm run visuals:generate -- --course train-py --day 1`, then `npm run visuals:check -- --course train-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D01] visuals for train-py day 1`

Key	Part title
<code>train-py:1:0</code>	Parameters take up memory
<code>train-py:1:1</code>	How much compute training takes
<code>train-py:1:2</code>	From FLOPs to days
<code>train-py:1:3</code>	Why one GPU is not enough
<code>train-py:1:4</code>	Validating inputs like a professional
<code>train-py:1:5</code>	Practice time: memory and time

C-train-py-D02 · Day 2: Training From Scratch: Gradient Descent on a Tiny Model

- **Command:** `npm run visuals:generate -- --course train-py --day 2`, then `npm run visuals:check -- --course train-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D02] visuals for train-py day 2`

Key	Part title
train-py:2:0	The training loop
train-py:2:1	A tiny model and its loss
train-py:2:2	Gradients by hand
train-py:2:3	Gradient descent in action
train-py:2:4	Choosing the learning rate
train-py:2:5	Practice time: gradients and fitting

C-train-py-D03 · Day 3: Mini-Batches, Batch Size and Learning Rate Scaling

- **Command:** `npm run visuals:generate -- --course train-py --day 3`, then `npm run visuals:check -- --course train-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D03] visuals for train-py day 3`

Key	Part title
train-py:3:0	Why mini-batches
train-py:3:1	Making batches
train-py:3:2	Batch size and gradient noise
train-py:3:3	Learning rate scaling rules
train-py:3:4	Mini-batch training loop
train-py:3:5	Practice time: batches and scaling

C-train-py-D04 · Day 4: Data Parallelism: Sharding Data and Averaging Gradients

- **Command:** `npm run visuals:generate -- --course train-py --day 4`, then `npm run visuals:check -- --course train-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D04] visuals for train-py day 4`

Key	Part title
train-py:4:0	The idea of data parallelism
train-py:4:1	The distributed sampler
train-py:4:2	Averaging gradients
train-py:4:3	Same result as one big batch
train-py:4:4	Where data parallelism struggles
train-py:4:5	Practice time: sharding and averaging

C-train-py-D05 · Day 5: Collective Communication: All-Reduce, Broadcast and Ring All-Reduce

- **Command:** `npm run visuals:generate -- --course train-py --day 5`, then `npm run visuals:check -- --course train-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D05] visuals for train-py day 5`

Key	Part title
train-py:5:0	Collective operations
train-py:5:1	All-reduce
train-py:5:2	The naive approach and its bottleneck
train-py:5:3	Ring all-reduce
train-py:5:4	Choosing and using collectives
train-py:5:5	Practice time: collectives

R-train-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course train-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-train-py-1-5] screenshots for train-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-train-py-D06 · Day 6: Gradient Accumulation: Big Batches on Small Hardware

- **Command:** `npm run visuals:generate -- --course train-py --day 6`, then `npm run visuals:check -- --course train-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D06] visuals for train-py day 6`

Key	Part title
train-py:6:0	Micro-batches and the memory limit
train-py:6:1	Accumulating correctly
train-py:6:2	The effective batch size
train-py:6:3	Fewer all-reduces
train-py:6:4	A full accumulation loop
train-py:6:5	Practice time: accumulation

C-train-py-D07 · Day 7: Mixed Precision: FP16, BF16 and Loss Scaling

- **Command:** `npm run visuals:generate -- --course train-py --day 7`, then `npm run visuals:check -- --course train-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D07] visuals for train-py day 7`

Key	Part title
train-py:7:0	Floating-point formats
train-py:7:1	Overflow and underflow
train-py:7:2	Loss scaling
train-py:7:3	Dynamic loss scaling
train-py:7:4	Mixed precision in practice
train-py:7:5	Practice time: precision

C-train-py-D08 · Day 8: Memory Accounting: Parameters, Gradients, Optimizer States and Activations

- **Command:** `npm run visuals:generate -- --course train-py --day 8`, then `npm run visuals:check -- --course train-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D08] visuals for train-py day 8`

Key	Part title
train-py:8:0	Where training memory goes
train-py:8:1	Comparing setups
train-py:8:2	Activation memory
train-py:8:3	Does it fit?
train-py:8:4	Reading memory reports
train-py:8:5	Practice time: memory accounting

C-train-py-D09 · Day 9: ZeRO Stages 1-3: Sharding Optimizer States, Gradients and Parameters

- **Command:** `npm run visuals:generate -- --course train-py --day 9`, then `npm run visuals:check -- --course train-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D09] visuals for train-py day 9`

Key	Part title
train-py:9:0	The redundancy problem
train-py:9:1	Stage 1: shard the optimizer states
train-py:9:2	Stages 2 and 3
train-py:9:3	How big can we go?
train-py:9:4	Communication and offloading
train-py:9:5	Practice time: ZeRO

C-train-py-D10 · Day 10: Fully Sharded Data Parallel (FSDP): Gather, Compute, Free

- **Command:** `npm run visuals:generate -- --course train-py --day 10`, then `npm run visuals:check -- --course train-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D10] visuals for train-py day 10`

Key	Part title
train-py:10:0	FSDP in one picture
train-py:10:1	The schedule layer by layer
train-py:10:2	Flattening and sharding
train-py:10:3	Peak memory with FSDP
train-py:10:4	Using FSDP well
train-py:10:5	Practice time: FSDP

R-train-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course train-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-train-py-6-10] screenshots for train-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-train-py-D11 · Day 11: Tensor Parallelism: Splitting Matrix Multiplications

- **Command:** `npm run visuals:generate -- --course train-py --day 11`, then `npm run visuals:check -- --course train-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D11] visuals for train-py day 11`

Key	Part title
train-py:11:0	Matrix multiplication recap
train-py:11:1	Column parallelism
train-py:11:2	Row parallelism
train-py:11:3	The Megatron MLP pattern
train-py:11:4	Where tensor parallelism fits
train-py:11:5	Practice time: splitting layers

C-train-py-D12 · Day 12: Pipeline Parallelism: Micro-Batches and the Pipeline Bubble

- **Command:** `npm run visuals:generate -- --course train-py --day 12`, then `npm run visuals:check -- --course train-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-12.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-train-py-D12] visuals for train-py day 12

Key	Part title
train-py:12:0	Stages and the naive pipeline
train-py:12:1	Micro-batches and the GPipe schedule
train-py:12:2	The bubble fraction
train-py:12:3	1F1B and memory
train-py:12:4	Balancing stages
train-py:12:5	Practice time: pipelines

C-train-py-D13 · Day 13: Activation Checkpointing: Trading Compute for Memory

- **Command:** `npm run visuals:generate -- --course train-py --day 13`, then `npm run visuals:check -- --course train-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-train-py-D13] visuals for train-py day 13

Key	Part title
train-py:13:0	Why activations are stored
train-py:13:1	Checkpointing every k layers
train-py:13:2	The square-root rule
train-py:13:3	The compute cost
train-py:13:4	Putting the memory tools together
train-py:13:5	Practice time: checkpointing

C-train-py-D14 · Day 14: Optimizers From Scratch: SGD, Momentum and Adam

- **Command:** `npm run visuals:generate -- --course train-py --day 14`, then `npm run visuals:check -- --course train-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-train-py-D14] visuals for train-py day 14

Key	Part title
train-py:14:0	Plain SGD and its limits
train-py:14:1	Momentum
train-py:14:2	Adam
train-py:14:3	Optimizers and memory
train-py:14:4	Optimizers in distributed training
train-py:14:5	Practice time: optimizers

C-train-py-D15 · Day 15: Learning Rate Schedules: Warmup and Cosine Decay

- **Command:** `npm run visuals:generate -- --course train-py --day 15`, then `npm run visuals:check -- --course train-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D15] visuals for train-py day 15`

Key	Part title
train-py:15:0	Why change the learning rate
train-py:15:1	Linear warmup
train-py:15:2	Cosine decay
train-py:15:3	Step decay
train-py:15:4	Schedules in practice
train-py:15:5	Practice time: schedules

R-train-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course train-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-train-py-11-15] screenshots for train-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-train-py-D16 · Day 16: Gradient Clipping and Training Stability

- **Command:** `npm run visuals:generate -- --course train-py --day 16`, then `npm run visuals:check -- --course train-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D16] visuals for train-py day 16`

Key	Part title
train-py:16:0	Exploding gradients
train-py:16:1	The global gradient norm
train-py:16:2	Clipping by global norm
train-py:16:3	Detecting spikes and NaNs
train-py:16:4	Responding to instability
train-py:16:5	Practice time: stability

C-train-py-D17 · Day 17: Checkpoints: Saving, Sharding and Resuming Training

- **Command:** `npm run visuals:generate -- --course train-py --day 17`, then `npm run visuals:check -- --course train-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D17] visuals for train-py day 17`

Key	Part title
train-py:17:0	What a checkpoint contains
train-py:17:1	Saving and loading safely
train-py:17:2	Sharded checkpoints
train-py:17:3	How often to checkpoint
train-py:17:4	Resuming exactly
train-py:17:5	Practice time: checkpoints

C-train-py-D18 · Day 18: Fault Tolerance and Elastic Training

- **Command:** `npm run visuals:generate -- --course train-py --day 18`, then `npm run visuals:check -- --course train-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D18] visuals for train-py day 18`

Key	Part title
train-py:18:0	Failures at scale
train-py:18:1	The cost of a failure
train-py:18:2	Elastic training
train-py:18:3	Preemption and cloud capacity
train-py:18:4	Designing for failure
train-py:18:5	Practice time: resilience

C-train-py-D19 · Day 19: Data Loading at Scale: Sharding and Deterministic Shuffling

- **Command:** `npm run visuals:generate -- --course train-py --day 19`, then `npm run visuals:check -- --course train-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D19] visuals for train-py day 19`

Key	Part title
train-py:19:0	Data at scale
train-py:19:1	Deterministic shuffling
train-py:19:2	Balancing shards across workers
train-py:19:3	Avoiding input bottlenecks
train-py:19:4	Resuming the data stream
train-py:19:5	Practice time: data pipelines

C-train-py-D20 · Day 20: Communication Cost Models: Latency, Bandwidth and Topology

- **Command:** `npm run visuals:generate -- --course train-py --day 20`, then `npm run visuals:check -- --course train-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D20] visuals for train-py day 20`

Key	Part title
train-py:20:0	The latency-bandwidth model
train-py:20:1	Ring all-reduce time
train-py:20:2	Network topology
train-py:20:3	Is communication the bottleneck?
train-py:20:4	Reducing communication
train-py:20:5	Practice time: communication cost

R-train-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course train-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-train-py-16-20] screenshots for train-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-train-py-D21 · Day 21: Overlapping Compute and Communication: Gradient Bucketing

- **Command:** `npm run visuals:generate -- --course train-py --day 21`, then `npm run visuals:check -- --course train-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D21] visuals for train-py day 21`

Key	Part title
train-py:21:0	Gradients become ready in reverse
train-py:21:1	Gradient buckets
train-py:21:2	Simulating overlap
train-py:21:3	When overlap is not enough
train-py:21:4	Overlap in real frameworks
train-py:21:5	Practice time: overlap

C-train-py-D22 · Day 22: Scaling Laws and Compute-Optimal Training

- **Command:** `npm run visuals:generate -- --course train-py --day 22`, then `npm run visuals:check -- --course train-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-22.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-train-py-D22] visuals for train-py day 22

Key	Part title
train-py:22:0	Scaling laws
train-py:22:1	Chinchilla: balancing parameters and data
train-py:22:2	Beyond compute-optimal
train-py:22:3	From FLOPs to a budget
train-py:22:4	Using scaling laws responsibly
train-py:22:5	Practice time: budgets

C-train-py-D23 · Day 23: Measuring Throughput: Tokens per Second and MFU

- **Command:** `npm run visuals:generate -- --course train-py --day 23`, then `npm run visuals:check -- --course train-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-train-py-D23] visuals for train-py day 23

Key	Part title
train-py:23:0	Throughput
train-py:23:1	Model FLOPs utilisation
train-py:23:2	Step time statistics
train-py:23:3	Where the missing FLOPs go
train-py:23:4	Reporting performance honestly
train-py:23:5	Practice time: measuring speed

C-train-py-D24 · Day 24: Profiling and Finding Bottlenecks

- **Command:** `npm run visuals:generate -- --course train-py --day 24`, then `npm run visuals:check -- --course train-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-train-py-D24] visuals for train-py day 24

Key	Part title
train-py:24:0	Profiling a step
train-py:24:1	Three kinds of bottleneck
train-py:24:2	Fixes for each bottleneck
train-py:24:3	Stragglers and variation
train-py:24:4	A profiling workflow
train-py:24:5	Practice time: bottlenecks

C-train-py-D25 · Day 25: Mixture of Experts: Routing and Load Balancing

- **Command:** `npm run visuals:generate -- --course train-py --day 25`, then `npm run visuals:check -- --course train-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D25] visuals for train-py day 25`

Key	Part title
train-py:25:0	Sparse models
train-py:25:1	Top-k routing
train-py:25:2	Expert capacity
train-py:25:3	Load balancing
train-py:25:4	Expert parallelism
train-py:25:5	Practice time: experts

R-train-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course train-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-train-py-21-25] screenshots for train-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-train-py-D26 · Day 26: Parameter-Efficient Fine-Tuning: LoRA

- **Command:** `npm run visuals:generate -- --course train-py --day 26`, then `npm run visuals:check -- --course train-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D26] visuals for train-py day 26`

Key	Part title
train-py:26:0	Why full fine-tuning is expensive
train-py:26:1	The low-rank idea
train-py:26:2	The LoRA forward pass
train-py:26:3	Merging adapters
train-py:26:4	QLoRA and distributed fine-tuning
train-py:26:5	Practice time: LoRA

C-train-py-D27 · Day 27: Quantization: INT8 and 4-bit Weights

- **Command:** `npm run visuals:generate -- --course train-py --day 27`, then `npm run visuals:check -- --course train-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D27] visuals for train-py day 27`

Key	Part title
train-py:27:0	What quantisation does
train-py:27:1	Symmetric INT8 quantisation
train-py:27:2	Measuring quantisation error
train-py:27:3	4-bit quantisation
train-py:27:4	Quantisation in training and serving
train-py:27:5	Practice time: quantisation

C-train-py-D28 · Day 28: Knowledge Distillation: Teaching Small Models

- **Command:** `npm run visuals:generate -- --course train-py --day 28`, then `npm run visuals:check -- --course train-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D28] visuals for train-py day 28`

Key	Part title
train-py:28:0	Teachers and students
train-py:28:1	Softmax with temperature
train-py:28:2	The distillation loss
train-py:28:3	Distillation in practice
train-py:28:4	Choosing a small-model strategy
train-py:28:5	Practice time: distillation

C-train-py-D29 · Day 29: Cluster Scheduling and Training Cost

- **Command:** `npm run visuals:generate -- --course train-py --day 29`, then `npm run visuals:check -- --course train-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D29] visuals for train-py day 29`

Key	Part title
train-py:29:0	Clusters and schedulers
train-py:29:1	Placing jobs well
train-py:29:2	The full cost of a run
train-py:29:3	Utilisation of shared clusters
train-py:29:4	Planning the capacity you need
train-py:29:5	Practice time: clusters and cost

C-train-py-D30 · Day 30: 🏆 Capstone: Planning and Reporting a Distributed Training Run

- **Command:** `npm run visuals:generate -- --course train-py --day 30`, then `npm run visuals:check -- --course train-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/train-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-train-py-D30] visuals for train-py day 30`

Key	Part title
train-py:30:0	Choosing a strategy that fits
train-py:30:1	The complete plan
train-py:30:2	Reporting a training run
train-py:30:3	Everything you can now calculate
train-py:30:4	Where to go next
train-py:30:5	Capstone practice: plan and report

R-train-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course train-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-train-py-26-30] screenshots for train-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-train-py · Release: students can see Model Training Python (Month 10) pictures

- Add `'train-py'` to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-train-py] release visuals for train-py`.

Course 11: Vector Search Python (Month 11) (`vec-py`)

C-vec-py-D01 · Day 1: Embeddings: Turning Text into Vectors

- **Command:** `npm run visuals:generate -- --course vec-py --day 1`, then `npm run visuals:check -- --course vec-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D01] visuals for vec-py day 1`

Key	Part title
vec-py:1:0	Why vectors
vec-py:1:1	Bag-of-words vectors
vec-py:1:2	The hashing trick
vec-py:1:3	Unit-length vectors
vec-py:1:4	Learned embeddings
vec-py:1:5	Practice time: making vectors

C-vec-py-D02 · Day 2: Similarity Measures: Dot Product, Cosine and Euclidean Distance

- **Command:** `npm run visuals:generate -- --course vec-py --day 2`, then `npm run visuals:check -- --course vec-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D02] visuals for vec-py day 2`

Key	Part title
vec-py:2:0	The dot product
vec-py:2:1	Cosine similarity
vec-py:2:2	Euclidean distance
vec-py:2:3	Ranking by any metric
vec-py:2:4	Similarity in practice
vec-py:2:5	Practice time: similarity

C-vec-py-D03 · Day 3: Brute-Force k-Nearest-Neighbour Search

- **Command:** `npm run visuals:generate -- --course vec-py --day 3`, then `npm run visuals:check -- --course vec-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D03] visuals for vec-py day 3`

Key	Part title
vec-py:3:0	k-nearest neighbours
vec-py:3:1	Normalise once, dot many times
vec-py:3:2	Matrix form
vec-py:3:3	Why exact search stops scaling
vec-py:3:4	A tiny search service
vec-py:3:5	Practice time: exact search

C-vec-py-D04 · Day 4: Top-k Selection with Heaps

- **Command:** `npm run visuals:generate -- --course vec-py --day 4`, then `npm run visuals:check -- --course vec-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D04] visuals for vec-py day 4`

Key	Part title
vec-py:4:0	Why not just sort?
vec-py:4:1	Heaps in Python

Key	Part title
vec-py:4:2	Tie-breaking and determinism
vec-py:4:3	Merging shard results
vec-py:4:4	Top-k in real systems
vec-py:4:5	Practice time: top-k

C-vec-py-D05 · Day 5: Measuring Retrieval Quality: Recall@k, Precision and MRR

- **Command:** `npm run visuals:generate -- --course vec-py --day 5`, then `npm run visuals:check -- --course vec-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D05] visuals for vec-py day 5`

Key	Part title
vec-py:5:0	Relevance judgements
vec-py:5:1	Recall and precision at k
vec-py:5:2	Mean reciprocal rank
vec-py:5:3	Recall against exact search
vec-py:5:4	Reading the numbers
vec-py:5:5	Practice time: metrics

R-vec-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course vec-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-vec-py-1-5] screenshots for vec-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-vec-py-D06 · Day 6: Sparse Retrieval: TF-IDF and BM25

- **Command:** `npm run visuals:generate -- --course vec-py --day 6`, then `npm run visuals:check -- --course vec-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D06] visuals for vec-py day 6`

Key	Part title
vec-py:6:0	The inverted index
vec-py:6:1	TF-IDF
vec-py:6:2	BM25
vec-py:6:3	Length normalisation
vec-py:6:4	When keywords beat vectors

Key	Part title
vec-py:6:5	Practice time: keyword search

C-vec-py-D07 · Day 7: Hybrid Search: Reciprocal Rank Fusion and Weighted Scores

- **Command:** `npm run visuals:generate -- --course vec-py --day 7`, then `npm run visuals:check -- --course vec-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D07] visuals for vec-py day 7`

Key	Part title
vec-py:7:0	Why hybrid search
vec-py:7:1	Reciprocal rank fusion
vec-py:7:2	Normalised score fusion
vec-py:7:3	Choosing a fusion method
vec-py:7:4	Hybrid search in a pipeline
vec-py:7:5	Practice time: fusion

C-vec-py-D08 · Day 8: Chunking Documents for Retrieval

- **Command:** `npm run visuals:generate -- --course vec-py --day 8`, then `npm run visuals:check -- --course vec-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D08] visuals for vec-py day 8`

Key	Part title
vec-py:8:0	Why chunk
vec-py:8:1	Fixed-size chunks with overlap
vec-py:8:2	Sentence-aware chunks
vec-py:8:3	Choosing a chunk size
vec-py:8:4	Chunk metadata
vec-py:8:5	Practice time: chunking

C-vec-py-D09 · Day 9: Metadata Filtering: Pre-filtering and Post-filtering

- **Command:** `npm run visuals:generate -- --course vec-py --day 9`, then `npm run visuals:check -- --course vec-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D09] visuals for vec-py day 9`

Key	Part title
vec-py:9:0	Filters in search
vec-py:9:1	Pre-filtering
vec-py:9:2	Post-filtering and its pitfall
vec-py:9:3	Filter selectivity
vec-py:9:4	Richer filters
vec-py:9:5	Practice time: filtering

C-vec-py-D10 · Day 10: k-Means Clustering from Scratch

- **Command:** `npm run visuals:generate -- --course vec-py --day 10`, then `npm run visuals:check -- --course vec-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D10] visuals for vec-py day 10`

Key	Part title
vec-py:10:0	The idea of clustering
vec-py:10:1	Assigning points
vec-py:10:2	The k-means loop
vec-py:10:3	Choosing k and training data
vec-py:10:4	Evaluating a clustering
vec-py:10:5	Practice time: clustering

R-vec-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course vec-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-vec-py-6-10] screenshots for vec-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-vec-py-D11 · Day 11: Inverted File Indexes (IVF) and nprobe

- **Command:** `npm run visuals:generate -- --course vec-py --day 11`, then `npm run visuals:check -- --course vec-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D11] visuals for vec-py day 11`

Key	Part title
vec-py:11:0	From clusters to inverted lists
vec-py:11:1	Searching with nprobe
vec-py:11:2	Measuring the trade-off

Key	Part title
vec-py:11:3	Cost model and list balance
vec-py:11:4	IVF in real systems
vec-py:11:5	Practice time: IVF

C-vec-py-D12 · Day 12: Product Quantization

- **Command:** `npm run visuals:generate -- --course vec-py --day 12`, then `npm run visuals:check -- --course vec-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D12] visuals for vec-py day 12`

Key	Part title
vec-py:12:0	The memory problem
vec-py:12:1	Sub-vectors and codebooks
vec-py:12:2	Encoding
vec-py:12:3	Asymmetric distance computation
vec-py:12:4	Accuracy and tuning
vec-py:12:5	Practice time: product quantisation

C-vec-py-D13 · Day 13: Scalar and Binary Quantization with Hamming Distance

- **Command:** `npm run visuals:generate -- --course vec-py --day 13`, then `npm run visuals:check -- --course vec-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D13] visuals for vec-py day 13`

Key	Part title
vec-py:13:0	Scalar quantisation
vec-py:13:1	Binary quantisation
vec-py:13:2	Hamming distance
vec-py:13:3	Re-scoring
vec-py:13:4	Choosing a compression method
vec-py:13:5	Practice time: compact codes

C-vec-py-D14 · Day 14: Locality-Sensitive Hashing with Random Hyperplanes

- **Command:** `npm run visuals:generate -- --course vec-py --day 14`, then `npm run visuals:check -- --course vec-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-14.json`, `docs/visuals/py_cert_manifest.json`

- **Commit message:** [task:C-vec-py-D14] visuals for vec-py day 14

Key	Part title
vec-py:14:0	Locality-sensitive hashing
vec-py:14:1	Random hyperplanes
vec-py:14:2	Several hash tables
vec-py:14:3	Tuning LSH
vec-py:14:4	LSH in practice
vec-py:14:5	Practice time: hashing

C-vec-py-D15 · Day 15: Graph Search: Greedy Search on a Proximity Graph

- **Command:** `npm run visuals:generate -- --course vec-py --day 15`, then `npm run visuals:check -- --course vec-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-vec-py-D15] visuals for vec-py day 15

Key	Part title
vec-py:15:0	Proximity graphs
vec-py:15:1	Greedy search
vec-py:15:2	Local minima
vec-py:15:3	Beam search with ef
vec-py:15:4	Cost and recall
vec-py:15:5	Practice time: graph search

R-vec-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course vec-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** [task:R-vec-py-11-15] screenshots for vec-py days 11-15
- **Wait** for Claude's "approved" before the next day task.

C-vec-py-D16 · Day 16: HNSW: Layers, Entry Points and Neighbour Selection

- **Command:** `npm run visuals:generate -- --course vec-py --day 16`, then `npm run visuals:check -- --course vec-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** [task:C-vec-py-D16] visuals for vec-py day 16

Key	Part title
vec-py:16:0	Layers like a skip list
vec-py:16:1	Random levels

Key	Part title
vec-py:16:2	Neighbour selection
vec-py:16:3	Parameters: M, efConstruction and efSearch
vec-py:16:4	Searching all the layers
vec-py:16:5	Practice time: HNSW pieces

C-vec-py-D17 · Day 17: Tuning Indexes: Recall versus Latency

- **Command:** `npm run visuals:generate -- --course vec-py --day 17`, then `npm run visuals:check -- --course vec-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D17] visuals for vec-py day 17`

Key	Part title
vec-py:17:0	Benchmarking a configuration
vec-py:17:1	Dominated configurations
vec-py:17:2	Choosing with requirements
vec-py:17:3	Tuning in practice
vec-py:17:4	Beyond recall and latency
vec-py:17:5	Practice time: tuning

C-vec-py-D18 · Day 18: Vector Database Operations: Upserts, Deletes and Tombstones

- **Command:** `npm run visuals:generate -- --course vec-py --day 18`, then `npm run visuals:check -- --course vec-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D18] visuals for vec-py day 18`

Key	Part title
vec-py:18:0	Upserts and versions
vec-py:18:1	Deletes and tombstones
vec-py:18:2	Searching around tombstones
vec-py:18:3	Segments and compaction
vec-py:18:4	Consistency for readers
vec-py:18:5	Practice time: operations

C-vec-py-D19 · Day 19: Sharding and Replication for Vector Search

- **Command:** `npm run visuals:generate -- --course vec-py --day 19`, then `npm run visuals:check -- --course vec-py --day 19`

- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D19] visuals for vec-py day 19`

Key	Part title
vec-py:19:0	Why shard
vec-py:19:1	Hash-based placement
vec-py:19:2	Scatter-gather search
vec-py:19:3	Replication
vec-py:19:4	Operating a sharded cluster
vec-py:19:5	Practice time: shards

C-vec-py-D20 · Day 20: Multi-Tenancy, Namespaces and Access Control

- **Command:** `npm run visuals:generate -- --course vec-py --day 20`, then `npm run visuals:check -- --course vec-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D20] visuals for vec-py day 20`

Key	Part title
vec-py:20:0	Tenants and namespaces
vec-py:20:1	Access control lists
vec-py:20:2	Tenant-safe search
vec-py:20:3	Leaks through AI answers
vec-py:20:4	Testing isolation
vec-py:20:5	Practice time: access control

R-vec-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course vec-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-vec-py-16-20] screenshots for vec-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-vec-py-D21 · Day 21: Re-ranking and Diversity with Maximal Marginal Relevance

- **Command:** `npm run visuals:generate -- --course vec-py --day 21`, then `npm run visuals:check -- --course vec-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D21] visuals for vec-py day 21`

Key	Part title
vec-py:21:0	Two-stage retrieval
vec-py:21:1	A simple re-ranker
vec-py:21:2	The redundancy problem
vec-py:21:3	Maximal marginal relevance
vec-py:21:4	Putting the pipeline together
vec-py:21:5	Practice time: re-ranking

C-vec-py-D22 · Day 22: Query Rewriting and Expansion

- **Command:** `npm run visuals:generate -- --course vec-py --day 22`, then `npm run visuals:check -- --course vec-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D22] visuals for vec-py day 22`

Key	Part title
vec-py:22:0	Why queries need help
vec-py:22:1	Normalising queries
vec-py:22:2	Synonym expansion
vec-py:22:3	Searching with several variants
vec-py:22:4	Learning from query logs
vec-py:22:5	Practice time: queries

C-vec-py-D23 · Day 23: Retrieval-Augmented Generation: Context Assembly and Citations

- **Command:** `npm run visuals:generate -- --course vec-py --day 23`, then `npm run visuals:check -- --course vec-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D23] visuals for vec-py day 23`

Key	Part title
vec-py:23:0	Retrieval-augmented generation
vec-py:23:1	Token budgets
vec-py:23:2	Citations
vec-py:23:3	Grounded prompts
vec-py:23:4	Evaluating RAG answers
vec-py:23:5	Practice time: context and citations

C-vec-py-D24 · Day 24: Semantic Caching

- **Command:** `npm run visuals:generate -- --course vec-py --day 24`, then `npm run visuals:check -- --course vec-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D24] visuals for vec-py day 24`

Key	Part title
vec-py:24:0	Why cache semantically
vec-py:24:1	Building the cache
vec-py:24:2	Choosing the threshold
vec-py:24:3	Measuring savings
vec-py:24:4	Freshness and safety
vec-py:24:5	Practice time: caching

C-vec-py-D25 · Day 25: Freshness, Versioning and Embedding Model Migrations

- **Command:** `npm run visuals:generate -- --course vec-py --day 25`, then `npm run visuals:check -- --course vec-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D25] visuals for vec-py day 25`

Key	Part title
vec-py:25:0	Keeping indexes fresh
vec-py:25:1	Why models do not mix
vec-py:25:2	Blue-green index migration
vec-py:25:3	Migration costs
vec-py:25:4	Versioning everything
vec-py:25:5	Practice time: freshness and migration

R-vec-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course vec-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-vec-py-21-25] screenshots for vec-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-vec-py-D26 · Day 26: Evaluating Ranking with nDCG and Golden Sets

- **Command:** `npm run visuals:generate -- --course vec-py --day 26`, then `npm run visuals:check -- --course vec-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D26] visuals for vec-py day 26`

Key	Part title
vec-py:26:0	Graded relevance
vec-py:26:1	DCG and nDCG
vec-py:26:2	Evaluating a system on a golden set
vec-py:26:3	Comparing systems fairly
vec-py:26:4	Building a golden set that lasts
vec-py:26:5	Practice time: evaluation

C-vec-py-D27 · Day 27: Monitoring Vector Search: Latency Percentiles and Drift

- **Command:** `npm run visuals:generate -- --course vec-py --day 27`, then `npm run visuals:check -- --course vec-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D27] visuals for vec-py day 27`

Key	Part title
vec-py:27:0	Why averages hide slow queries
vec-py:27:1	Computing percentiles
vec-py:27:2	Embedding drift
vec-py:27:3	Quality signals in production
vec-py:27:4	Alerts that help
vec-py:27:5	Practice time: monitoring

C-vec-py-D28 · Day 28: Capacity Planning: Memory, Dimensions and Cost

- **Command:** `npm run visuals:generate -- --course vec-py --day 28`, then `npm run visuals:check -- --course vec-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D28] visuals for vec-py day 28`

Key	Part title
vec-py:28:0	Bytes per vector
vec-py:28:1	Graph index overhead
vec-py:28:2	Choosing a format for a budget
vec-py:28:3	From memory to cost
vec-py:28:4	Planning for growth
vec-py:28:5	Practice time: capacity

C-vec-py-D29 · Day 29: Privacy and Deletion in Vector Stores

- **Command:** `npm run visuals:generate -- --course vec-py --day 29`, then `npm run visuals:check -- --course vec-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D29] visuals for vec-py day 29`

Key	Part title
vec-py:29:0	Personal data in vector search
vec-py:29:1	Redacting before embedding
vec-py:29:2	Deleting a user's data
vec-py:29:3	Retention and minimisation
vec-py:29:4	Security threats specific to RAG
vec-py:29:5	Practice time: privacy

C-vec-py-D30 · Day 30: 🏆 Capstone: A Hybrid Search Engine with Filters and an Evaluation Report

- **Command:** `npm run visuals:generate -- --course vec-py --day 30`, then `npm run visuals:check -- --course vec-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/vec-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-vec-py-D30] visuals for vec-py day 30`

Key	Part title
vec-py:30:0	The capstone design
vec-py:30:1	Hybrid scoring
vec-py:30:2	Evaluating the engine
vec-py:30:3	Writing the report
vec-py:30:4	What you have built
vec-py:30:5	Capstone practice: engine and report

R-vec-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course vec-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-vec-py-26-30] screenshots for vec-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-vec-py · Release: students can see Vector Search Python (Month 11) pictures

- Add `'vec-py'` to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-vec-py] release visuals for vec-py`.

Course 12: AI Safety Python (Month 12) (`safe-py`)

C-safe-py-D01 · Day 1: What AI Safety Means in Production: Harms, Risks and a Risk Register

- **Command:** `npm run visuals:generate -- --course safe-py --day 1`, then `npm run visuals:check -- --course safe-py --day 1`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-01.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D01] visuals for safe-py day 1`

Key	Part title
safe-py:1:0	Safety is an engineering job
safe-py:1:1	Kinds of harm
safe-py:1:2	Likelihood times impact
safe-py:1:3	The risk register
safe-py:1:4	Controls and residual risk
safe-py:1:5	Where this course goes

C-safe-py-D02 · Day 2: Threat Modelling LLM Applications with the OWASP LLM Top 10

- **Command:** `npm run visuals:generate -- --course safe-py --day 2`, then `npm run visuals:check -- --course safe-py --day 2`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-02.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D02] visuals for safe-py day 2`

Key	Part title
safe-py:2:0	What threat modelling is
safe-py:2:1	The OWASP Top 10 for LLM Applications
safe-py:2:2	Tagging findings with keywords
safe-py:2:3	Controls for each category
safe-py:2:4	Finding gaps
safe-py:2:5	Threat modelling in practice

C-safe-py-D03 · Day 3: Input Validation: Length Limits, Control Characters and Token Budgets

- **Command:** `npm run visuals:generate -- --course safe-py --day 3`, then `npm run visuals:check -- --course safe-py --day 3`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-03.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D03] visuals for safe-py day 3`

Key	Part title
safe-py:3:0	Why validate input
safe-py:3:1	Checking rules in order
safe-py:3:2	Control characters
safe-py:3:3	Tokens and budgets
safe-py:3:4	Truncating to a budget
safe-py:3:5	Validation in a pipeline

C-safe-py-D04 · Day 4: Detecting Direct Prompt Injection

- **Command:** `npm run visuals:generate -- --course safe-py --day 4`, then `npm run visuals:check -- --course safe-py --day 4`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-04.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D04] visuals for safe-py day 4`

Key	Part title
safe-py:4:0	What prompt injection is
safe-py:4:1	Regular expressions for attack phrases
safe-py:4:2	Scoring a message
safe-py:4:3	Allow, review or block
safe-py:4:4	Limits of pattern matching
safe-py:4:5	Putting the detector in the pipeline

C-safe-py-D05 · Day 5: Indirect Prompt Injection and Untrusted Content

- **Command:** `npm run visuals:generate -- --course safe-py --day 5`, then `npm run visuals:check -- --course safe-py --day 5`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-05.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D05] visuals for safe-py day 5`

Key	Part title
safe-py:5:0	Indirect prompt injection
safe-py:5:1	Marking content as data
safe-py:5:2	Escaping so tags cannot be closed
safe-py:5:3	Invisible characters
safe-py:5:4	Stripping hidden characters
safe-py:5:5	Defending against indirect injection

R-safe-py-1-5 · Checkpoint: Claude reviews days 1-5

- **Command:** `npm run visuals:pick -- --course safe-py --days 1-5`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-safe-py-1-5] screenshots for safe-py days 1-5`
- **Wait** for Claude's "approved" before the next day task.

C-safe-py-D06 · Day 6: Validating Structured Outputs Against a Schema

- **Command:** `npm run visuals:generate -- --course safe-py --day 6`, then `npm run visuals:check -- --course safe-py --day 6`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-06.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D06] visuals for safe-py day 6`

Key	Part title
safe-py:6:0	Model output is untrusted
safe-py:6:1	Parsing JSON safely
safe-py:6:2	Extracting JSON from chatter
safe-py:6:3	Checking a schema
safe-py:6:4	The boolean trap
safe-py:6:5	Output validation in the pipeline

C-safe-py-D07 · Day 7: Protecting Payment Data: Luhn Checks and Card Masking

- **Command:** `npm run visuals:generate -- --course safe-py --day 7`, then `npm run visuals:check -- --course safe-py --day 7`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-07.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D07] visuals for safe-py day 7`

Key	Part title
safe-py:7:0	Why card numbers are special
safe-py:7:1	The Luhn check
safe-py:7:2	Cleaning and length rules
safe-py:7:3	Finding card numbers in text
safe-py:7:4	Masking in place
safe-py:7:5	Personal data on both sides

C-safe-py-D08 · Day 8: Content Moderation: Categories, Thresholds and Trade-offs

- **Command:** `npm run visuals:generate -- --course safe-py --day 8`, then `npm run visuals:check -- --course safe-py --day 8`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-08.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D08] visuals for safe-py day 8`

Key	Part title
safe-py:8:0	Moderation classifiers
safe-py:8:1	Per-category thresholds
safe-py:8:2	Four outcomes
safe-py:8:3	The threshold trade-off
safe-py:8:4	Moderation in the pipeline
safe-py:8:5	Where moderation goes wrong

C-safe-py-D09 · Day 9: Measuring Classifiers: Confusion Matrices, Precision and Recall

- **Command:** `npm run visuals:generate -- --course safe-py --day 9`, then `npm run visuals:check -- --course safe-py --day 9`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-09.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D09] visuals for safe-py day 9`

Key	Part title
safe-py:9:0	The confusion matrix
safe-py:9:1	Precision
safe-py:9:2	Recall
safe-py:9:3	F1: balancing both
safe-py:9:4	Base rates and accuracy
safe-py:9:5	Evaluating a safety check

C-safe-py-D10 · Day 10: Hallucination Checks: Groundedness Against Sources

- **Command:** `npm run visuals:generate -- --course safe-py --day 10`, then `npm run visuals:check -- --course safe-py --day 10`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-10.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D10] visuals for safe-py day 10`

Key	Part title
safe-py:10:0	What hallucination is
safe-py:10:1	Content words
safe-py:10:2	Scoring support
safe-py:10:3	Numbers are where it hurts
safe-py:10:4	Stronger groundedness checks
safe-py:10:5	Groundedness in the pipeline

R-safe-py-6-10 · Checkpoint: Claude reviews days 6-10

- **Command:** `npm run visuals:pick -- --course safe-py --days 6-10`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-safe-py-6-10] screenshots for safe-py days 6-10`
- **Wait** for Claude's "approved" before the next day task.

C-safe-py-D11 · Day 11: Verifying Citations

- **Command:** `npm run visuals:generate -- --course safe-py --day 11`, then `npm run visuals:check -- --course safe-py --day 11`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-11.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D11] visuals for safe-py day 11`

Key	Part title
safe-py:11:0	Why citations can mislead
safe-py:11:1	Does the cited source support the sentence?
safe-py:11:2	Finding bad citations
safe-py:11:3	Which sentences need a citation?
safe-py:11:4	Measuring citation coverage
safe-py:11:5	Citations in the pipeline

C-safe-py-D12 · Day 12: Balancing Refusal and Helpfulness

- **Command:** `npm run visuals:generate -- --course safe-py --day 12`, then `npm run visuals:check -- --course safe-py --day 12`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-12.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D12] visuals for safe-py day 12`

Key	Part title
safe-py:12:0	Two ways to get refusal wrong
safe-py:12:1	Detecting refusals
safe-py:12:2	Over- and under-refusal rates
safe-py:12:3	Balancing the two
safe-py:12:4	Refusing well
safe-py:12:5	Refusal testing in the pipeline

C-safe-py-D13 · Day 13: Safe Tool Use: Allowlists and Argument Validation

- **Command:** `npm run visuals:generate -- --course safe-py --day 13`, then `npm run visuals:check -- --course safe-py --day 13`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-13.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D13] visuals for safe-py day 13`

Key	Part title
safe-py:13:0	From text to actions
safe-py:13:1	A tool registry
safe-py:13:2	Validating a tool call
safe-py:13:3	Path traversal
safe-py:13:4	A safe join
safe-py:13:5	Tool safety in the pipeline

C-safe-py-D14 · Day 14: Agent Permissions and Human Approval

- **Command:** `npm run visuals:generate -- --course safe-py --day 14`, then `npm run visuals:check -- --course safe-py --day 14`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-14.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D14] visuals for safe-py day 14`

Key	Part title
safe-py:14:0	Human in the loop
safe-py:14:1	Approval rules as data
safe-py:14:2	Agent plans
safe-py:14:3	Running a plan safely
safe-py:14:4	Making approval meaningful
safe-py:14:5	Approval in the pipeline

C-safe-py-D15 · Day 15: Rate Limiting and Abuse Detection

- **Command:** `npm run visuals:generate -- --course safe-py --day 15`, then `npm run visuals:check -- --course safe-py --day 15`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-15.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D15] visuals for safe-py day 15`

Key	Part title
safe-py:15:0	Why limit usage
safe-py:15:1	Fixed windows and their flaw
safe-py:15:2	A sliding-window limiter
safe-py:15:3	Per-user limits
safe-py:15:4	Detecting abuse
safe-py:15:5	Limits and abuse in the pipeline

R-safe-py-11-15 · Checkpoint: Claude reviews days 11-15

- **Command:** `npm run visuals:pick -- --course safe-py --days 11-15`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-safe-py-11-15] screenshots for safe-py days 11-15`
- **Wait** for Claude's "approved" before the next day task.

C-safe-py-D16 · Day 16: Red-Teaming: Attack Suites and Attack Success Rate

- **Command:** `npm run visuals:generate -- --course safe-py --day 16`, then `npm run visuals:check -- --course safe-py --day 16`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-16.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D16] visuals for safe-py day 16`

Key	Part title
safe-py:16:0	What red-teaming is
safe-py:16:1	Attack categories
safe-py:16:2	Attack success rate
safe-py:16:3	Small samples are unreliable
safe-py:16:4	Automated red-teaming
safe-py:16:5	Red-teaming in the pipeline

C-safe-py-D17 · Day 17: Robustness to Obfuscation: Normalising Text

- **Command:** `npm run visuals:generate -- --course safe-py --day 17`, then `npm run visuals:check -- --course safe-py --day 17`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-17.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D17] visuals for safe-py day 17`

Key	Part title
safe-py:17:0	How attackers disguise text
safe-py:17:1	Unicode normalisation
safe-py:17:2	Undoing leetspeak
safe-py:17:3	Removing spacing tricks
safe-py:17:4	Finding banned words
safe-py:17:5	Normalisation in the pipeline

C-safe-py-D18 · Day 18: Fairness Metrics: Demographic Parity and Equal Opportunity

- **Command:** `npm run visuals:generate -- --course safe-py --day 18`, then `npm run visuals:check -- --course safe-py --day 18`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-18.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D18] visuals for safe-py day 18`

Key	Part title
safe-py:18:0	Why fairness is a safety issue
safe-py:18:1	Demographic parity
safe-py:18:2	Equal opportunity
safe-py:18:3	Fairness measures can conflict
safe-py:18:4	Fairness for language models
safe-py:18:5	Fairness checks in the pipeline

C-safe-py-D19 · Day 19: Calibration: Confidence and Expected Calibration Error

- **Command:** `npm run visuals:generate -- --course safe-py --day 19`, then `npm run visuals:check -- --course safe-py --day 19`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-19.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D19] visuals for safe-py day 19`

Key	Part title
safe-py:19:0	What calibration means
safe-py:19:1	Confidence bins
safe-py:19:2	Expected calibration error
safe-py:19:3	Reliability tables
safe-py:19:4	Fixing calibration
safe-py:19:5	Calibration in the pipeline

C-safe-py-D20 · Day 20: Abstention: Answering Only When Confident

- **Command:** `npm run visuals:generate -- --course safe-py --day 20`, then `npm run visuals:check -- --course safe-py --day 20`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-20.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D20] visuals for safe-py day 20`

Key	Part title
safe-py:20:0	Knowing when not to answer
safe-py:20:1	Coverage and selective accuracy
safe-py:20:2	The risk-coverage curve
safe-py:20:3	Choosing a threshold for a target
safe-py:20:4	What to do instead of answering
safe-py:20:5	Abstention in the pipeline

R-safe-py-16-20 · Checkpoint: Claude reviews days 16-20

- **Command:** `npm run visuals:pick -- --course safe-py --days 16-20`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-safe-py-16-20] screenshots for safe-py days 16-20`
- **Wait** for Claude's "approved" before the next day task.

C-safe-py-D21 · Day 21: Evaluation Harnesses and Regression Gates

- **Command:** `npm run visuals:generate -- --course safe-py --day 21`, then `npm run visuals:check -- --course safe-py --day 21`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-21.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D21] visuals for safe-py day 21`

Key	Part title
safe-py:21:0	Why an evaluation harness
safe-py:21:1	Test cases with expected decisions
safe-py:21:2	Running the suite
safe-py:21:3	Regression gates
safe-py:21:4	What to measure in the gate
safe-py:21:5	The harness in the pipeline

C-safe-py-D22 · Day 22: Audit Logging Without Leaking Data

- **Command:** `npm run visuals:generate -- --course safe-py --day 22`, then `npm run visuals:check -- --course safe-py --day 22`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-22.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D22] visuals for safe-py day 22`

Key	Part title
safe-py:22:0	Why audit logs
safe-py:22:1	Hashing identifiers
safe-py:22:2	A privacy-safe audit entry
safe-py:22:3	What else to log
safe-py:22:4	Summarising the log
safe-py:22:5	Audit logs in the pipeline

C-safe-py-D23 · Day 23: Incident Response for AI Systems

- **Command:** `npm run visuals:generate -- --course safe-py --day 23`, then `npm run visuals:check -- --course safe-py --day 23`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-23.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D23] visuals for safe-py day 23`

Key	Part title
safe-py:23:0	What an AI incident looks like
safe-py:23:1	Severity levels
safe-py:23:2	Containment first
safe-py:23:3	Measuring response times
safe-py:23:4	Communicating during an incident
safe-py:23:5	Blameless post-mortems

C-safe-py-D24 · Day 24: Model Cards and System Documentation

- **Command:** `npm run visuals:generate -- --course safe-py --day 24`, then `npm run visuals:check -- --course safe-py --day 24`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-24.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D24] visuals for safe-py day 24`

Key	Part title
safe-py:24:0	Why document AI systems
safe-py:24:1	What goes in each section
safe-py:24:2	Checking completeness
safe-py:24:3	Rendering Markdown
safe-py:24:4	Keeping cards up to date
safe-py:24:5	Cards in the pipeline

C-safe-py-D25 · Day 25: Governance: EU AI Act Risk Tiers and the NIST AI RMF

- **Command:** `npm run visuals:generate -- --course safe-py --day 25`, then `npm run visuals:check -- --course safe-py --day 25`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-25.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D25] visuals for safe-py day 25`

Key	Part title
safe-py:25:0	Why governance matters
safe-py:25:1	The EU AI Act risk tiers
safe-py:25:2	Triage by keywords
safe-py:25:3	The NIST AI Risk Management Framework
safe-py:25:4	Checking framework coverage
safe-py:25:5	Governance in the pipeline

R-safe-py-21-25 · Checkpoint: Claude reviews days 21-25

- **Command:** `npm run visuals:pick -- --course safe-py --days 21-25`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-safe-py-21-25] screenshots for safe-py days 21-25`
- **Wait** for Claude's "approved" before the next day task.

C-safe-py-D26 · Day 26: Training Data Hygiene: Deduplication and Licence Filtering

- **Command:** `npm run visuals:generate -- --course safe-py --day 26`, then `npm run visuals:check -- --course safe-py --day 26`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-26.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D26] visuals for safe-py day 26`

Key	Part title
safe-py:26:0	Data is where behaviour comes from
safe-py:26:1	Why duplicates matter
safe-py:26:2	Normalised deduplication
safe-py:26:3	Licences and data rights
safe-py:26:4	Filtering by licence
safe-py:26:5	Data hygiene in the pipeline

C-safe-py-D27 · Day 27: Watermarking and Detecting AI-Generated Text

- **Command:** `npm run visuals:generate -- --course safe-py --day 27`, then `npm run visuals:check -- --course safe-py --day 27`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-27.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D27] visuals for safe-py day 27`

Key	Part title
safe-py:27:0	Why mark AI-generated text
safe-py:27:1	The green-list idea
safe-py:27:2	Deciding green with a keyed hash
safe-py:27:3	Detecting with a z-score
safe-py:27:4	Limits of watermarking
safe-py:27:5	Watermarking in the pipeline

C-safe-py-D28 · Day 28: Preference Data and Reward Models

- **Command:** `npm run visuals:generate -- --course safe-py --day 28`, then `npm run visuals:check -- --course safe-py --day 28`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-28.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D28] visuals for safe-py day 28`

Key	Part title
safe-py:28:0	Learning from preferences
safe-py:28:1	The Bradley-Terry model
safe-py:28:2	The pairwise loss
safe-py:28:3	Fitting ratings from comparisons
safe-py:28:4	Reward hacking
safe-py:28:5	Preferences in the pipeline

C-safe-py-D29 · Day 29: Monitoring Safety in Production

- **Command:** `npm run visuals:generate -- --course safe-py --day 29`, then `npm run visuals:check -- --course safe-py --day 29`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-29.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D29] visuals for safe-py day 29`

Key	Part title
safe-py:29:0	Why monitor in production
safe-py:29:1	A decision dashboard
safe-py:29:2	Baselines
safe-py:29:3	Spike alerts
safe-py:29:4	Alerts people trust
safe-py:29:5	Monitoring in the pipeline

C-safe-py-D30 · Day 30: 🏆 Capstone: A Guardrail Pipeline and Safety Scorecard

- **Command:** `npm run visuals:generate -- --course safe-py --day 30`, then `npm run visuals:check -- --course safe-py --day 30`
- **Allowed files:** `src/lib/data/lessonVisuals/safe-py/day-30.json`, `docs/visuals/py_cert_manifest.json`
- **Commit message:** `[task:C-safe-py-D30] visuals for safe-py day 30`

Key	Part title
safe-py:30:0	Looking back
safe-py:30:1	Collecting reasons, not stopping early
safe-py:30:2	From reasons to a decision
safe-py:30:3	The full guardrail
safe-py:30:4	The safety scorecard
safe-py:30:5	Your safety pipeline

R-safe-py-26-30 · Checkpoint: Claude reviews days 26-30

- **Command:** `npm run visuals:pick -- --course safe-py --days 26-30`
- **Allowed files:** `docs/visuals/shot_keys.txt`
- **Commit message:** `[task:R-safe-py-26-30] screenshots for safe-py days 26-30`
- **Wait** for Claude's "approved" before the next day task.

L-safe-py · Release: students can see AI Safety Python (Month 12) pictures

- Add 'safe-py' to `src/lib/visuals/enabledCourses.ts`. Allowed file: that one only. Commit `[task:L-safe-py] release visuals for safe-py`.

M-12m · Milestone: the 12m plan has visuals in every course

- **The owner** opens 3 lessons of each newly released course on a phone and on a laptop, and signs off.
- **Antigravity** sends the latest green CI link with the status table.